

UNIVERSITY OF SCIENCE
ADVANCED PROGRAM IN COMPUTER SCIENCE

LE QUOC HUY
NGUYEN VINH KHANG

AGENTIC AI-BASED VIETNAMESE
FINANCIAL MARKET DATA ANALYSIS
AND SUMMARIZATION SYSTEM

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

HO CHI MINH CITY, 2026

UNIVERSITY OF SCIENCE

ADVANCED PROGRAM IN COMPUTER SCIENCE

LE QUOC HUY – 22125031

NGUYEN VINH KHANG – 22125035

**AGENTIC AI-BASED VIETNAMESE
FINANCIAL MARKET DATA ANALYSIS
AND SUMMARIZATION SYSTEM**

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

THESIS ADVISOR

Dr. Nguyen Thi Minh Tuyen

HO CHI MINH CITY, 2026

ACKNOWLEDGMENTS

We would like to express our deepest and most sincere gratitude to our thesis advisor, Dr. Nguyen Thi Minh Tuyen, for her invaluable guidance, insightful feedback, and unwavering support throughout every stage of this project. From the earliest formulation of the research problem to the final refinement of the system and thesis manuscript, her thoughtful comments were instrumental in shaping the direction and quality of this work.

We would also like to extend our sincere thanks to Ms. Trinh, Securities Analyst at FPT Securities Joint Stock Company, for her valuable feedback and practical insights, which greatly contributed to the refinement and completion of our product.

07/2026, Ho Chi Minh City
Le Quoc Huy & Nguyen Vinh Khang

TABLE OF CONTENTS

ACKNOWLEDGMENTS	i
TABLE OF CONTENTS	ii
LIST OF FIGURES	vi
LIST OF TABLES	vii
ABSTRACT	viii
Chapter 1 Introduction	1
1.1 Motivation	1
1.2 Application Objectives	2
1.3 Application Scope	3
1.3.1 Target Users and Interfaces	3
1.3.2 Functional Scope	3
1.3.3 Market and Data Scope	3
1.3.4 Evaluation Scope	4
1.4 Thesis Outline	4
Chapter 2 Related Work and Foundation Technologies	5
2.1 Related Work	5
2.1.1 LLM-Based Agent Systems for Financial Decision Support	5
2.1.2 Finance-Specific Large Language Models	5
2.1.3 Financial News Sentiment Analysis	6
2.1.4 Prediction Research for the Vietnamese Stock Market	6
2.1.5 Research Gap and Positioning of Stockrium	7
2.2 Foundation Technologies	8
2.2.1 Financial Analysis Foundations	8
2.2.2 Large Language Models	9
2.2.3 AI Agents, Design Patterns, and Agentic Workflows	10
2.2.4 Hybrid LLM–Rule Decision Support	10
2.2.5 Backtesting and Evaluation Foundations	11

2.3	Development Technologies	11
2.3.1	Frontend Technologies	12
2.3.2	Backend, Authentication, and Security Technologies.....	12
2.3.3	Data Acquisition, Processing, and Storage Technologies ...	13
2.3.4	AI, Agent Orchestration, and State Management Technologies	14
2.3.5	Deployment and Development Tools	14
Chapter 3	Proposed Application	15
3.1	Application Idea	15
3.1.1	Stakeholders	15
3.1.2	Use Case Model.....	16
3.1.3	Main Application Flows.....	18
3.2	Key Features	23
3.2.1	Authentication and Account Management	24
3.2.2	Market Overview and Stock Discovery	24
3.2.3	Stock Details and Interactive Technical Charts	25
3.2.4	Financial News Exploration	26
3.2.5	Investment-Horizon Profile and Configurable AI Analysis .	27
3.2.6	Conversational Financial Assistant	29
3.2.7	Administrative Analysis and Backtesting	30
Chapter 4	System Design and Implementation	33
4.1	System Architecture Overview	33
4.1.1	Architectural Requirements	33
4.1.2	Logical Architecture	34
4.1.3	End-to-End Request Flows	36
4.2	Mobile Application Design.....	36
4.2.1	Navigation Structure.....	37
4.2.2	Screen and Component Organization.....	39
4.2.3	Financial Data Visualization	40
4.2.4	API Integration and Client State.....	43
4.3	Backend Service Architecture	46
4.3.1	FastAPI Layered Design	46
4.3.2	Domain API Modules	48
4.3.3	Request Validation and Response Contract.....	51
4.3.4	Backend Lifecycle, Concurrency, and Failure Containment.	53
4.4	Data Architecture and Acquisition Pipelines	54
4.4.1	Supabase Data Model.....	54
4.4.2	Market Price Acquisition Pipeline	56

4.4.3	Company and Fundamental Data Pipeline	58
4.4.4	Financial News Pipeline.....	60
4.4.5	Storage Responsibilities	62
4.5	Agentic Stock Analysis Design	64
4.5.1	Graph Topology and Shared State	64
4.5.2	Orchestration and Horizon-Aware Planning	66
4.5.3	Specialist Evidence Agents	67
4.5.4	Aggregation and Structured Recommendation.....	70
4.5.5	Deterministic Confidence and Decision Controls.....	71
4.6	Backtesting Framework	73
4.6.1	Evaluation Architecture.....	74
4.6.2	Technical Signal Construction.....	74
4.6.3	Trade Simulation and Cost Model.....	75
4.6.4	Measures, Benchmarks, and Artifacts	75
Chapter 5	System Testing	77
5.1	Testing Objectives and Scope	77
5.2	Testing Methods, Tools, and Environment	78
5.2.1	Testing Methods and Tools.....	78
5.2.2	Test Environment and Data.....	78
5.3	Functional and Black-Box Testing	78
5.4	Integration and API Testing	80
5.5	Non-Functional Testing.....	81
5.5.1	Performance and Reliability.....	81
5.5.2	Security Testing.....	81
5.5.3	Usability and User Acceptance Testing.....	82
5.6	Compatibility and Device Testing.....	82
5.7	Overall Results and Discussion	83
5.8	Chapter Summary	84
Chapter 6	Experiments and Evaluation.....	85
6.1	Evaluation Methodology	85
6.1.1	Evidence Sources	85
6.1.2	Measures and Comparisons	86
6.2	Historical Backtesting Evaluation	86
6.2.1	VN30 Experimental Configuration	86
6.2.2	Full-Pipeline and Baseline Results.....	87
6.2.3	Confidence, Regime, and Walk-Forward Results	88
6.2.4	Single-Indicator Ablation	89

6.3	Discussion and Threats to Validity.....	89
6.3.1	Answers to the Evaluation Questions.....	89
6.3.2	Practical Efficiency for Financial Analysis.....	90
6.3.3	Validity Limitations.....	91
6.3.4	Overall Evaluation.....	93
Chapter 7	Conclusion and Future Work.....	94
7.1	Conclusion.....	94
7.2	Achievement of the Project Objectives.....	95
7.3	Principal Contributions.....	96
7.4	Remaining Limitations.....	97
7.5	Future Work.....	98
7.5.1	Point-in-Time Data and Dataset Publication.....	98
7.5.2	More Realistic and Reproducible Evaluation.....	98
7.5.3	Analytical and Agentic Improvements.....	98
7.5.4	System Quality and User Experience.....	99
7.6	Closing Remarks.....	99
References		100

LIST OF FIGURES

3.1	Stockrium actors and principal use cases	17
3.2	Mobile decision-support flow: account access, stock discovery, and analysis configuration	19
3.3	Mobile decision-support flow: analysis processing, recommendation, and follow-up	21
3.4	Administrative analysis and backtesting flow	23
3.5	Authentication and Account Management	24
3.6	Market Overview and Stock Discovery	25
3.7	Stock Details and Interactive Technical Charts	26
3.8	Financial News Explorations	27
3.9	Initial input for Agents	28
3.10	Agents' analysis results	29
3.11	Conversational Financial Assistant	30
3.12	Admin Backtest Screen	31
3.13	Admin Backtest Result Screen	31
3.14	Admin Analyze Screen	32
3.15	Admin Analyzer Result Screen	32
4.1	Overall logical architecture of Stockrium	34
4.2	Principal navigation structure of the Stockrium mobile application	38
4.3	Coordinated access-token refresh in the mobile HTTP client	44
4.4	Layered design of the Stockrium backend service	47
4.5	Reduced ERD of the principal Stockrium data relationships	56
4.6	Historical, live, and post-session market-data acquisition	58
4.7	Offline company and fundamental-data preparation pipeline	60
4.8	Initial and weekly financial-news processing flow	62
4.9	Execution topology of the Stockrium analysis graph	65
4.10	Execution flow of the Stockrium backtesting framework	74

LIST OF TABLES

3.1	Principal stakeholders of Stockrium	16
3.2	Use-case groups by actor	18
4.1	Organization of the mobile application source code	39
4.2	Principal financial visualizations in the mobile application	41
4.3	Client-state scopes in the Stockrium mobile application	45
4.4	Active backend API modules and responsibilities: core data services	49
4.5	Active backend API modules and responsibilities: user and operational services	50
4.6	Standard API response envelope	52
4.7	Principal relational-table groups in Stockrium	55
4.8	Storage responsibility boundaries	63
4.9	Principal fields in the analysis graph state	66
4.10	Responsibilities and outputs of specialist analysis nodes	68
4.11	Deterministic positive-signal conditions in the technical agent	69
4.12	Default confidence weights by investment horizon	72
4.13	Implemented horizon-specific trading-plan boundaries	73
5.1	Scope of Stockrium system testing	77
5.2	Summary of functional and black-box test cases	79
5.3	Summary of integration and API test cases	80
5.4	Observed response-time results	81
5.5	Summary of security test cases	82
5.6	Summary of compatibility and device test cases	83
5.7	Overall system-testing results	83
6.1	Full agentic pipeline compared with the engine-only baseline	87
6.2	Pooled confidence-tier outcomes in the saved pipeline report	88
6.3	VN30 single-indicator ablation results, 2023–2025	89
6.4	Principal evaluation limitations and recommended improvements	91
7.1	Achievement of the Stockrium project objectives	95

ABSTRACT

Vietnamese retail investors must often combine fragmented market prices, company fundamentals, technical indicators, and financial news before making an informed investment decision. This process requires both financial knowledge and substantial time, while existing tools commonly present only one analytical perspective or provide recommendations whose supporting evidence is difficult to inspect. This thesis presents *Stockrium*, an integrated Vietnamese equity-market decision-support and research platform designed to organize heterogeneous financial information and produce explainable, user-configurable analysis.

Stockrium consists of a React Native/Expo mobile application, a FastAPI backend, an administrative dashboard, data-acquisition pipelines, and a LangGraph-based multi-agent workflow. The platform supports market exploration, stock search, interactive price and technical-indicator charts, company and fundamental information, financial news, watchlists, investment-horizon preferences, AI-assisted analysis, and conversational follow-up. Technical, fundamental, and article specialists analyze their respective evidence domains in parallel. An aggregator combines their outputs into a structured recommendation containing component scores, confidence, explanations, and risk-related information, while deterministic validation and decision controls constrain unsupported outputs. Crawling and scheduled collection processes acquire and normalize market, company, financial-statement, and news data for application use and historical evaluation.

System testing covered functional behavior, API and external-service integration, security, reliability, performance, usability, and Android/iOS compatibility. The tested workflows produced their expected results, and a complete agentic analysis was generated within seconds, demonstrating the platform’s potential to reduce the time required for preliminary financial research and synthesis. Historical evaluation was conducted on 30 large-cap Vietnamese securities using engine-only, buy-and-hold, random-entry, and single-indicator comparisons. The agentic layer reduced executed trades by 41.6% and lowered mean maximum drawdown from 25.69% to 16.66%, but it also achieved lower mean return than the engine-only baseline and outperformed buy-and-hold for only 4 of 30 securities. Statistical and robustness evidence remained limited. Therefore, Stockrium’s principal contribution is an integrated, inspectable, and time-efficient environment for Vietnamese-market decision support and experimentation, rather than evidence of guaranteed profitability or a replacement for professional judgment.

Keywords: Vietnamese stock market, decision support, agentic AI, multi-agent systems, financial analysis, financial news, backtesting.

Chapter 1 Introduction

This chapter describes the motivation and significance of the research, states the objectives of the study, and highlights the expected outcomes. Furthermore, it provides an overview of the thesis organization.

1.1 Motivation

The rapid growth of the Vietnamese stock market has made securities investment accessible to a broader population. By the end of 2025, domestic investors held approximately 11.9 million securities trading accounts [1], while retail investors contributed a substantial proportion of market trading activity [2]. However, greater access to trading platforms does not necessarily provide investors with the analytical capability required to make informed and disciplined decisions.

Evaluating a stock requires the combined consideration of price trends and technical indicators, corporate fundamentals, and financial news. In practice, these data are fragmented across different platforms and presented in forms that can be difficult for inexperienced investors to collect and interpret. Limited financial knowledge, attention constraints, behavioral biases, and unverified opinions from online communities can further encourage speculative decisions [3]. Therefore, investors need an integrated tool that organizes relevant evidence and explains analytical results clearly, while system administrators need a controlled way to evaluate the analytical pipeline against historical data.

Large language models offer useful capabilities for summarizing financial documents, interpreting news, and communicating complex information [4, 5]. Nevertheless, using a general-purpose model alone in a financial context may produce inconsistent or unsupported conclusions. This motivates a hybrid multi-agent approach in which specialized agents separately analyze technical, fundamental, and news evidence, while deterministic computations calculate indicators, aggregate scores, and enforce decision constraints.

Accordingly, this thesis develops *Stockrium*, an agentic artificial intelligence (AI)-based decision-support system for the Vietnamese equity market. Its mobile application integrates multiple analytical perspectives and generates explainable recommendations according to the user’s selected investment horizon, while its internal backtesting environment allows administrators to evaluate the analytical pipeline historically. *Stockrium* is not intended to replace investors, execute trades, or guarantee profits; its purpose is to reduce information-processing effort and provide a transparent, structured environment

in which Vietnamese retail investors can form and evaluate investment hypotheses more systematically.

1.2 Application Objectives

The overall objective of this thesis is to develop *Stockrium* as a practical and research-oriented platform for the Vietnamese stock market. The project combines an investor-facing application, an internal historical evaluation tool, and a curated dataset. Its three primary objectives are defined as follows.

O1. Develop an intelligent decision-support application for individual investors.

Stockrium aims to help Vietnamese retail investors examine a stock through technical, fundamental, and news-based perspectives. A coordinated multi-agent workflow analyzes these evidence sources and produces an explainable recommendation that considers the user’s selected investment horizon. The mobile application also provides market exploration, configurable analytical components, and natural-language interaction so that users can access and interpret financial information more conveniently. The generated results are intended to support, rather than replace, the investor’s judgment.

O2. Provide an internal backtesting tool for analytical-pipeline evaluation.

The project aims to provide administrators with an internal historical simulation environment for evaluating Stockrium’s analytical pipeline and supported configurations on Vietnamese market data. The tool reports interpretable performance and risk measures, enabling administrators to examine system behavior, compare configurations, and assess whether the pipeline operates consistently with its intended decision rules. It is not a general-purpose strategy builder, and its results are used for evaluation and experimentation rather than as evidence of guaranteed future returns.

O3. Construct a curated, research-ready dataset for the Vietnamese market.

Stockrium aims to curate a consistently structured dataset covering all three analytical lenses used by the system: historical price and volume data, company fundamentals, and financial news with sentiment labels. The dataset supports application development, backtesting, and reproducible research on the Vietnamese market. All three data groups are planned for public release after completion of this thesis to provide a reusable foundation for subsequent studies and system development.

Together, these objectives position Stockrium as both a usable financial decision-support system and a research platform for integrated Vietnamese market analysis. The

boundaries within which they are pursued are specified in the following section.

1.3 Application Scope

1.3.1 Target Users and Interfaces

Stockrium primarily serves Vietnamese retail investors. The mobile application is intended for both beginners who need accessible financial information and experienced individual investors who wish to configure analytical inputs and examine the resulting assessments. The administrative web interface is intended only for authorized administrators, who manage data, run analysis over selected stock groups, evaluate Stockrium’s analytical pipeline and supported configurations through internal backtesting, and review stored results; it is not a public investment interface.

1.3.2 Functional Scope

The functional scope of the mobile application includes authentication, market exploration, stock search, company information, price charts, technical indicators, fundamental data, financial news, a personal watchlist, and natural-language interaction. Its intelligent-analysis function combines technical, fundamental, and news evidence through specialized agents to generate an explainable recommendation and confidence score. For an accepted *Buy* recommendation, the result may additionally include entry, take-profit, stop-loss, and holding-period information. Users may also configure selected analytical components and their relative weights. Backtesting is restricted to the administrative web interface and evaluates the Stockrium pipeline rather than arbitrary user-defined trading strategies. Stockrium is limited to decision support: it does not place, modify, or cancel buy or sell orders; connect to brokerage firms or other financial institutions; transfer funds; automatically execute recommendations; or perform portfolio construction, optimization, and rebalancing. It is not intended for high-frequency trading or professional financial advisory services.

1.3.3 Market and Data Scope

The data scope is limited to the Vietnamese equity market, including stocks listed on HOSE, HNX, and UPCoM and representative domestic market indices. The system processes current and historical prices, trading volumes, company profiles, financial statements and ratios, technical indicators, and Vietnamese financial news, subject to the availability and quality of the integrated sources. Foreign equities, bonds, commodities, foreign exchange, cryptocurrencies, and derivatives are outside the scope of this thesis.

1.3.4 Evaluation Scope

The system is evaluated using historical Vietnamese market data to examine the behavior, explainability, risk controls, and performance of the analytical pipeline. Backtesting reports measures such as trade count, win rate, return, Sharpe ratio, and maximum drawdown. Because the evaluation is a simplified historical simulation, it does not reproduce every condition of live execution, including order-book depth, latency, and partial fills. Its results are used to assess the implemented approach and do not guarantee future investment performance.

1.4 Thesis Outline

This thesis is organized into seven chapters as follows.

Chapter 1 – Introduction presents the motivation for developing Stockrium and the problem addressed by the thesis. It defines the project’s primary objectives, application scope, and overall thesis structure.

Chapter 2 – Related Work and Foundation Technologies reviews studies on financial decision support, large language models, multi-agent systems, news sentiment, and Vietnamese stock-market prediction. It then introduces the financial, agentic AI, backtesting, and development technologies required to understand Stockrium.

Chapter 3 – Proposed Application describes the application idea, target stakeholders, principal use cases, and user flow. It also presents the key mobile, analytical, conversational, administrative, and backtesting capabilities of the proposed system.

Chapter 4 – System Design and Implementation explains the overall architecture and the design of the mobile application, backend services, storage, and data-acquisition pipelines. It also details the agentic stock-analysis workflow, the integration of its specialized analytical components, and the internal backtesting framework.

Chapter 5 – System Testing defines the testing objectives, environment, and verification strategy for the implemented system. It reports functional and integration testing of the application, backend services, authentication mechanisms, and AI workflows.

Chapter 6 – Experiments and Evaluation defines the evaluation questions and evaluates the analytical pipeline on historical VN30 data against explicit baselines. It assesses robustness and statistical significance, reports qualitative feedback from a securities analyst after hands-on experience with the application, and discusses threats to validity.

Chapter 7 – Conclusion and Future Work summarizes the research outcomes, principal contributions, and degree to which the stated objectives were achieved. It discusses the remaining limitations and proposes directions for extending the data, analytical methods, system, and evaluation.

Chapter 2 Related Work and Foundation Technologies

This chapter reviews research related to multi-agent financial systems, finance-specific language models, financial-news sentiment analysis, and stock-market prediction in Vietnam. It then introduces the principal concepts and technologies that form the foundation of Stockrium.

2.1 Related Work

Research related to Stockrium lies at the intersection of financial decision support, large language models, multi-agent systems, financial-news sentiment analysis, and Vietnamese stock-market prediction. This section reviews representative studies in these areas and identifies the gap addressed by the proposed application.

2.1.1 LLM-Based Agent Systems for Financial Decision Support

LLM-based agent systems organize financial reasoning through specialized roles, memory structures, or both. TradingAgents is a representative multi-agent framework that models a trading firm through analyst agents, Bull and Bear researchers, a trader, and a risk-management team [6]. The authors report that this structured collaboration combines heterogeneous evidence and improves selected return and risk measures. Its multi-round debates nevertheless require repeated model calls and can therefore increase latency and inference cost, while the final action remains strongly dependent on the language model’s reasoning path. FinMem instead equips a single trading agent with layered memory that retrieves financial information according to temporal relevance [7], highlighting the value of maintaining context across changing market conditions, although its main objective remains autonomous trading performance. Stockrium adopts role specialization but uses a smaller graph designed for Vietnamese equities: technical, fundamental, and news agents operate in parallel, after which their evidence is synthesized. Deterministic scoring and validation are added around the LLM output to improve traceability and constrain the final recommendation. Thus, Stockrium is positioned as a user-facing decision-support system rather than an autonomous trading agent or a debate-based simulation of a financial institution.

2.1.2 Finance-Specific Large Language Models

Finance-specific language models show that domain knowledge can improve the interpretation of specialized terminology, disclosures, and market communication. BloombergGPT

is a 50-billion-parameter model trained on both financial and general-purpose corpora; its evaluation indicates that domain-focused pretraining improves performance on financial NLP tasks while retaining general capabilities [4]. FinGPT provides a more accessible, open-source alternative centered on automated financial-data curation and lightweight model adaptation for tasks such as sentiment analysis, robo-advising, and algorithmic trading [5]. These studies establish the usefulness of LLMs for extracting information, understanding financial text, and generating explanations. Nevertheless, a financial language model alone does not constitute a complete decision-support application: it does not necessarily retrieve synchronized market and company data, reconcile multiple analytical perspectives, apply user configurations, enforce numerical risk constraints, or evaluate strategies historically. The cited models are also not specifically designed for Vietnamese financial language or local market conventions. Consequently, Stockrium uses general LLM capabilities within a controlled application workflow, while structured data processing and deterministic rules remain responsible for calculations and output validation.

2.1.3 Financial News Sentiment Analysis

Financial news may contain events and expectations that are not fully reflected in historical prices, but its specialized vocabulary limits the effectiveness of general sentiment models. FinBERT addresses this problem through domain-specific pretraining and fine-tuning, achieving improved positive, neutral, and negative classification on financial datasets [8]. FinBERT-LSTM subsequently combined news sentiment with historical prices for NASDAQ-100 prediction, showing that textual information can supplement numerical time series [9]; however, it remains a price-prediction pipeline and does not reconcile news with fundamental and technical evidence. Vietnamese studies provide more locally relevant findings. Tung et al. reported that PhoBERT outperformed conventional models on Vietnamese stock-article titles, reaching approximately 93% accuracy [10]. Using a year-long collection from three Vietnamese financial-news platforms, Vu et al. achieved over 81% accuracy in binary positive–negative sentiment classification [11]. Their event study found significant changes in the return variance of the VN30 and HNX30 indices around negative news over the examined 30-day windows, but no statistically significant changes in mean returns. These results suggest that sentiment can provide useful contextual and risk information but should not independently determine an investment action. Stockrium therefore summarizes and assesses stock-related news as one analytical source whose output is combined with technical and fundamental evidence.

2.1.4 Prediction Research for the Vietnamese Stock Market

Vietnamese stock-market research has primarily used historical numerical data to forecast index or stock movements. Truong et al. compared LSTM variants for VN-Index

prediction and found bidirectional LSTM effective across the examined horizons [12]. Phuoc et al. incorporated moving averages, MACD, and RSI into an LSTM-based model for the VN-Index and VN30 stocks, reporting values around or above 93% for most examined series under the study’s price-error-based accuracy measure [13]. Son et al. later compared multilayer perceptron, LSTM, bidirectional LSTM, and Transformer models and obtained horizon-dependent results: Transformer performed better for short-term forecasting, whereas bidirectional LSTM was stronger over medium and longer horizons [14]. These studies confirm the relevance of machine learning, technical indicators, and investment horizon in the Vietnamese market. Nevertheless, their main evaluation target is forecast error or directional accuracy, which does not directly indicate whether an analytical pipeline controls drawdown, remains effective after transaction costs, or provides understandable evidence for a recommendation. None of these reviewed forecasting studies integrates price behavior with both company fundamentals and Vietnamese financial news. Stockrium extends this research direction by emphasizing integrated and explainable decision support together with internal historical evaluation rather than price prediction alone.

2.1.5 Research Gap and Positioning of Stockrium

The literature reveals a separation among financial AI research streams: forecasting studies focus mainly on prices and technical indicators, sentiment studies classify news, finance-specific LLMs address language understanding, and multi-agent frameworks often target autonomous trading in international markets. Among the reviewed works, none combines technical, fundamental, and Vietnamese news evidence in a user-facing application while also supporting configurable analysis, investment-horizon awareness, explainable recommendations, deterministic decision constraints, and internal backtesting. Stockrium is designed to address this integration gap. It coordinates specialized agents through an explicit workflow, allows users to select analytical inputs and their relative importance, and uses an LLM to interpret and synthesize evidence rather than perform every calculation. Deterministic components calculate indicators and confidence, validate minimum conditions, and restrict unsupported recommendations, while the administrative backtesting facility enables Stockrium’s analytical pipeline and supported configurations to be evaluated on historical Vietnamese data. The contribution is therefore not a new forecasting model or a claim of guaranteed superior returns; it is the construction of a localized, configurable, and transparent decision-support environment that brings previously separate analytical capabilities into a coherent system.

2.2 Foundation Technologies

Stockrium combines financial-analysis methods, large language models, multi-agent coordination, deterministic rules, and historical evaluation. This section briefly introduces the concepts required to understand the system; their implementation is presented in Chapter 4.

2.2.1 Financial Analysis Foundations

Equity analysis commonly considers three complementary sources of evidence: market behavior, company fundamentals, and financial news. Each describes a different aspect of a stock and should therefore be interpreted together rather than treated as an independent guarantee of future performance.

Technical analysis. Technical analysis examines historical price and volume data to describe trend, momentum, volatility, and possible turning points [15]. Stockrium uses several complementary indicators. A simple moving average (SMA) is the arithmetic mean of recent closing prices, whereas an exponential moving average (EMA) assigns greater weight to newer observations; both smooth short-term noise and help reveal trend direction. Bollinger Bands place an upper and lower band around a moving average—typically the 20-period SMA plus or minus two standard deviations—so band width reflects recent volatility and the price’s position within the bands provides context about relative strength or weakness.

The Relative Strength Index (RSI) is a momentum oscillator bounded between 0 and 100. Values above 70 are conventionally treated as overbought and values below 30 as oversold, although these conditions do not by themselves imply an immediate reversal. Moving Average Convergence Divergence (MACD) uses the difference between the 12-period and 26-period EMAs as its main line, a 9-period EMA of that difference as the signal line, and their difference as the histogram. The relative lines and changing histogram indicate whether momentum is strengthening or weakening. KDJ is a stochastic oscillator named after its K, D, and J lines: K measures the closing price’s position within a recent high–low range, D smooths K, and $J = 3K - 2D$ amplifies the separation between them. Values above 80 or below 20 conventionally indicate overbought or oversold momentum. These indicators are lagging or sensitive to noise, so crossings and threshold values are treated as supporting evidence rather than automatic buy or sell instructions.

Fundamental analysis. Fundamental analysis evaluates the financial condition and operating performance of a company using its balance sheet, income statement, cash-flow statement, and derived ratios [16]. Stockrium groups non-financial-company indicators

into liquidity, leverage, efficiency, profitability, and valuation. The current ratio compares current assets with current liabilities and indicates short-term payment capacity. The debt-to-equity ratio compares liabilities with shareholders' equity and describes financial leverage, while asset turnover measures how efficiently assets generate revenue. Return on Assets (ROA) relates profit to total assets, and Return on Equity (ROE) relates profit to shareholders' equity. Under the DuPont decomposition, ROE can be interpreted as net profit margin multiplied by asset turnover and financial leverage, thereby distinguishing profitability, operating efficiency, and debt-supported returns.

Valuation indicators describe the market price relative to an accounting measure. The price-to-earnings ratio (P/E) compares share price with earnings per share, whereas the price-to-book ratio (P/B) compares market value with book value. A lower ratio is not automatically better because growth expectations, business risk, and sector structure also affect valuation. For financial institutions, Stockrium additionally considers the net interest margin (NIM), capital adequacy ratio (CAR), loan-to-deposit ratio (LDR), and non-performing loan ratio (NPL). These respectively compare net interest income with average earning assets, regulatory capital with risk-weighted assets, gross loans with customer deposits, and non-performing loans with gross loans. Thus, they provide views of interest-earning efficiency, capital resilience, funding liquidity, and asset quality. Compound annual growth rate (CAGR), calculated as $(V_{\text{end}}/V_{\text{start}})^{1/n} - 1$ for positive endpoint values separated by n years, summarizes a multi-year trend as a smoothed annual rate. All ratios are interpreted through time trends and industry comparisons because a favorable value in one sector or period may not have the same meaning in another.

Financial news analysis. Financial news provides information about corporate events, macroeconomic changes, and market expectations that may not yet appear in accounting data. Sentiment analysis classifies this content as positive, neutral, or negative, while summarization makes long articles easier to review. However, sentiment depends on the affected company, publication time, source, and market context; Vietnamese studies also show that textual polarity alone is not a reliable trading rule [10, 11]. Stockrium therefore treats news as supporting evidence alongside technical and fundamental information.

2.2.2 Large Language Models

Large language models (LLMs) are neural models trained on extensive text collections to understand and generate natural language. Most modern LLMs are based on the Transformer architecture, which uses attention to represent relationships among words in a context [17]. Their instruction-following capability allows one model to summarize articles, interpret financial observations, classify content, and generate explanations without training a separate model for every task [18]. Nevertheless, LLMs can produce

unsupported or inconsistent statements, and a valid response structure does not guarantee factual or numerical correctness. In Stockrium, LLMs are consequently used mainly for interpretation and explanation, while exact calculations and selected validation rules remain in conventional code.

2.2.3 AI Agents, Design Patterns, and Agentic Workflows

An artificial-intelligence (AI) agent combines a model with a defined role, available information, state, and actions such as calling an application programming interface (API) or returning a structured result. The Reasoning and Acting (ReAct) pattern illustrates how model reasoning can be interleaved with external actions and their observations [19]. A multi-agent system divides a larger task among cooperating agents [20]. This separation allows each Stockrium agent to focus on technical data, fundamentals, or news and exposes intermediate results for later synthesis. The approach improves modularity but can also increase latency and propagate errors, making clear responsibilities and compatible output formats necessary.

Several common patterns organize agent-based applications. A *specialized agent* owns one domain; an *orchestrator-worker* structure assigns work to the appropriate agents; *fan-out/fan-in* executes independent branches in parallel before joining them; and an *aggregator* reconciles their outputs. Shared state provides a common data contract, while conditional routing selects the next node from the current state. Stockrium combines these patterns to make its analytical flow explicit and bounded rather than relying on unrestricted agent-to-agent conversation.

An agentic workflow represents processing as a graph of nodes and transitions. Nodes may retrieve data, perform calculations, invoke an LLM, or update shared state, while edges define sequential, conditional, or parallel execution. LangGraph provides these graph and state abstractions together with checkpointing for persistent conversations [21]. In Stockrium, one graph coordinates the stock-analysis agents and another routes chatbot requests. This structure makes the allowed execution paths visible and allows deterministic functions and language-model nodes to operate within the same workflow.

2.2.4 Hybrid LLM–Rule Decision Support

A hybrid decision-support system assigns qualitative interpretation to LLMs and reproducible calculations to deterministic components. Stockrium’s agents interpret evidence and propose structured assessments, whereas conventional code calculates indicators and a normalized confidence score, checks minimum conditions, and constrains selected recommendation values according to the investment horizon. The confidence score lies between 0 and 1 and is a weighted aggregation of the active technical, fundamental, and news components; it is an internal strength-of-evidence measure, not a statistically

calibrated probability of profit. This arrangement improves consistency and traceability but does not eliminate uncertainty because the inputs and qualitative interpretations may still be incomplete or incorrect. The generated result must therefore be understood as decision support rather than a guaranteed investment outcome.

2.2.5 Backtesting and Evaluation Foundations

Backtesting applies a defined rule set or analytical pipeline to historical data to estimate how it would have behaved under past market conditions. A meaningful simulation specifies signal, entry, exit, holding-period, and transaction-cost assumptions while preventing the use of information that was unavailable at the simulated decision time. In this thesis, total return compounds successive closed-trade returns as $\prod_{i=1}^N (1 + r_i) - 1$ for each security; win rate is the proportion of the N closed trades with a positive net return; and maximum drawdown is the largest peak-to-trough decline in the simulated equity curve. Trade count is reported because percentages derived from very few observations are less reliable.

The Sharpe ratio generally measures excess return per unit of return variability [22]. Stockrium’s implementation uses closed-trade returns, a zero risk-free benchmark, and computes $\sqrt{N} \bar{r} / s_r$, where $\bar{r}$ and s_r are the sample mean and sample standard deviation of trade returns. It returns zero when fewer than two trades are available or when their returns have no variation. The result is therefore a trade-level comparative statistic rather than a conventional annualized daily-return Sharpe ratio. Simple baselines provide reference behavior, market-regime analysis separates results under different broad market trends, and walk-forward evaluation repeats testing over successive time windows instead of relying on one fixed split. Nevertheless, historical results can be distorted by overfitting, survivorship bias, look-ahead bias, incomplete data, and simplified execution assumptions [23]; they describe past behavior and do not prove future profitability. Stockrium’s backtesting facility is consequently an internal evaluation tool for its pipeline and supported configurations, not a general-purpose strategy-construction platform.

Together, these foundations explain the principal division of responsibilities in Stockrium: financial methods define the evidence, LLMs interpret it, agents organize the work, deterministic rules constrain the result, and backtesting evaluates historical behavior.

2.3 Development Technologies

Stockrium consists of mobile and web interfaces, a backend application programming interface (API), data pipelines, persistent storage, and agentic artificial-intelligence (AI) workflows. This section introduces the principal technologies used in these components

and the responsibility of each technology within the system.

2.3.1 Frontend Technologies

Mobile application. The mobile client for individual investors is developed with React Native, React, and TypeScript. React supplies the component model, React Native maps components to native mobile user-interface (UI) elements [24], and TypeScript adds static types for market data, financial information, and analytical results. Expo supplies the development runtime and device services, while Expo Router maps files to application routes and navigation paths [25]. Reanimated and Gesture Handler support responsive animation and touch interaction.

Web application. The administrative web interface is implemented with HyperText Markup Language (HTML), Cascading Style Sheets (CSS), and JavaScript. It communicates with protected backend endpoints to support data operations, analysis over selected stock groups, internal backtesting, and result inspection. The interface is intended only for authorized administrators and is not deployed as a public web application.

Data visualization. TradingView Lightweight Charts is embedded through React Native WebView, which hosts web content inside a native screen, to display interactive candlestick, volume, and recommendation charts [26]. Victory Native provides reusable chart components, React Native Skia supplies accelerated two-dimensional drawing [27], and Wagmi Charts supports interactive time-series and indicator lines. Data-Driven Documents (D3) Hierarchy calculates the nested rectangular layout used by the market treemap. These libraries allow each financial visualization to use a rendering approach suited to its data and interaction requirements.

2.3.2 Backend, Authentication, and Security Technologies

The backend is implemented in Python with FastAPI and served by Uvicorn, an Asynchronous Server Gateway Interface (ASGI) server. FastAPI defines Representational State Transfer (REST) endpoints over the Hypertext Transfer Protocol (HTTP), supports asynchronous request handling, and generates interactive API documentation [28]. Pydantic models validate the structure and type of request and response data [29], while Pydantic Settings loads configuration and credentials from environment variables [30]. Requests and HTTPX provide HTTP clients for external services. The source code follows a route–service structure: routes handle transport concerns, services implement application logic, and service or utility clients communicate with Supabase, SSI, and other external providers.

Authentication uses bcrypt, an adaptive one-way password-hashing algorithm, so plaintext passwords are not stored. PyJWT creates and verifies signed JSON Web Tokens (JWTs): short-lived access tokens authorize API requests, while refresh tokens obtain new

access tokens without requiring the user to enter credentials again. Server-side session records in Redis allow issued tokens to be checked and invalidated, while Expo SecureStore protects token values stored on the mobile device [35]. The system also supports Google Sign-In, time-limited one-time passwords (OTPs) for account verification and password recovery, and transactional email delivery through the Resend HTTP API [39]. Secrets, provider credentials, and API keys are kept in environment configuration rather than embedded in source code.

2.3.3 Data Acquisition, Processing, and Storage Technologies

SSI Securities Corporation’s FastConnect (FC) Data services provide Vietnamese security, price, and market-index data through historical APIs and real-time WebSocket streams. A WebSocket maintains a persistent connection for receiving continuous market updates, unlike repeated request–response polling. Offline utilities use Selenium WebDriver to automate the dynamically rendered SSI iBoard interface and collect company profiles and financial reports [36]. The news pipeline uses Serper to discover relevant pages and Newspaper3k to extract article text before OpenAI-based summarization and sentiment labeling.

pandas represents and transforms tabular data, NumPy provides numerical arrays and calculations, and OpenPyXL allows pandas to read downloaded Microsoft Excel workbooks. The Technical Analysis Library (TA-Lib) calculates technical indicators such as moving averages, RSI, MACD, Bollinger Bands, and KDJ, while SciPy supplies statistical functions for backtest evaluation [37, 38]. These acquisition and preparation pipelines populate the database independently of ordinary mobile requests; they are not executed synchronously when a user opens a screen.

Supabase provides the principal PostgreSQL relational database for account, market, company, fundamental, news, and chatbot data [33]. PostgreSQL uses Structured Query Language (SQL) for relational data operations. The Supabase client supports ordinary table access, whereas Psycopg provides direct PostgreSQL connections for LangGraph checkpoint persistence and related chat operations. Supabase Storage stores exported JavaScript Object Notation (JSON) backtest artifacts when they are uploaded. Redis supplies expiring key–value storage for short-lived authentication data, including token sessions, OTPs, attempt counters, and password-reset sessions [34]; it is not the primary store for market or financial records.

2.3.4 AI, Agent Orchestration, and State Management Technologies

The OpenAI Python software development kit (SDK) connects the backend and data pipelines to GPT-4o mini, a Generative Pre-trained Transformer (GPT) model used for article summarization and sentiment labeling, financial interpretation, chatbot responses, and recommendation synthesis. OpenAI Structured Outputs and Pydantic schemas constrain model responses to predefined fields and data types that can be validated and exchanged between workflow components [31]. Schema compliance improves interoperability but does not prove that generated content is factually correct. Numerical indicators, confidence calculations, and selected recommendation constraints therefore remain in deterministic Python code.

LangGraph coordinates the orchestrator, specialist agents, aggregator, and chatbot through explicit stateful graphs, while LangChain Core supplies supporting message abstractions. During stock analysis, a typed shared state carries the request, execution plan, retrieved source data, and agent results. Reducers define how concurrent updates are combined, allowing the technical, fundamental, and news branches to merge their outputs before aggregation without overwriting one another. For the chatbot, LangGraph checkpoints save graph state by session in PostgreSQL through Psycopg, allowing a later request to resume the correct conversation [32]. A checkpoint is a durable snapshot of workflow state, whereas the in-memory shared state exists only during an individual execution. This distinction separates temporary coordination data from persistent conversation history.

2.3.5 Deployment and Development Tools

The FastAPI backend and the separate article-updating service are deployed on Railway, with production configuration supplied through environment variables [40]. Docker packages the scheduled article pipeline and its system dependencies into a reproducible container image. Expo Application Services (EAS) Build produces the mobile application as an installable Android application package (APK) [41]; the administrative web interface is not deployed. npm and pip manage JavaScript and Python dependencies, ESLint performs static checks on frontend source code, and Git records version-controlled changes.

Together, these technologies support the complete Stockrium workflow, from user interaction and financial-data acquisition to agentic analysis, persistence, security, and deployment.

Chapter 3 Proposed Application

This chapter presents Stockrium from the application and user perspectives. It identifies the stakeholders and principal actors, describes the use cases and end-to-end interaction flows, and explains the key features through which the application supports Vietnamese stock-market exploration and analysis.

3.1 Application Idea

Stockrium is proposed as a decision-support platform that helps individual investors examine Vietnamese listed companies without collecting information from many disconnected sources. The mobile application guides the user through a continuous process of discovering a stock, reviewing market and company information, configuring an analysis, receiving a multi-perspective assessment, and investigating the supporting evidence. Technical, fundamental, and news perspectives remain distinguishable so that the recommendation is presented as an explainable synthesis rather than an unexplained prediction.

The project also provides an administrative web interface for controlled analysis and historical backtesting, together with a curated dataset covering price and volume history, company fundamentals, and financial news with sentiment labels. Stockrium remains analytical rather than transactional: it does not place securities orders, transfer funds, or connect to brokerage firms. Any investment action following an analysis remains the user's responsibility.

3.1.1 Stakeholders

The proposed system considers four principal stakeholder categories. Individual investors are the primary users, while the administrator operates system and experimental functions through the restricted web interface. A securities analyst contributes practical evaluation, and technical teams and service providers support the data and software infrastructure.

Table 3.1: Principal stakeholders of Stockrium

Stakeholder	Primary interest	Relationship with Stockrium
Individual investors	Accessible and explainable analysis of Vietnamese stocks	Use the mobile application to explore stocks, configure analysis, review recommendations, and ask follow-up questions
Administrator	Controlled system operation and repeatable experimentation	Uses the internal web interface for individual or batch analysis, back-testing, progress monitoring, and result inspection
Securities analyst	Practical usefulness, clarity, and analytical transparency	Experiences the application and provides qualitative feedback for the evaluation
Technical teams and service providers	Reliable software, data, infrastructure, and external services	Develop and operate Stockrium or supply market data, news discovery, storage, and language-model capabilities

Beginner and experienced investors are treated as profiles within the same stakeholder group rather than as different account roles. Beginners may rely more on default configurations and explanations, whereas experienced users may inspect individual evidence sources and adjust the analytical configuration.

3.1.2 Use Case Model

Stockrium has three direct actors. A guest accesses onboarding and account functions, an authenticated investor uses the personalized mobile features, and an administrator accesses restricted operational and experimental functions. Figure 3.1 presents these actors and their principal use cases.

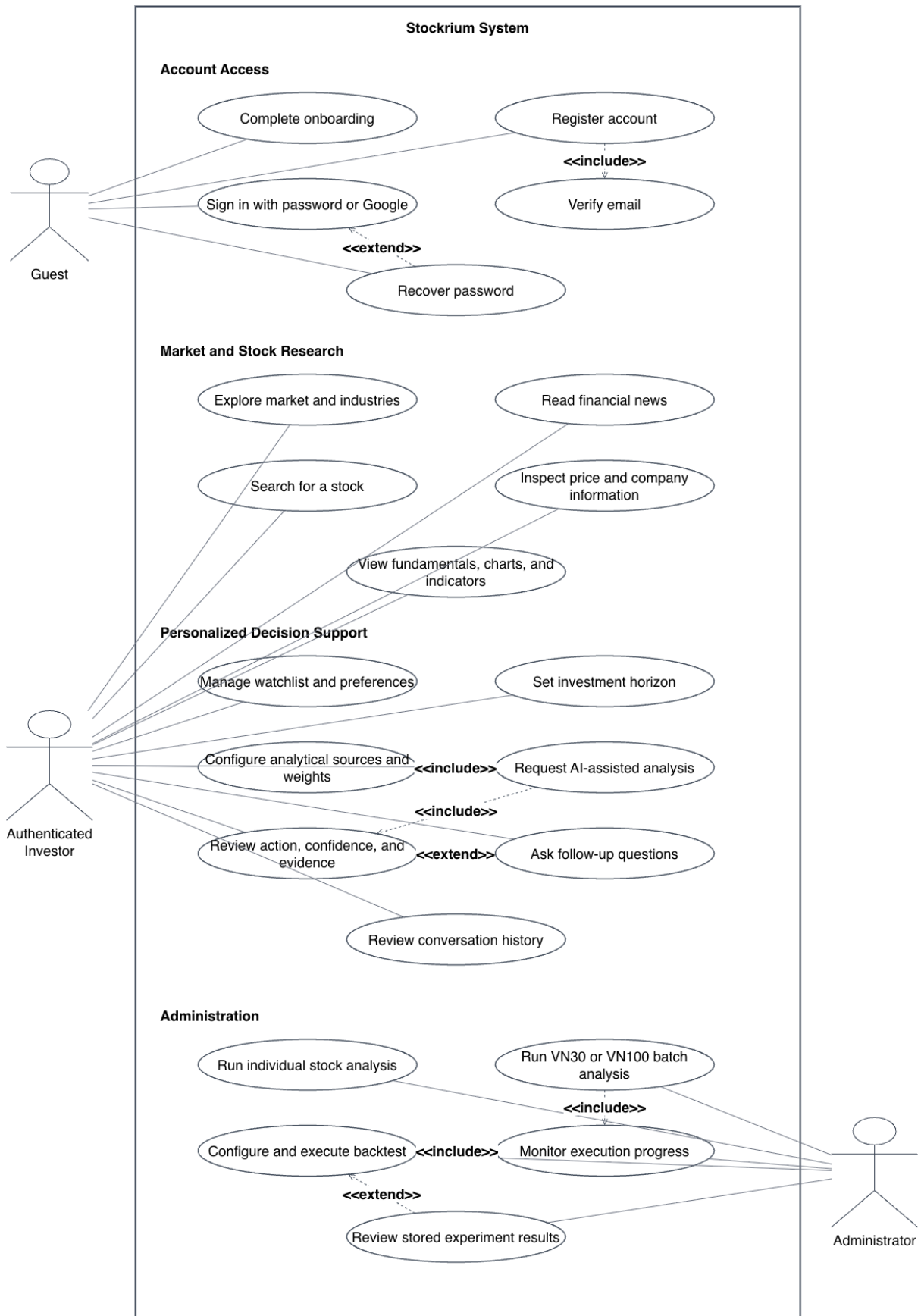


Figure 3.1: Stockrium actors and principal use cases

Table 3.2: Use-case groups by actor

Actor	Principal use cases
Guest	Complete onboarding; register and verify an account; sign in with a password or Google; recover account access
Authenticated investor	Explore the market and news; search and inspect stocks; manage watchlist and preferences; configure and request AI analysis; review recommendations; use the conversational assistant
Administrator	Run individual or batch analysis; configure and execute backtests; monitor progress; review stored results

Market-data providers, news services, and language-model services are supporting dependencies rather than direct actors because they do not initiate the user goals shown in the model. Their interactions with Stockrium are described in the system architecture and data-pipeline sections of Chapter 4.

3.1.3 Main Application Flows

The use cases form two primary experiences: an investor-facing mobile journey and an administrative workflow. These flows describe user-visible behavior and intentionally treat the internal agent system as a single analytical service; its nodes, state, and execution logic are presented in Chapter 4.

Mobile Exploration and Stock-Analysis Flow

A guest first completes onboarding and authenticates through registration, password sign-in, or the supported Google Sign-In flow. After authentication, the investor can enter a stock context from the market overview, search, financial news, or the watchlist. The stock-detail view consolidates price and volume charts, company information, fundamental data, technical indicators, and related news, allowing the investor to examine the available evidence before requesting an assessment.

The user may proceed with an automatic analysis configuration or manually select the evidence sources and their relative weights. The request combines the selected configuration, stock symbol, and saved investment horizon. Figure 3.2 summarizes the first part of the mobile journey, from account access and stock discovery to submission of the analysis request.

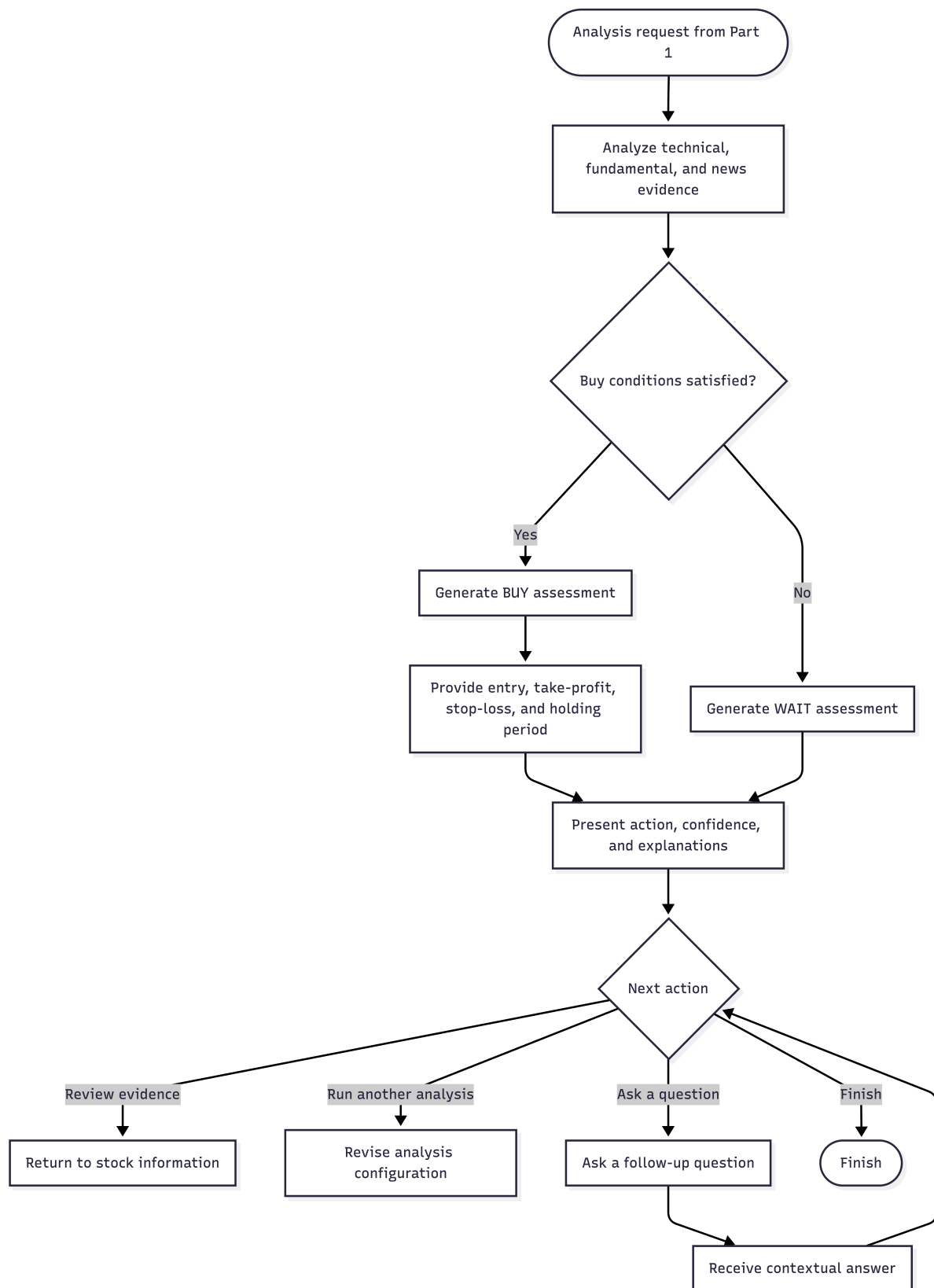


Figure 3.2: Mobile decision-support flow: account access, stock discovery, and analysis configuration

After submission, Stockrium processes the enabled technical, fundamental, and news perspectives and returns either a *Buy* or *Wait* assessment. The result includes confidence and source-specific explanations; an accepted *Buy* assessment may additionally contain an entry price, take-profit price, stop-loss price, and maximum holding period. Figure 3.3 presents the second part of the journey, including evidence evaluation, recommendation validation, and subsequent user choices.

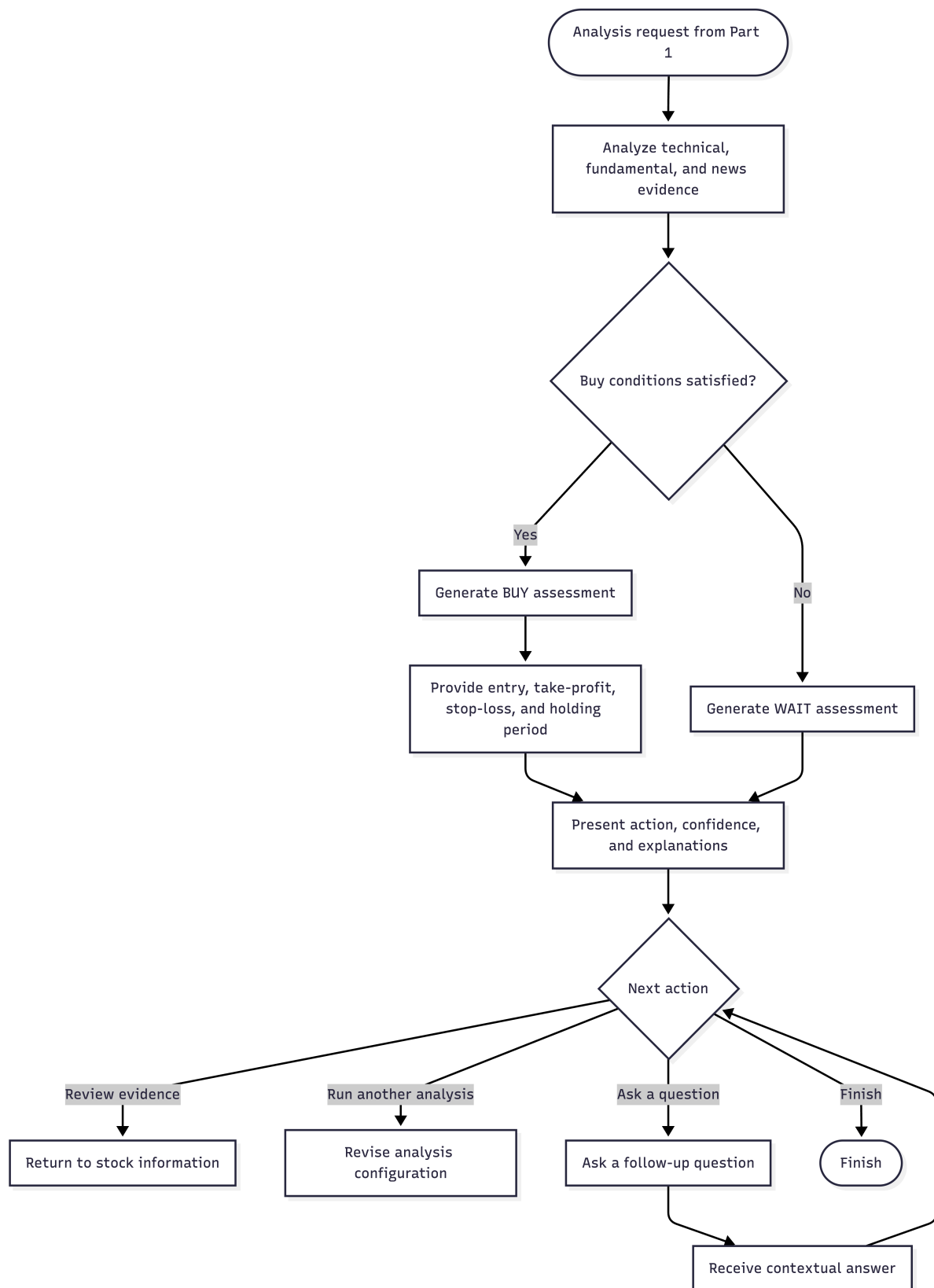


Figure 3.3: Mobile decision-support flow: analysis processing, recommendation, and follow-up

The recommendation is not the final step of the user journey. The investor can return to the supporting charts and company data, revise the configuration, update the watchlist, or ask a contextual question. These paths form an iterative cycle in which the user continues to examine the evidence rather than acting solely on the generated action label.

Conversational Follow-up Flow

The conversational assistant supports both independent financial questions and follow-up questions about an existing analysis. An authenticated investor may create or revisit a conversation, submit a Vietnamese natural-language question, receive an answer based on available system information, and continue with additional turns. The assistant helps explain terminology and analytical evidence, but its answers remain informational and are subject to the same data and model limitations as the main analysis.

Administrative Analysis and Backtesting Flow

The administrator uses the internal web interface for controlled analysis and historical evaluation. AI-assisted analysis can be requested for an individual stock or for the VN30 or VN100 universe. Backtesting can be executed for an individual stock or sequentially for the VN30 universe. The administrator can monitor batch progress and inspect or export the resulting reports. A completed backtest presents price charts, generated entries and exits, performance and risk measures, and relevant analytical results. Figure 3.4 separates these branches and shows how their outputs return to the dashboard for later review.

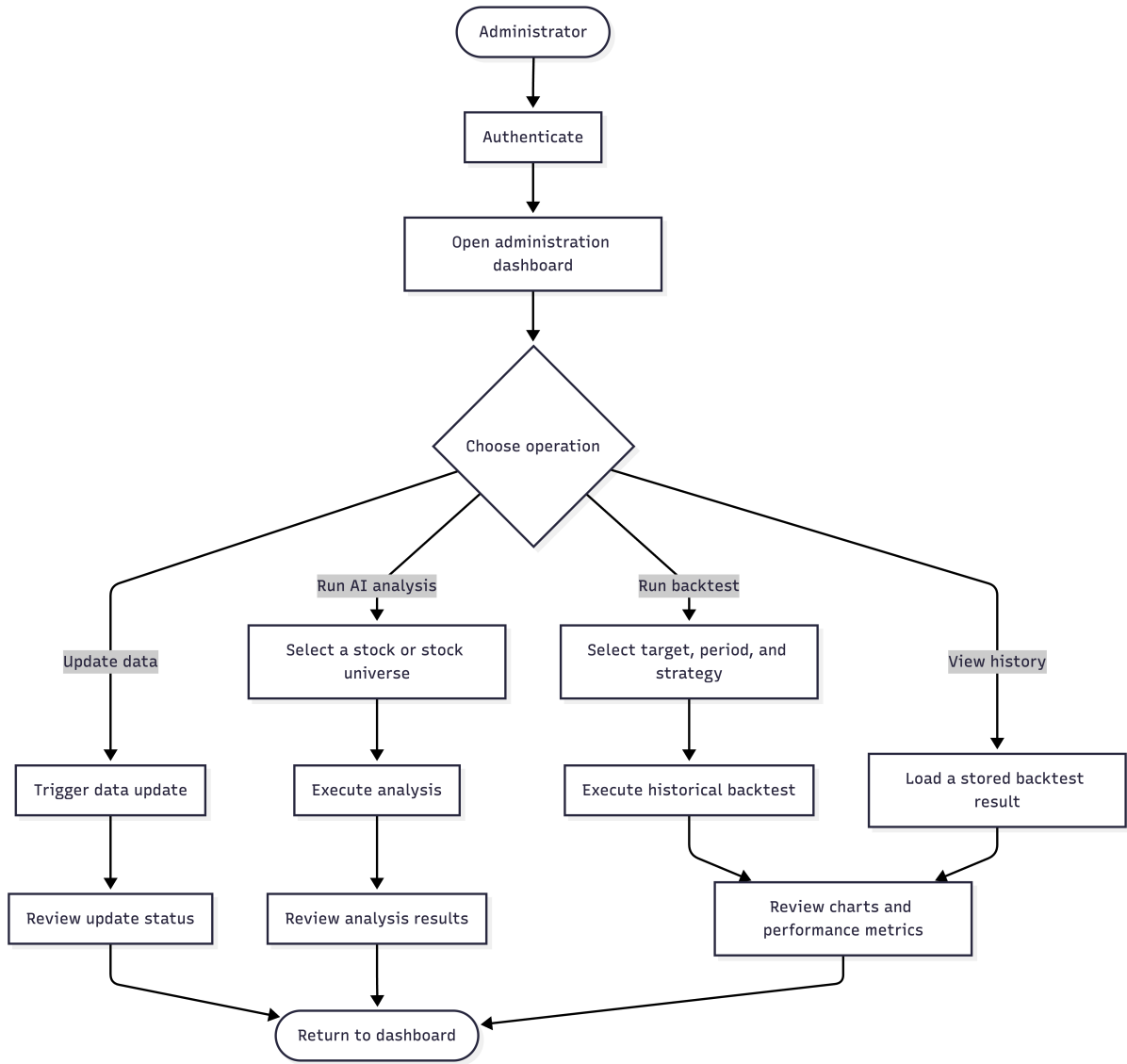


Figure 3.4: Administrative analysis and backtesting flow

The administrative workflow supports system operation and thesis evaluation; it is not a public web trading platform and does not cause any real-market transaction. Together with the mobile journey, it positions Stockrium as an evidence-centered application and a controlled environment for evaluating Stockrium’s analytical pipeline and supported configurations.

3.2 Key Features

Stockrium provides a connected set of features for exploring the Vietnamese stock market, examining financial evidence, and obtaining AI-assisted analysis. The mobile application is intended for individual investors, while a separate administrative web interface supports controlled analysis and historical backtesting. This section summarizes

the main user-visible capabilities; their architecture and implementation are presented in Chapter 4.

3.2.1 Authentication and Account Management

Users can create an account with an email address, verify it through a one-time password, and sign in using either a password or Google Sign-In. The application also supports password recovery, session persistence, and secure sign-out. After authentication, users can manage their investment horizon, conversation history, theme, and language preferences. They can also add or remove stocks from a personal watchlist for later monitoring. This watchlist stores only selected symbols and does not manage quantities, purchase prices, profit and loss, or capital allocation. The security mechanisms used to protect credentials and sessions are discussed in Chapter 4.

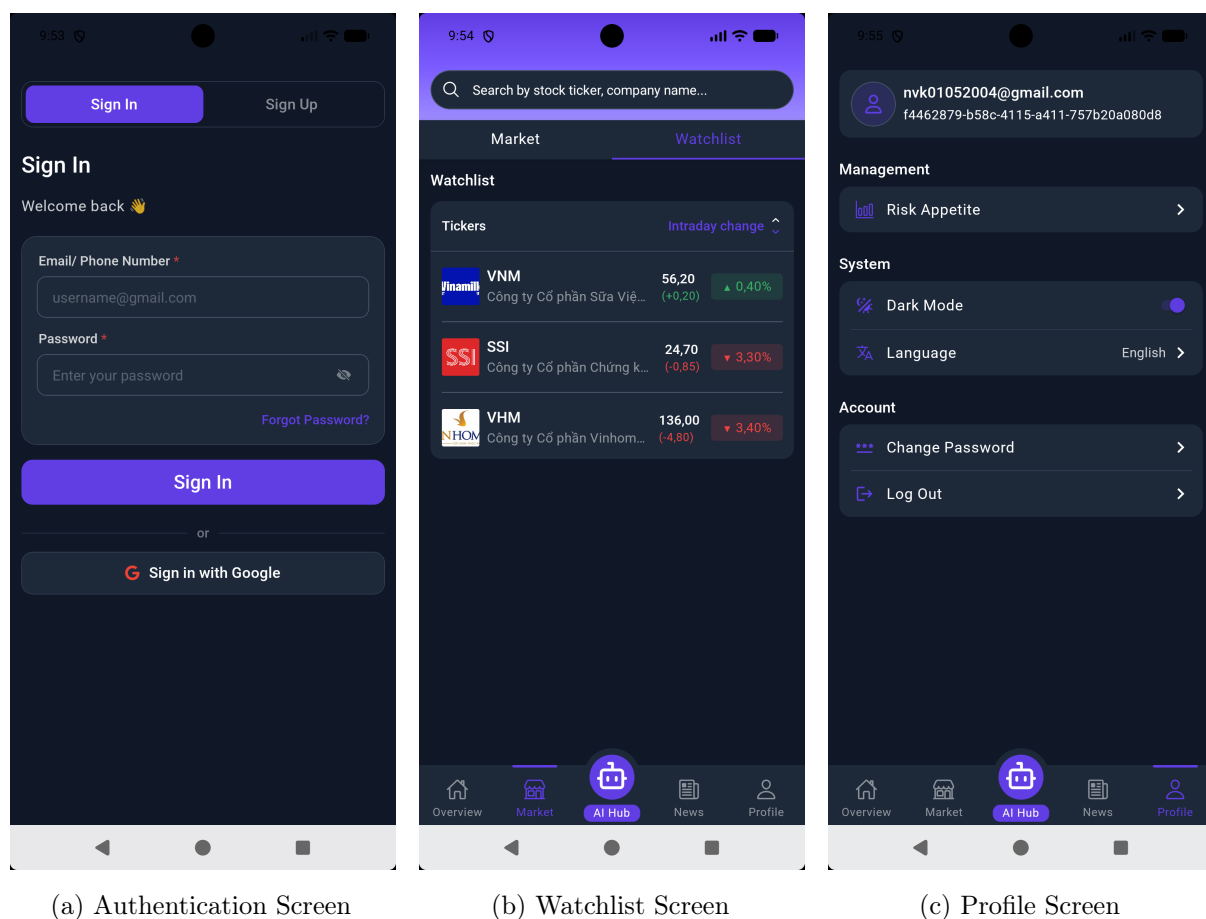
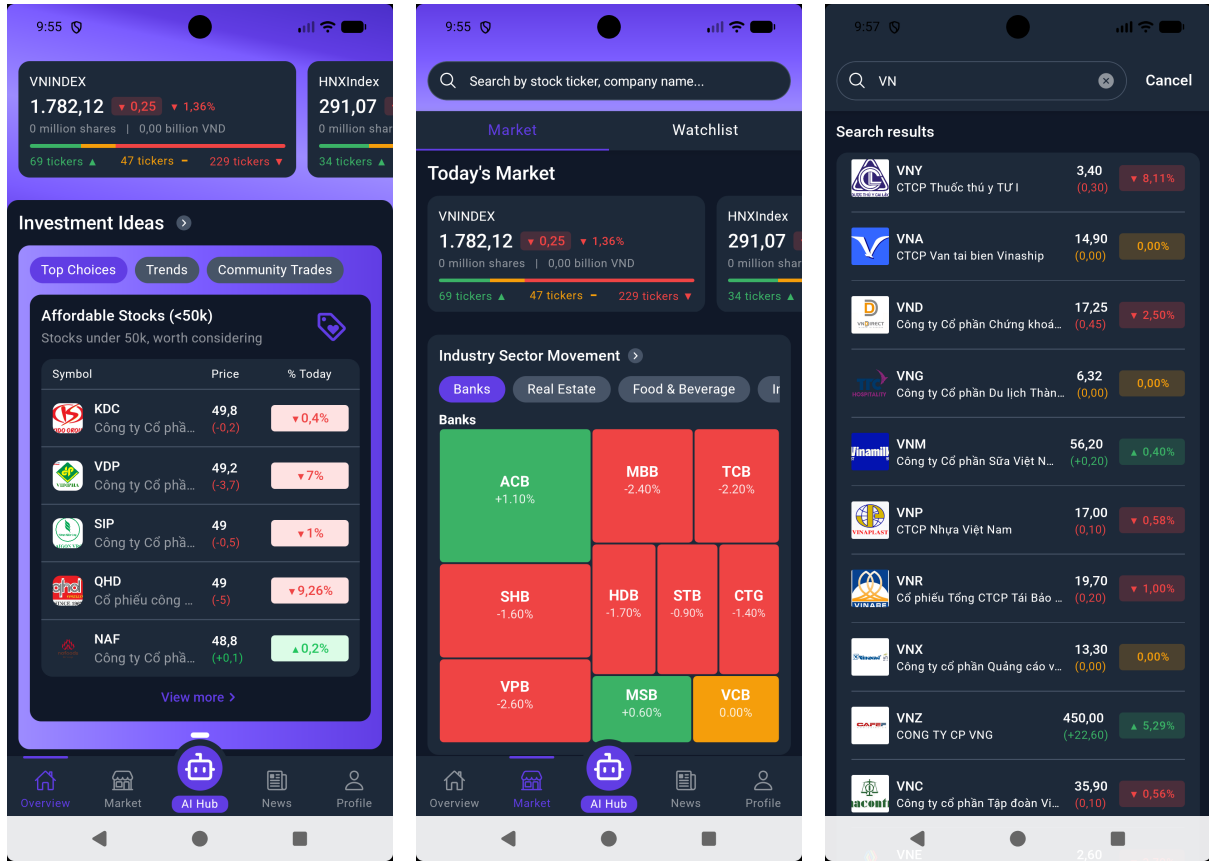


Figure 3.5: Authentication and Account Management

3.2.2 Market Overview and Stock Discovery

The Overview and Market tabs provide a concise view of current Vietnamese market conditions. Users can inspect major indices, industry movements, financial news, and

stock collections such as top gainers, top decliners, highly traded stocks, and frequently followed symbols. The search function allows users to locate listed companies by stock symbol or company name and revisit previous queries through their search history. These discovery features support navigation rather than constitute investment recommendations. A selected index, industry, stock, or article can be opened for further examination.



(a) Home Screen

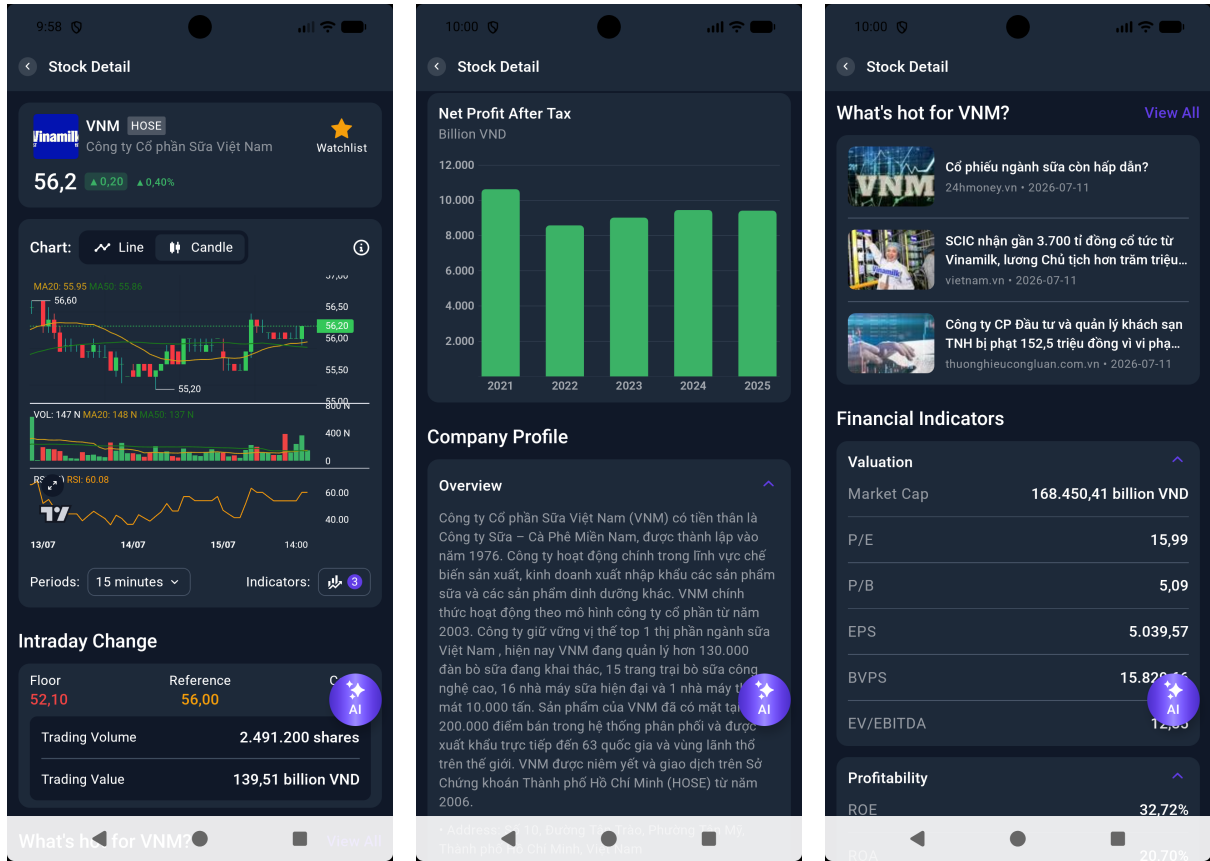
(b) Market Screen

(c) Search Screen

Figure 3.6: Market Overview and Stock Discovery

3.2.3 Stock Details and Interactive Technical Charts

The stock-detail screen combines current and historical market data with company information, fundamental indicators, related news, and similar securities. Interactive area and candlestick charts allow users to examine price, volume, and selected technical indicators over different periods. The same screen also presents the company's business profile, leadership information, historical financial performance, financial statements, and representative measures of valuation, profitability, liquidity, and financial strength. Together, these features help users compare market behavior with the company's operating condition. The displayed information supports independent inspection but does not place orders or initiate financial transactions; AI-assisted analysis is requested separately.



(a) Stock Details 1

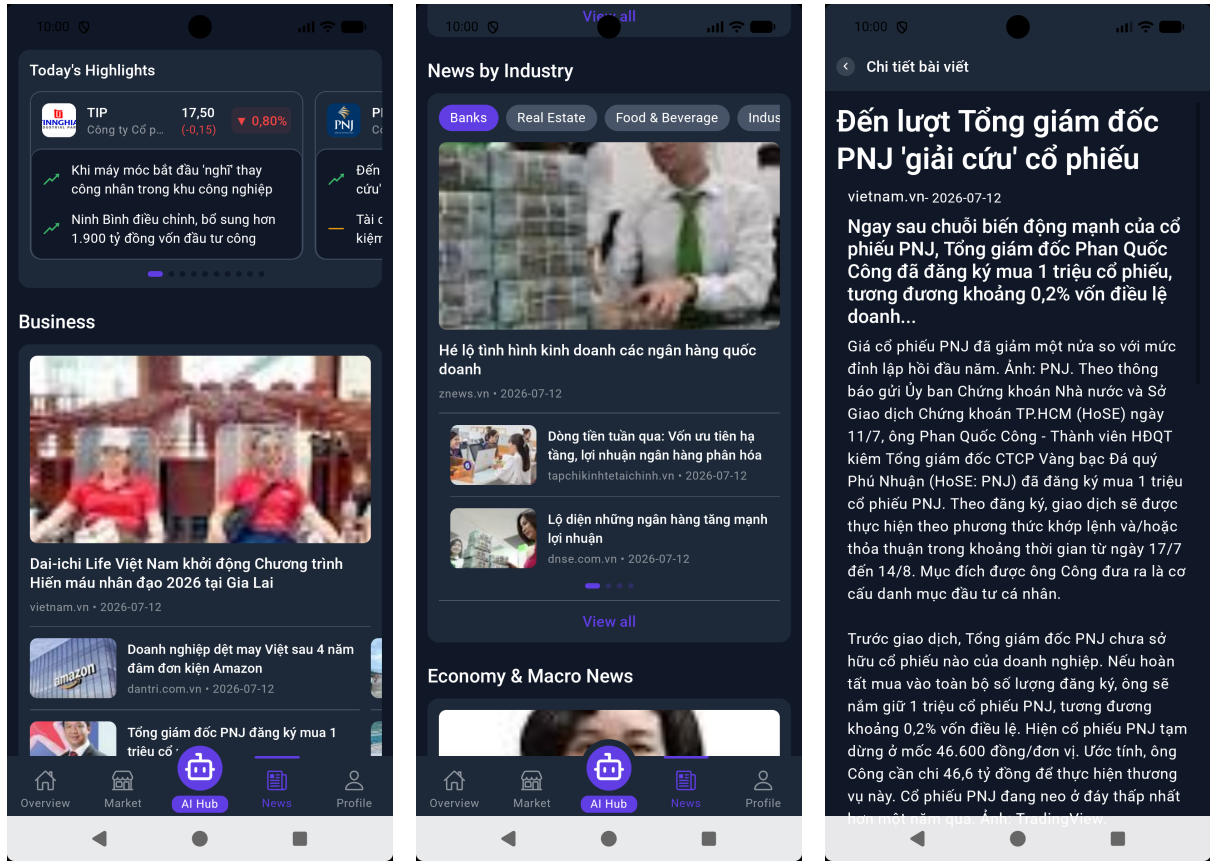
(b) Stock Details 2

(c) Stock Details 3

Figure 3.7: Stock Details and Interactive Technical Charts

3.2.4 Financial News Exploration

The News tab organizes financial articles into business, macroeconomic, and topic-based views. Stock-specific articles are also accessible from the corresponding stock-detail screen, allowing users to relate company events and market sentiment to price and fundamental data. The collection pipeline enriches stored articles with short summaries and positive, neutral, or negative sentiment labels. The mobile application presents article content and surfaces sentiment in selected news views, while the processed fields also provide evidence for subsequent analysis. News collection and processing occur in a separate scheduled pipeline described in Chapter 4.



(a) News Screen 1

(b) News Screen 2

(c) News Screen 3

Figure 3.8: Financial News Explorations

3.2.5 Investment-Horizon Profile and Configurable AI Analysis

Users can save an intended investment horizon as short term (under three months), medium term (three to twelve months), or long term (over one year). Stockrium provides automatic and manual modes for AI-assisted stock analysis. Automatic mode uses all available analytical perspectives with default weights determined by the saved investment horizon, which the system also maps to a daily, weekly, or monthly technical-analysis interval. Manual mode allows the user to enable or disable news, individual technical indicators, and fundamental-indicator categories, and to adjust the relative weights of the technical, fundamental, and news perspectives. The multi-agent system coordinates the enabled components and returns a *Buy* or *Wait* assessment with a normalized confidence score, technical, fundamental, and news scores, and written explanations. Confidence represents the system's internal strength of evidence and is not a probability of future profit. For an accepted *Buy* assessment, the saved investment horizon determines the permitted ranges for the proposed entry price, take-profit price, stop-loss price, and maximum holding period. Within the current application scope, the investment-horizon profile considers only time horizon and is not a comprehensive assessment of financial capacity or psychological risk tolerance. The feature provides a simple default path for beginners and greater control

for experienced users, while its outputs remain decision-support information rather than guaranteed returns or executable orders.

10:09

< Risk Appetite

How long do you plan to invest?

This helps us suggest the right investment strategy for you.

☐ Short-term
Under 3 months

☒ Mid-term
3 - 12 months

☐ Long-term
Over 1 year

Save

(a) Risk appetite

10:10

< AI Analysis

☒ Custom
Manually choose data sources to analyze

Weight allocation

Drag the handles to adjust how much each source contributes

Technical 33% Fundamental 33% News 34%

Data sources

News 1/1 ☒

Technical 5/5 ☒

- ☒ Moving Average (MA)
- ☒ Bollinger Bands
- ☒ RSI
- ☒ MACD
- ☒ KDJ

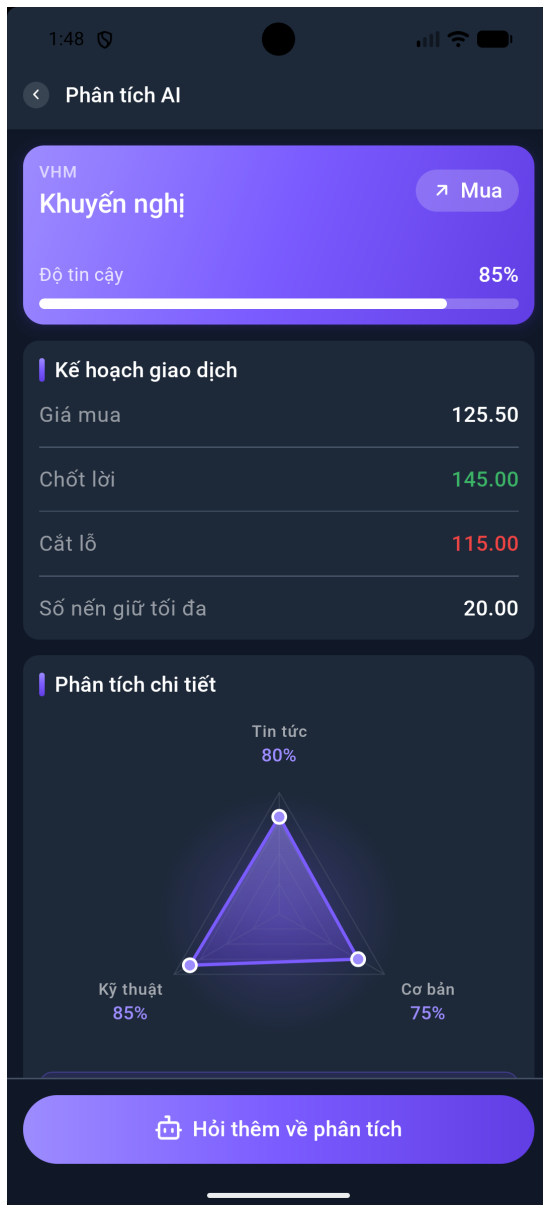
Fundamental 5/5 ☒

- ☒ Liquidity
- ☒ Leverage

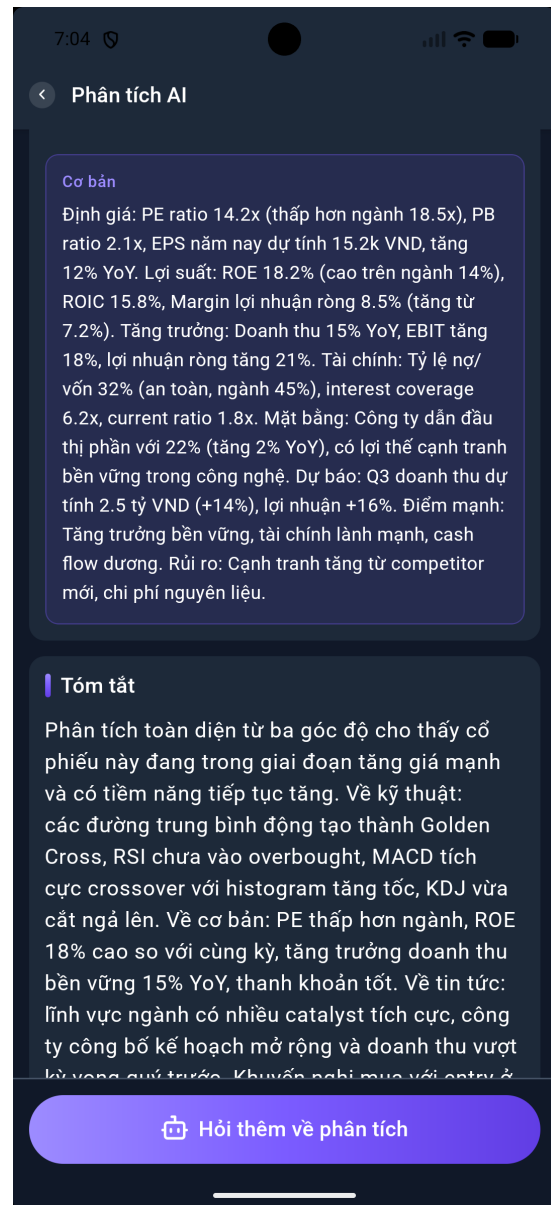
Analyze

(b) Manual configs

Figure 3.9: Initial input for Agents



(a) Analysis result 1

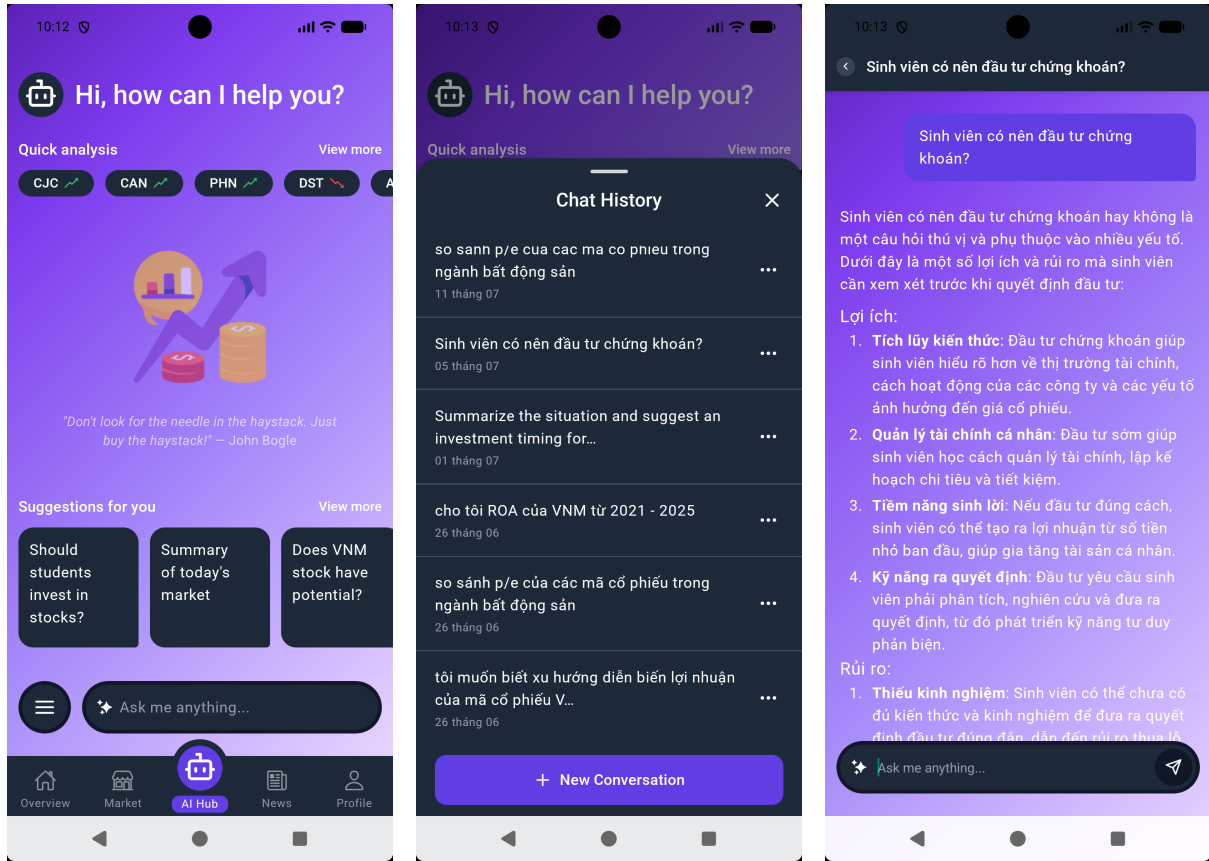


(b) Analysis result 2

Figure 3.10: Agents' analysis results

3.2.6 Conversational Financial Assistant

The conversational assistant allows users to ask natural-language questions about the financial and market information available in Stockrium. Conversations are stored as sessions so that users can create, reopen, continue, or delete previous discussions. A conversation may begin as a general inquiry or from an AI-analysis result, enabling follow-up questions about the recommendation and its supporting evidence. The assistant explains and summarizes available information but does not act as a brokerage service or licensed financial adviser.



(a) Chatbot Screen

(b) Chat History

(c) Conversation Screen

Figure 3.11: Conversational Financial Assistant

3.2.7 Administrative Analysis and Backtesting

The administrative web interface provides protected tools for running stock analysis and historical backtests. An administrator can request AI-assisted analysis for an individual stock or the VN30 or VN100 index basket. Backtesting can be configured for an individual stock or executed sequentially across VN30 using the supported inputs and analytical selections. The administrator can monitor batch progress and inspect or export the resulting reports. Backtesting summarizes historical behavior through measures such as total return, win rate, Sharpe ratio, maximum drawdown, and trade history, with visualizations linking generated signals to price data. This interface is an internal evaluation and operational tool; it is not publicly deployed and does not execute live securities transactions.

Together, these features form a decision-support environment that connects market discovery, evidence inspection, AI-assisted synthesis, explanation, and conversational follow-up. Stockrium helps users organize and interpret Vietnamese market information while keeping investment decisions and any subsequent transactions under the user's responsibility.

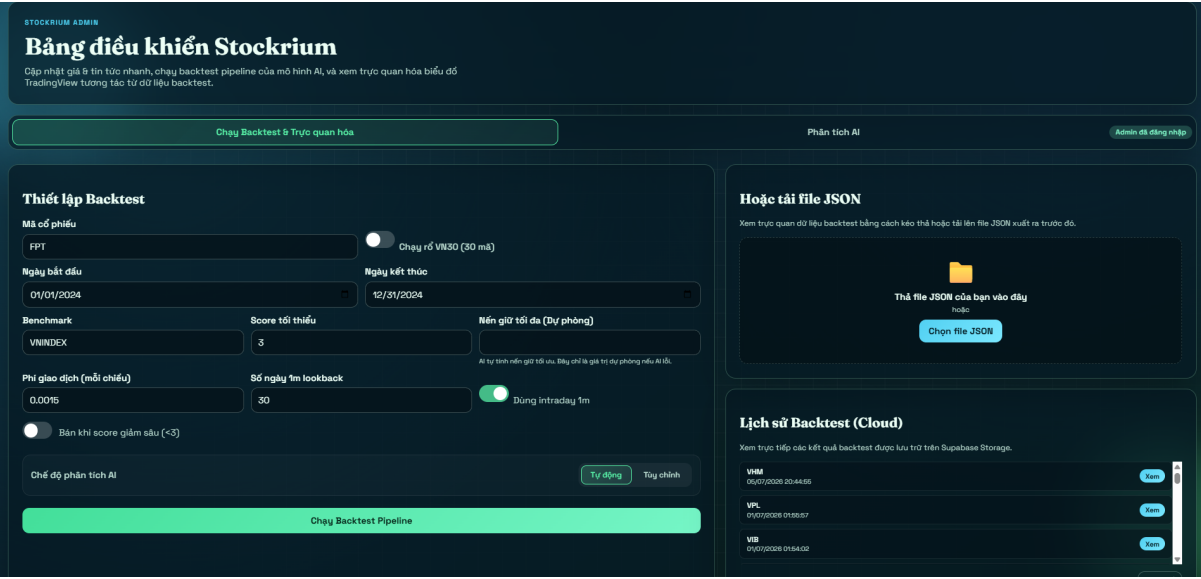


Figure 3.12: Admin Backtest Screen



Figure 3.13: Admin Backtest Result Screen

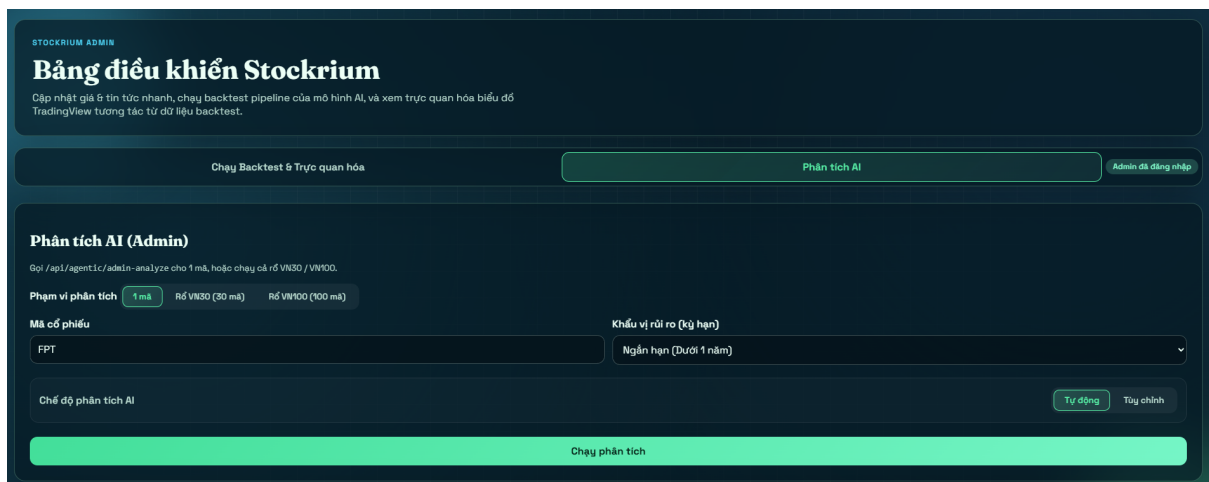


Figure 3.14: Admin Analyze Screen

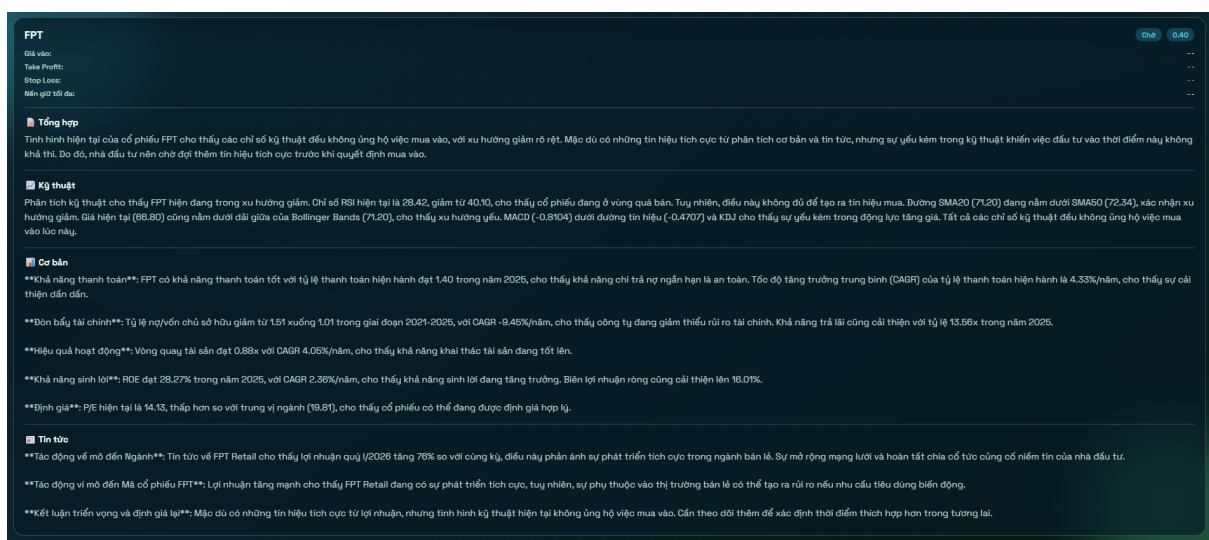


Figure 3.15: Admin Analyzer Result Screen

Chapter 4 System Design and Implementation

This chapter describes the architecture and implementation of Stockrium. It explains how the mobile client, administrative dashboard, backend services, persistent data stores, external data pipelines, agentic analysis workflow, conversational assistant, backtesting framework, security mechanisms, and deployment environment cooperate to realize the proposed application.

4.1 System Architecture Overview

Stockrium follows a client–server architecture supported by independent background and offline data-acquisition pipelines. The React Native mobile application is the primary interface for individual investors, while an internal web dashboard allows administrators to run controlled analysis and backtesting. Both clients communicate with a FastAPI backend that centralizes authentication, business logic, data access, AI workflows, and standardized API responses.

Supabase PostgreSQL stores the application’s durable business, market, financial, news, and conversational data, while Redis maintains temporary authentication and verification state. Separate workers collect and prepare information from external sources such as SSI and online news providers. This separation keeps provider credentials and processing logic outside the clients and allows most user requests to read normalized data that have already been prepared.

4.1.1 Architectural Requirements

The architecture must support mobile market exploration, configurable multi-agent analysis, conversational queries, and administrative backtesting without coupling these responsibilities to the user interface. It must integrate heterogeneous financial evidence, protect user-specific and administrative operations, preserve conversation state, and update market and news data without blocking normal requests. Maintainability is supported by separating presentation, domain services, AI workflows, and data access. Most importantly, the architecture reflects the application’s decision-support scope: no component connects to a brokerage account, transfers funds, or executes securities transactions.

Several quality attributes influence this organization. Security requires provider keys and privileged data access to remain on the server side. Responsiveness requires ordinary mobile requests to read prepared records instead of waiting for crawlers or large imports. Maintainability requires the API, analytical workflows, and acquisition jobs to evolve independently, while traceability is supported by structured results and persistent

records for conversations and historical experiments. The architecture therefore prioritizes clear responsibility boundaries, traceable processing, and repeatable evaluation rather than the low-latency execution required by a trading platform.

4.1.2 Logical Architecture

The logical architecture is organized into presentation, API and application services, AI and evaluation, data integration, and persistence. Background workers connect external providers to the data layer independently of user requests. Figure 4.1 summarizes the principal components and their relationships.

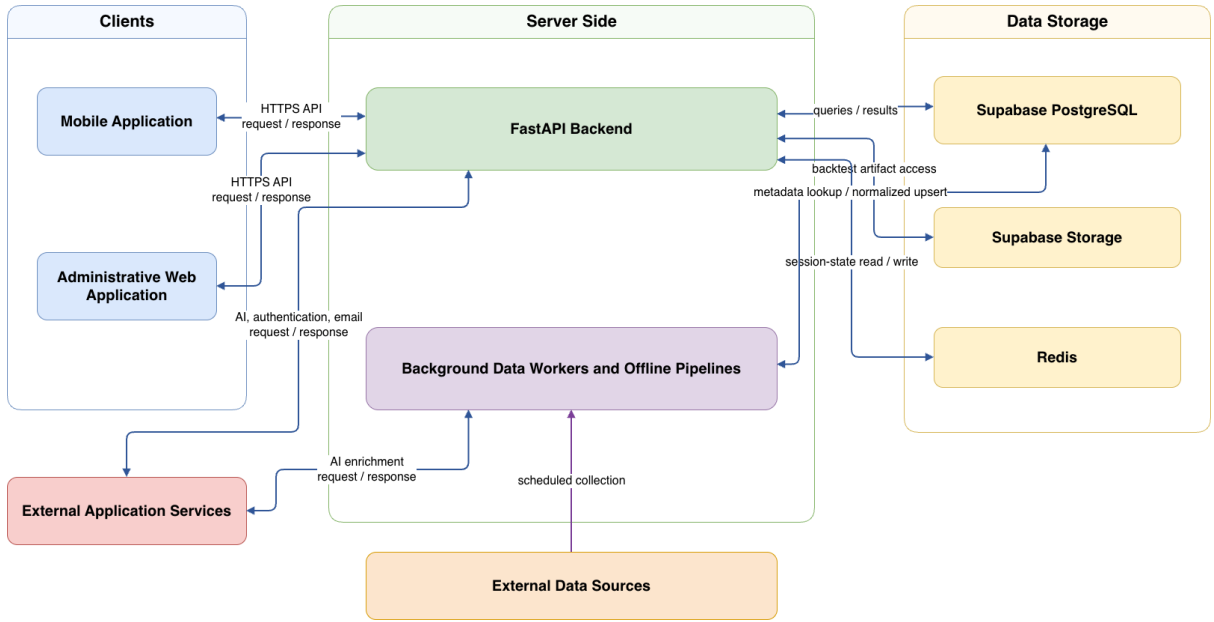


Figure 4.1: Overall logical architecture of Stockrium

On the client side, the mobile application and administrative web application serve different actors while sharing the same server boundary. The mobile application supports everyday investor activities, including authentication, market discovery, stock details, charts, news, the watchlist, investment-horizon management, AI-assisted analysis, and conversational follow-up. It retains only interface state, user preferences, and securely stored authentication tokens; authoritative financial and user data remain on the server. The web application is restricted to administrators. It supports AI-assisted analysis for an individual stock, VN30, or VN100 and backtesting for an individual stock or VN30, together with progress monitoring, result inspection, and export. Backtesting evaluates Stockrium’s analytical pipeline and supported configurations; it is not a general-purpose strategy builder or a publicly deployed web equivalent of the mobile product.

On the server side, FastAPI acts as the single entry point for Stockrium application requests from both clients. It validates request schemas, authenticates the current session,

checks administrative authorization where required, invokes the relevant application service, and returns the standardized Stockrium response structure. Domain services organize operations for accounts, market data, company profiles, financial statements, articles, watchlists, search history, and investment horizons. This route–service separation prevents the clients from depending on database-specific queries and permits the same business operations to be reused by analytical agents, chatbot tools, or backtesting services.

The server also owns the computation-intensive and security-sensitive capabilities. The stock-analysis workflow coordinates technical, fundamental, and news evidence before producing a structured recommendation. The conversational workflow maintains multi-turn state and selects suitable financial tools, while the backtesting subsystem applies the analytical logic to historical data under supported backtest inputs. These components may call external language-model services, but API keys, prompts, validation rules, and post-processing remain within the server-side environment. Their internal designs are described separately later in this chapter.

The storage side is divided according to data lifetime and format. Supabase PostgreSQL is the authoritative relational store for accounts, personalization, stocks, prices, market indices, company records, financial statements, articles, classifications, chat-session metadata, and LangGraph checkpoints. Supabase Storage holds backtest artifacts that are more suitable as files than relational rows. Redis stores revocable or short-lived authentication state, including access and refresh token registrations, one-time passwords, attempt counters, and password-reset sessions. Interactive client operations retrieve and update these stores through the FastAPI backend, while authorized acquisition workers access the required data stores independently of client requests.

Background workers form a second server-side execution path that is independent of interactive API requests. The market-price worker receives SSI data, prepares intraday and daily records, and performs post-session normalization. The article worker discovers and extracts financial news, checks duplicates, generates summaries and sentiment labels, links articles to stocks or categories, and persists the processed results. Company and fundamental utilities collect profile and financial information through Selenium and downloadable files before transforming it with pandas. These jobs can read database metadata or existing records before upserting normalized results into PostgreSQL.

External application services support both synchronous and background operations. OpenAI models are used by the analytical workflows and selected enrichment jobs, while Resend delivers account-verification email. These services are accessed by the backend or workers so that credentials and output validation remain in the server-side environment. Google Sign-In follows a different boundary: the mobile application initiates authentication with Google Identity, receives an identity token, and sends it to the backend for verification and issuance of Stockrium access and refresh tokens. None of these request–response integrations is an authoritative source of Stockrium’s business or market records.

4.1.3 End-to-End Request Flows

For an ordinary data-retrieval request, a client sends a Hypertext Transfer Protocol (HTTP) request to FastAPI and includes a Stockrium authentication token where required; deployed mobile requests use Hypertext Transfer Protocol Secure (HTTPS). The route validates the input and resolves the current user before invoking a domain service. The service reads the corresponding prepared records from PostgreSQL, applies any required transformation, and returns them through the standard response envelope. The client then maps the response into cards, lists, charts, or detail views. User actions such as updating the watchlist, search history, or investment horizon follow the same path but write the authorized change before returning the updated state.

An AI-analysis request follows the same client-server boundary but invokes a server-side graph after the relevant market, fundamental, and news evidence has been retrieved. Only the selected analytical components participate, and their outputs are synthesized and validated before the result is returned to the client. A conversational request additionally loads and updates its LangGraph checkpoint, whereas an administrative backtest repeatedly applies analytical logic to historical records and may expose or store an experiment artifact. Although these operations differ in duration and state, none bypasses backend authentication or grants a client direct access to the database or model provider.

Data acquisition follows a path independent of interactive requests. At scheduled times, during market hours, or through a controlled offline preparation run, a worker or utility reads from its external provider, normalizes the received representation, checks required database context, and upserts the prepared data into PostgreSQL. The FastAPI service later reads the resulting records in response to client requests. Separating this path from the web-server lifecycle prevents a market stream, browser crawler, article-processing batch, or large import from delaying ordinary API traffic and avoids duplicating scheduled jobs when the API process restarts or scales.

Overall, Stockrium combines thin client applications, a validated FastAPI boundary, modular domain and AI services, independent acquisition pipelines, and storage selected according to data lifetime. The following sections describe each of these areas in greater detail.

4.2 Mobile Application Design

The Stockrium mobile application is implemented with React Native, Expo, TypeScript, and Expo Router. Its design follows a screen-oriented structure in which every route represents a user-visible task and reusable components encapsulate market cards, charts, forms, bottom sheets, and financial sections. The client is intentionally responsible

for interaction, presentation, lightweight input validation, and local preferences. Authentication decisions, authoritative financial data, technical calculations, and AI execution remain behind the backend API.

The application combines Expo Router’s file-based stack navigation with a custom five-tab interface. File-based routes manage transitions between major screens, while the main application shell uses a pager to support horizontal tab movement and preserve the distinctive central Chatbot control. This hybrid navigation model provides conventional stack behavior for detailed tasks without restricting the visual and interaction design of the primary tabs.

4.2.1 Navigation Structure

The root layout establishes the providers and infrastructure that every screen requires. It first loads the Roboto font family and then nests the application inside a safe-area provider, gesture-handler root, localization provider, and theme provider. An Expo Router stack manages screen transitions with the default header disabled because Stockrium screens provide their own headers. A global session-expired modal is rendered beside the stack so that authentication failure can be communicated regardless of which screen is currently active.

The index route acts as a navigation gateway rather than a visible home screen. It reads the persistent onboarding flag and the stored session. If onboarding has not been completed, the user is redirected to the Onboarding route. If onboarding is complete, both token values are present, and the refresh token has not expired, the user is redirected to the main Tabs route. Otherwise, the Authentication route is opened. The gateway does not require the access token itself to remain valid because an expired access token can be renewed transparently by the centralized HTTP client.

The authentication branch contains two related paths. The account-creation path proceeds from the sign-up form to one-time password (OTP) verification. Successful verification confirms the email address and returns the user to authentication; it does not create an authenticated session automatically. The account-recovery path proceeds from email input to OTP verification and then to the new-password screen before returning to authentication. Returning users can sign in with application credentials or Google and move directly to the Tabs route. Navigation replacement is used after onboarding or successful authentication so that the Android back action does not return the user to a completed entry screen.

The main shell contains Overview, Market, Chatbot, News, and Profile. It is implemented with a pager and a custom bottom bar rather than Expo Router’s default tab navigator. A tab is mounted only after it has been visited, reducing the initial work required to construct data-heavy market and news screens. The active-tab indicator

is animated, and the Chatbot is represented by a raised central button instead of a conventional tab item. Swiping the pager or selecting a bottom-bar item updates the same active index.

Figure 4.2 summarizes the principal route relationships. It groups routes according to user goals rather than listing every modal bottom sheet as an independent destination.

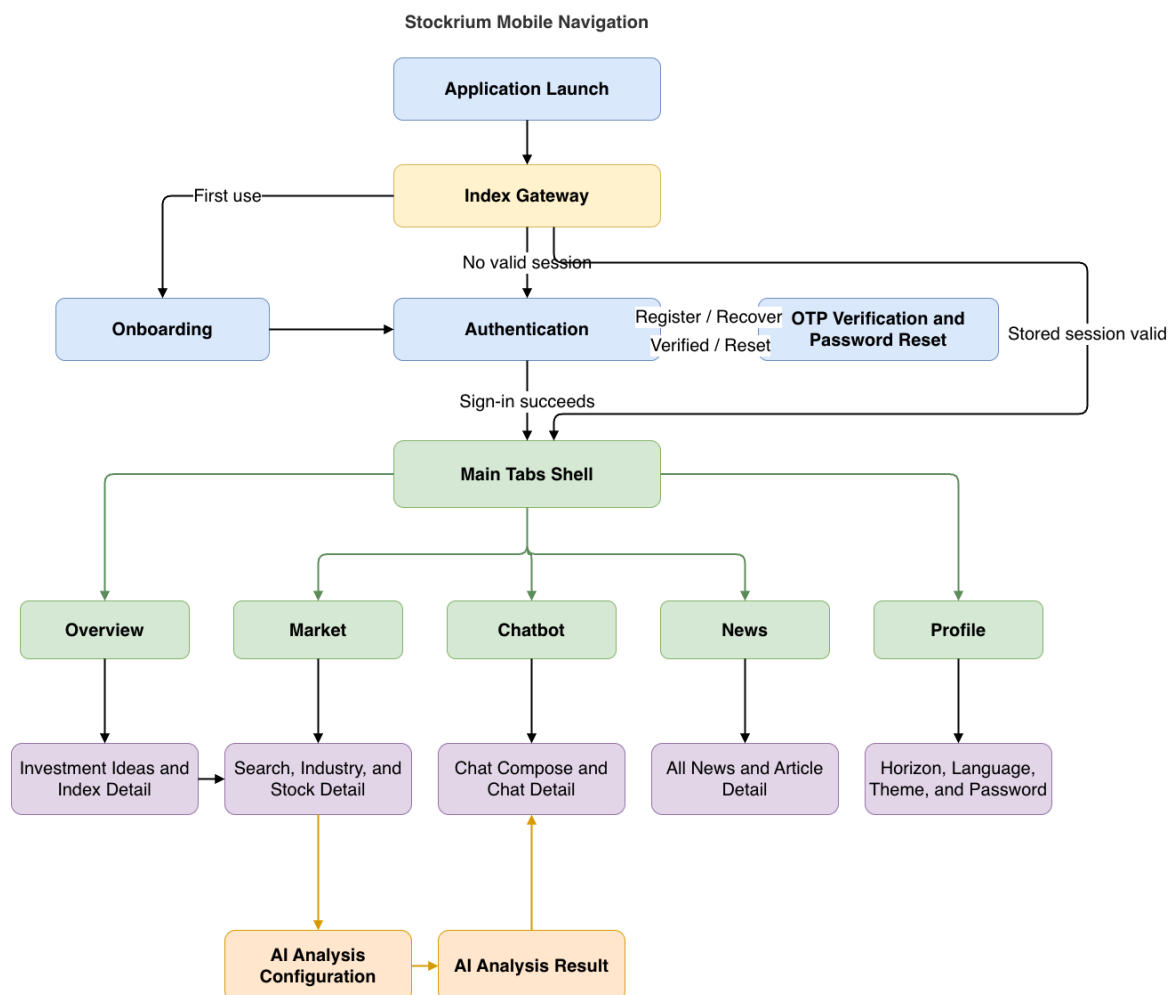


Figure 4.2: Principal navigation structure of the Stockrium mobile application

The Stock Detail route is a particularly important navigation hub. It can be opened from search results, watchlist entries, investment-idea lists, industry visualizations, market highlights, and related-stock cards. It receives the stock symbol as a route parameter and passes the same symbol to its child sections. A floating AI button opens the configuration route with that symbol, after which the result route receives the analysis mode and serialized manual selection. This parameterized navigation keeps the user focused on one company across data inspection and AI analysis.

News and chatbot routes follow a similar pattern. General or categorized lists open a shared article-detail route using article parameters. The Chatbot tab displays available

sessions; composing or selecting a session opens Chat Detail with a session identifier. An AI result can create and seed a new session before navigating to that same Chat Detail route, allowing ordinary conversations and analysis follow-ups to reuse one interface.

4.2.2 Screen and Component Organization

The source tree separates route-level orchestration from reusable presentation and API access. Table 4.1 shows the principal frontend directories and their responsibilities.

Table 4.1: Organization of the mobile application source code

Directory	Responsibility	Representative contents
<code>app/</code>	Route-level screens and navigation parameters. Screens compose domain components and manage task-specific state.	Home, Market, Detail, Search, Investment Ideas, AI Analysis, Chatbot, News, Profile, authentication, and password-recovery routes.
<code>components/</code>	Reusable domain and visual components that can be embedded in multiple screens.	Authentication forms, market sections, stock-detail sections, chatbot bottom sheets, chart wrappers, shared inputs, text, headers, cards, and loading states.
<code>helpers/</code>	Typed feature-oriented functions that translate screen requests into API calls or local utility operations.	Authentication, market, stock detail, fundamental analysis, company profile, search, profile, and agentic helpers.
<code>helpers/api/</code>	Shared network and session infrastructure used by every feature helper.	Base-URL selection, bearer-token injection, refresh coordination, secure token persistence, authentication events, and tab events.
<code>hooks/</code>	Cross-screen providers and reusable interaction hooks.	Theme and localization contexts, together with a keyboard-visibility hook.
<code>constants/</code>	Centralized design tokens and bilingual content.	Theme colors, typography, font names, image references, and Vietnamese and English translation dictionaries.
<code>assets/</code>	Static resources packaged with the application.	Application icons, splash images, fonts, and graphical resources.

Route-level screens are responsible for composition rather than implementing every visual element directly. For example, the Detail route arranges a price-chart component, stock-related news, financial indicators, income-statement visualization, company intro-

duction, and related stocks. Each child receives the stock symbol and, where appropriate, a callback through which it registers its refresh operation. The parent’s pull-to-refresh handler executes the registered functions concurrently with `Promise.all`. This refresh-registry pattern preserves section ownership of data-fetching logic while providing one coherent gesture at screen level.

The same approach is used on the Overview, Market, and News screens. Their child sections load different endpoints but register with the visible parent. The Market screen keeps separate registries for its Market and Watchlist pages so that refreshing one page does not unnecessarily reload the other. This design avoids a single large screen component with detailed knowledge of every endpoint.

Reusable domain components are grouped by business context. The `market` group contains index, industry, watchlist, and news-list sections. The `detail` group contains chart controls, financial sections, company information, related-stock cards, and educational bottom sheets. The `chatbot` group manages history and conversation options. The `authentication` group contains sign-in, sign-up, social login, and session-expiration presentation. Components that are independent of a specific feature—such as the typographic Text wrapper, screen header, search bar, dropdown, news item, and visual charts—are placed under `ui`.

The custom Text component and centralized typography constants give visual consistency to headings, labels, body text, and captions. Theme colors are obtained from context rather than duplicated as literal values in every screen. Safe-area insets are included in layouts that touch device boundaries, and gesture handling is established at the root so that pagers, charts, bottom sheets, and press interactions can coexist.

Loading behavior is designed to preserve context. Data-heavy screens use skeleton components on the first request instead of showing an empty layout. Subsequent chart time-frame changes use a lighter loading state and reduce chart opacity, allowing the user to understand that the same view is being updated rather than replaced. Errors generally preserve the surrounding screen and omit or reset only the unavailable section. Backend error semantics and comprehensive error testing are discussed in later sections.

4.2.3 Financial Data Visualization

Financial information contains temporal, categorical, and multi-dimensional relationships that are difficult to understand as unformatted text. Stockrium therefore uses several visualization strategies rather than one general chart component. Table [4.2](#) summarizes the main visualizations and their roles.

Table 4.2: Principal financial visualizations in the mobile application

Visualization	Implementation approach	Purpose and interaction
Price and candlestick chart	TradingView Lightweight Charts executed inside a React Native WebView. TypeScript converts API records into serializable open, high, low, and close (OHLC) series and injects them into the chart runtime.	Displays historical area or candlestick prices, supports time navigation, and provides a detailed stock-centered view.
Volume and technical indicators	Volume, moving average (MA), Bollinger Bands, RSI, MACD, and KDJ series are prepared with memoized transformations and passed to the same chart runtime.	Lets the user combine one main-chart overlay, one oscillator-style subchart, and optional volume without permanently overcrowding the price view.
AI radar chart	React Native SVG with Reanimated paths and points.	Compares normalized technical, fundamental, and news scores; pressing an axis selects and reveals the corresponding written explanation.
Industry tree map	D3 hierarchy computes a squarified rectangle layout, while React Native SVG draws the result.	Encodes trading value through rectangle area and price movement through color, with a distinct highlight for large positive changes; selecting a rectangle opens stock information and navigation.
Financial bar charts	Victory Native and Shopify Skia render historical revenue and profit series.	Makes changes across reporting periods easier to compare than a list of yearly values.
Progress and confidence indicators	Native views and animations render percentages, loading progress, and selection state.	Communicates AI confidence, active configuration, loading state, and completion without requiring a separate charting runtime.

Interactive Price-Chart Architecture

The main price visualization uses a `WebView` because the `Lightweight Charts` runtime is browser-oriented and provides efficient financial time-series behavior. The `React Native` wrapper creates the local `HTML` and `JavaScript` source once, retains a reference to the `WebView`, and waits for a `chart-ready` message. Once ready, the wrapper injects serialized price, volume, moving-average, Bollinger, `MACD`, `RSI`, and `KDJ` arrays. Later changes update data or chart options without recreating the entire screen.

The Price Chart component keeps UI state for chart type, time frame, overlay indicator, subchart indicator, volume visibility, guide visibility, and loading status. Eight user-selectable time frames are mapped to backend intervals: one minute, five minutes, fifteen minutes, thirty minutes, one hour, one day, one week, and one month. Price and indicator requests are performed concurrently for the selected interval. The returned data are converted into the timestamp and field structure required by the chart.

Memoization is used for these transformations because the same price series contributes to several visual layers. `OHLC` values form the primary series; the close–open relationship determines volume color; indicator records become `MA`, Bollinger, `RSI`, `MACD`, `KDJ`, and volume-moving-average series. Recalculation occurs only when the relevant backend data or theme colors change.

Chart options are sent through small injected functions. The native side can switch between area and candlestick series, update the theme, show or hide volume, and select technical modes. The `WebView` posts readiness events back to `React Native`. This two-way boundary isolates browser chart code from the screen while preserving native controls and theme integration.

On the Stock Detail route, one bottom sheet provides time-frame selection, another controls indicators, and a multi-step guide explains chart structure, volume, controls, and representative candlestick patterns. These educational controls make the local `Lightweight Charts` visualization usable for both experienced investors and users who are still learning financial charts. A separate `TradingView` route embeds the external `TradingView` chart page in a `WebView` and provides additional viewing space with portrait and rotated landscape modes; it is distinct from the locally constructed chart and its bottom-sheet controls.

Native Analytical Visualizations

The AI radar chart remains native because its data contain only three normalized values and benefit from direct interaction with result-screen state. Concentric `SVG` polygons and axes form the grid; the data polygon animates from the center when values arrive. Each axis label displays a percentage and can be pressed. The selected index controls which source explanation is visible below the chart.

The industry tree map combines D3 layout computation with native SVG rendering. Each stock becomes a leaf whose area is proportional to trading value. Color distinguishes positive, negative, and unchanged movement, with a separate highlight for changes above the application's large-increase threshold. Selecting a rectangle opens a detail modal and allows navigation to the stock. This visualization makes it possible to see both market direction and relative activity across many stocks without requiring a long sorted list.

Historical revenue and profit are displayed with bar-chart primitives suited to periodic categorical comparison. These charts are intentionally separate from the market-price runtime because annual financial statements have a different scale and interaction model. Across all visualizations, colors are read from the theme when possible so that success, error, neutral, background, and text semantics remain consistent in light and dark modes.

4.2.4 API Integration and Client State

The mobile application does not call `fetch` directly from every screen. Feature-specific helpers expose typed functions such as market-index retrieval, stock-price retrieval, company-profile retrieval, watchlist operations, investment-horizon persistence, analysis execution, and chat-session management. These helpers build endpoint paths and request bodies, then delegate network behavior to a single HTTP function. This structure keeps route strings, response interpretation, and authentication recovery out of visual components.

Base URL and Request Construction

The API base URL depends on the build environment. Production builds use the deployed Railway backend. During development, iOS uses the local host, Android or a physical Expo device derives the development machine address from the Metro host when possible, and the Android-emulator fallback uses its host mapping. Centralizing this decision prevents screens from embedding environment-specific addresses.

For each request, the shared client reads the access and refresh tokens from SecureStore. It constructs JSON headers and adds the bearer access token when present. The feature helper then receives the parsed Stockrium response and converts its error code and data into a compact success/failure shape appropriate for the calling screen.

Automatic Token Refresh

An HTTP 401 Unauthorized response triggers coordinated token renewal. If no refresh is currently running, the failed request becomes the refresh leader and submits the stored refresh token to the authentication endpoint. Other requests that receive 401 during this interval subscribe to the same operation instead of issuing additional refresh

calls. If renewal succeeds, a new access token is saved and the subscribed requests retry using the new value, followed by the original leader request. Figure 4.3 illustrates this sequence.

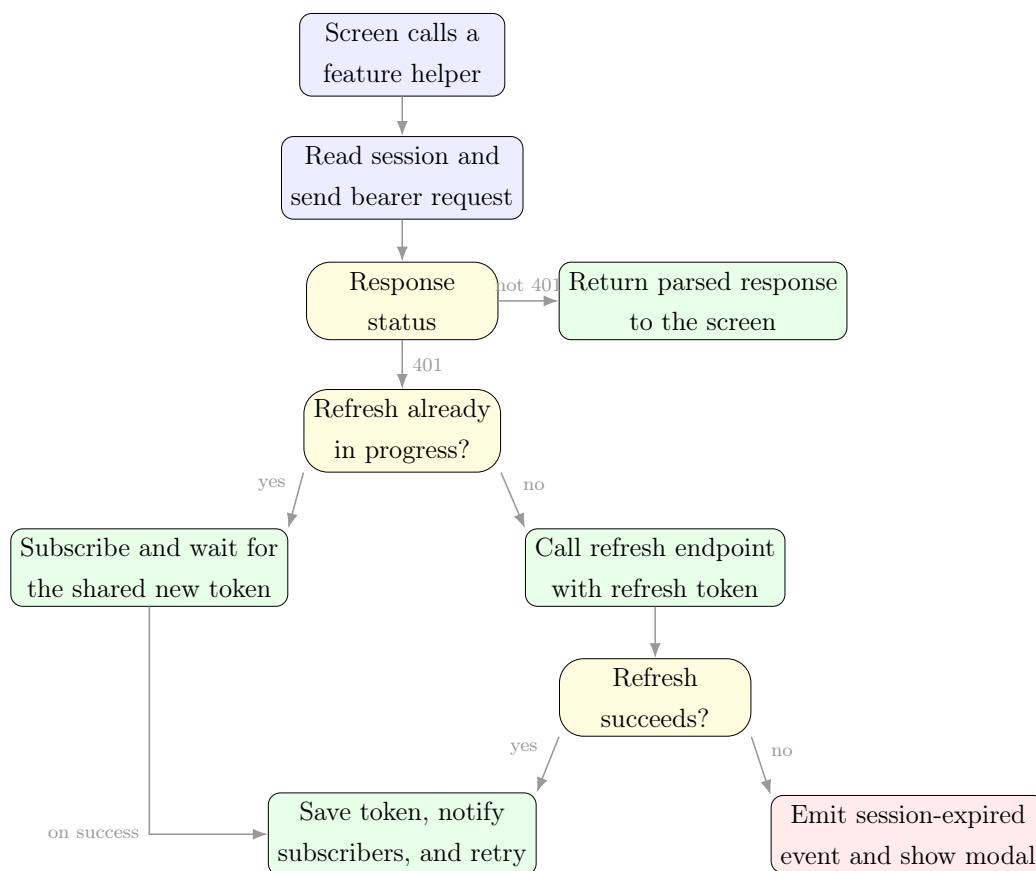


Figure 4.3: Coordinated access-token refresh in the mobile HTTP client

If refresh fails or returns incomplete token data, the client emits a global authentication-expired event. The modal registered in the root layout becomes visible, removes local session data after confirmation, and replaces the current route with Authentication. This event-based mechanism avoids passing an expiration callback through every component.

State Scope and Persistence

Stockrium uses local React state for screen-specific data, React context for a small number of application-wide preferences, SecureStore for persistent device state, and backend storage for user-owned domain state. A large global state framework is not required because market and detail screens can request their own independent data and because user-specific records such as watchlist entries and conversations already have authoritative server representations.

Table 4.3: Client-state scopes in the Stockrium mobile application

State scope	Mechanism	Examples
Screen-local transient state	React state, refs, memoization, effects, and a keyboard-visibility hook	Loading flags, fetched lists, chart type, selected time frame, active indicator, keyboard visibility, bottom-sheet visibility, search text, selected radar axis, and form validation.
Cross-screen presentation state	React Context	Current light/dark theme, selected language, and translated text lookup.
Persistent device state	Expo SecureStore	Access token, refresh token, onboarding completion, theme, language, and the saved AI-analysis preset.
Navigation state	Expo Router and the custom PagerView	Current stack route, route parameters, active main tab, and the set of tabs that have been visited and mounted.
Global transient events	Event emitters	Authentication expiration and programmatic switching of the custom tab shell.
Authoritative user data	Backend API and Supabase	Watchlist symbols, search history, investment horizon, account profile, chat sessions, and chat messages.

The Theme and Localization providers load their stored values before rendering children. This prevents a visible flash in which the application initially renders with the wrong theme or language. Theme changes and language selection are immediately reflected through context and persisted for the next launch. The AI configuration uses the same hydration principle: its saved mode, source toggles, indicator groups, and weights are read before subsequent changes are written, preventing the default state from overwriting an existing preset during initialization.

The manual analysis screen also maintains consistency among related controls. Submission is enabled only when at least one evidence source remains usable; an enabled technical or fundamental source must contain at least one selected subcategory; and changing enabled sources redistributes weights among the active choices. These client checks improve the request before submission, while the backend schema remains responsible for authoritative validation.

Overall, the mobile design keeps screen composition, visualization, session recovery, and user preferences close to the client while delegating business rules and authoritative data to backend services. File-based routes make task transitions explicit, domain components reduce duplication, and the shared HTTP layer gives every screen consistent

authentication behavior. The next section describes the server-side architecture that receives these client requests.

4.3 Backend Service Architecture

The Stockrium backend is the authoritative application boundary between the mobile and administrative clients and the project’s financial data, user records, and AI workflows. It is implemented as a FastAPI application served by Uvicorn. In addition to exposing HTTP endpoints, the backend validates input, establishes the authenticated user context, applies domain rules, normalizes heterogeneous data, invokes analytical workflows, and converts results into client-oriented response objects. This concentration of authority prevents the mobile application from directly accessing database credentials, market-provider credentials, or language-model keys.

The implementation is organized primarily into route, model, service, and utility or infrastructure-client modules. This organization is deliberately lightweight: the project does not introduce a repository class for every database table, while database and external-service access is primarily separated from presentation logic and grouped with the domain service that owns the corresponding operation. The resulting structure provides clear responsibility boundaries without adding abstractions that would contribute little to the current application size.

4.3.1 FastAPI Layered Design

FastAPI constructs the Hypertext Transfer Protocol (HTTP) interface from typed Python endpoint declarations and integrates Pydantic validation and OpenAPI schema generation [28]. Stockrium uses these capabilities to establish a layered request path. Figure 4.4 shows the logical direction of a normal backend call. A request enters through either the mobile application or the internal administrative web application. It then passes through cross-cutting HTTP behavior, an application programming interface (API) router, a typed model where applicable, and a domain service. The service can query persisted data, call an upstream provider, perform deterministic computation, or invoke an agent graph.

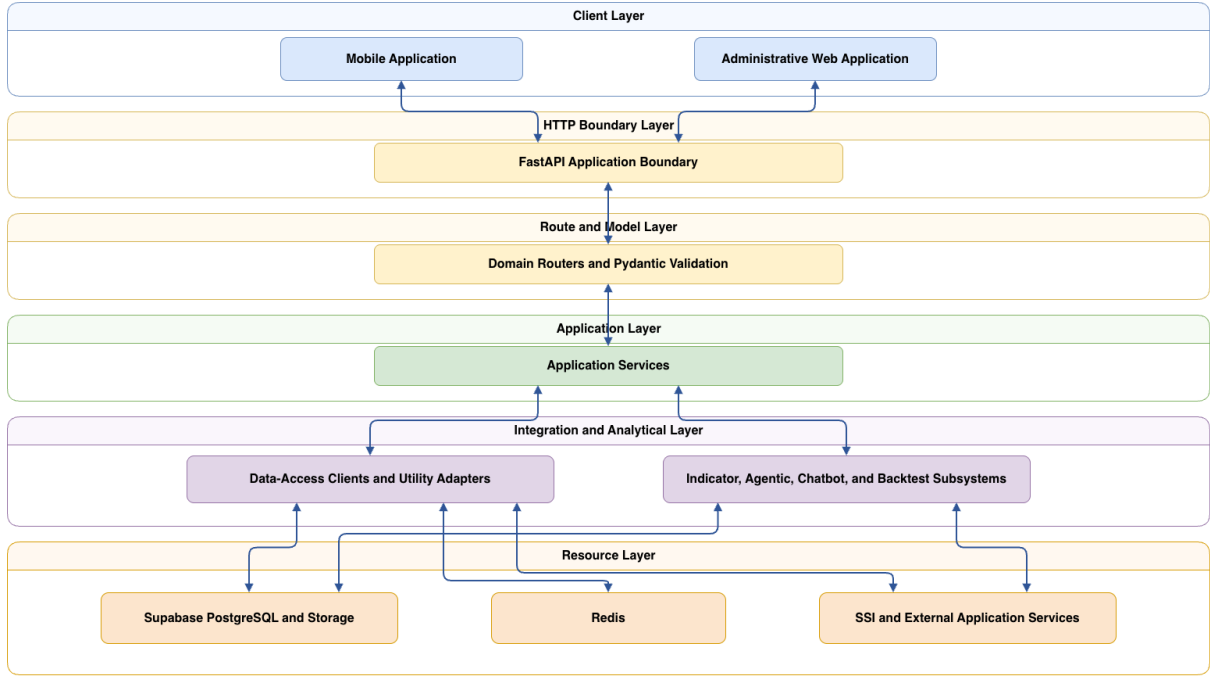


Figure 4.4: Layered design of the Stockrium backend service

The layers in Figure 4.4 have the following responsibilities:

- **Application boundary.** The FastAPI application registers routers, configures Cross-Origin Resource Sharing (CORS) for allowed browser origins, exposes health endpoints, mounts generated backtest visualizations, and installs common exception handlers. Configuration is loaded from typed environment settings so deployment-specific addresses and credentials are not embedded in route code.
- **Route layer.** Route modules define URL paths, HTTP methods, query and body parameters, access dependencies, and response construction. They remain relatively thin: a route extracts validated values, calls a service, and translates a domain or infrastructure failure into an HTTP result.
- **Model layer.** Pydantic classes describe input and output structures. Once validation succeeds, services receive typed objects rather than unstructured request dictionaries [29]. These models are particularly important for configurable artificial intelligence (AI) analysis, manual weights, investment-horizon records, authentication, and backtest parameters.
- **Service layer.** Services implement application operations such as obtaining a price series, searching stocks, retrieving financial statements, managing a watchlist entry, calculating technical indicators, running an analysis graph, or maintaining a chat session. A service may combine several tables or providers into one response suited to a user task.
- **Utility and infrastructure layer.** Supabase clients, the direct PostgreSQL connection pool, Redis session operations, SSI FastConnect market-data access,

email and Google Identity utilities, model software development kits (SDKs), and Supabase Storage helps isolate protocol-specific interactions from HTTP routing.

This flow is visible in a technical-indicator request. The protected route first obtains the current user from the authentication dependency and validates the requested interval. The market service loads an ordered price series for the symbol, and pandas represents and transforms it as a DataFrame. The technical-indicator service uses the Technical Analysis Library (TA-Lib) to calculate the simple moving average (SMA), exponential moving average (EMA), relative strength index (RSI), moving average convergence divergence (MACD), Bollinger Bands, and stochastic KDJ oscillator. Non-finite warm-up values are converted to `null` so that the result is valid JavaScript Object Notation (JSON). Finally, the route aligns every indicator value with its trading date and time and returns records suitable for chart rendering. The route does not contain the formulas, and the indicator service does not handle bearer tokens or HTTP status codes.

An AI-analysis request follows the same outer structure but changes at the service boundary. The route validates the analysis configuration and passes the symbol, mode, investment horizon, data-selection switches, and optional weights to the agentic service. The service creates the initial graph state and invokes a compiled LangGraph workflow. Consequently, the HTTP layer is independent of the internal number of specialist agents; later changes to graph routing do not require the mobile request contract to be redesigned.

4.3.2 Domain API Modules

Routers are divided by application domain instead of by screen. A stock-detail screen can therefore compose calls from the market, company, article, fundamental, and technical modules, while the same modules can be reused by other screens and analytical workflows. Tables [4.4](#) and [4.5](#) summarize the modules that belong to the current application.

Table 4.4: Active backend API modules and responsibilities: core data services

Module	Representative interface	Principal responsibility	Main dependency
Authentication	/api/auth/*	Account creation, email verification, password or Google sign-in, token refresh and revocation, password recovery, and current-user retrieval	Supabase, Redis, Google, Resend
Market and discovery	/api/all-stocks /api/price/{symbol} /api/market-index /api/investing-idea	Symbol discovery, current and historical prices, industry movement, index summaries, impact stocks, and deterministic investment-idea lists	Supabase and SSI
Company and fundamentals	/api/company/* /api/fundamental-analysis/*	Company profile, leaders, subsidiaries, financial statements, ratios, stored summaries, and industry aggregates	Supabase
Technical analysis	/api/technical-indicators/{symbol}	Load price candles, calculate supported technical-indicator series, align timestamps, and serialize chart-ready values	Market service, pandas, and TA-Lib
Articles	/api/articles/*	Latest, macroeconomic, business, highlighted, category-specific, and stock-specific news retrieval	Supabase

Table 4.5: Active backend API modules and responsibilities: user and operational services

Module	Representative interface	Principal responsibility	Main dependency
Personalization	/api/favorite /api/search-history /api/risk-appetite	User-owned watchlist entries, recent search records, and the selected investment horizon	Supabase and user dependency
Analysis and chat	/api/agentric/analyze /api/agentric/chat /api/agentric/chat/sessions	Structured stock analysis, stateful conversation, session seeding, history retrieval, and session deletion	LangGraph, model providers, PostgreSQL
Administration and evaluation	/api/agentric/admin-analyze /api/agentric/admin-universe/* /api/agentric/backtest /api/agentric/backtests	Role-protected single-symbol or universe analysis, historical pipeline evaluation, and result-artifact listing	Agent graph, backtest services, Supabase Storage
Operations	/health /visualizations/*	Basic process health reporting and delivery of generated backtest visualizations	FastAPI and local artifact directory

Authentication is applied through FastAPI dependencies at two levels. Routers containing exclusively protected information—including market, company, fundamental, technical, and article endpoints—declare the current-user dependency once for the entire router. Mixed modules apply the dependency to individual operations. The user dependency verifies the access JSON Web Token (JWT) and also checks that the same token remains present in Redis. This second check enables immediate logout or revocation instead of accepting every cryptographically valid token until its expiry. Administrative operations compose an additional dependency that reads the user’s current role from the authoritative `User` table. A role change therefore takes effect without waiting for a new JWT.

Several services deliberately reuse other domain services. Technical calculation loads candles through the market service instead of implementing a second price-query path. Watchlist and search-history responses enrich stored symbols with profile and current-price information so the mobile client can render useful cards without issuing one request per symbol. Fundamental services resolve a symbol through the `Stock` relation before obtaining statements and ratios. The agent and chatbot tools also consume the same market, fundamental, technical, and article capabilities used by HTTP routes. Reuse helps keep a value such as the current price or a company identity consistent between a visible screen and an AI response.

4.3.3 Request Validation and Response Contract

Validation occurs before domain execution whenever an endpoint declares a Pydantic request model. Simple constraints include required fields, numerical bounds, enumerated values, and default settings. Cross-field validation is used when a field cannot be assessed independently. For example, manual AI-analysis weights for news, technical analysis, and fundamental analysis must each be between zero and one, contain no more than two decimal places, and total exactly one after rounding. Backtest parameters bound the maximum holding period, minimum signal score, and lookback window. These checks reduce the chance that invalid configurations reach an expensive model invocation or a long historical simulation.

Validation and normalization are not identical. Pydantic confirms the request shape, while domain services normalize values that carry business meaning. Stock symbols are converted to uppercase before lookup; universe names are stripped and normalized before comparison with VN30 and VN100; and Vietnamese chat messages are converted to Unicode Normalization Form C (NFC) before entering the chatbot. The Unicode step is important because visually identical Vietnamese strings can otherwise have different underlying code-point sequences, affecting tokenization, checkpoint content, and comparison behavior.

Client-facing domain endpoints are expected to follow the response envelope shown

in Table 4.6. The envelope separates transport success from the domain payload and gives both clients a stable place to inspect errors. A new universally unique identifier (UUID) is generated for each constructed response. Although the present version does not yet propagate this request identifier through a complete distributed tracing system, it provides a correlation value that can be included in logs and support reports.

Table 4.6: Standard API response envelope

Field	Type	Meaning
data	object, array, or scalar	Requested payload on success; normally an empty object when a failure has no safe payload.
errorCode	integer	Zero for a successful operation; a domain-specific nonzero code for failure.
errorDesc	string	Empty on success; a client-readable validation, authorization, lookup, or service error description on failure.
requestId	UUID string	Identifier assigned to the constructed response for correlation and diagnosis.
result	boolean	Indicates whether the requested application operation succeeded.

The global validation handler intentionally reports HTTP status 400 rather than FastAPI’s default 422 response and assigns error code 400003. It extracts the first validation failure to keep messages concise for mobile presentation. The common HTTP-exception handler maps frequently used statuses to stable application codes: malformed requests to 400001, unauthenticated requests to 401001, forbidden requests to 403001, missing resources to 404001, semantically invalid requests to 422001, and server failures to 500001. Route-specific response construction may override this general mapping when a more precise domain translation is required. For example, requesting another user’s chat session produces a forbidden response, while requesting a nonexistent owned session produces a not-found response.

The response contract also defines a useful client rule: the HTTP status is used for transport and protocol behavior, while **result**, **errorCode**, and **errorDesc** drive application feedback. The mobile HTTP helper can consequently handle authentication failure centrally and still allow a screen to display a domain-specific empty state. Operational endpoints such as `/health`, the root status route, and mounted static visualization files are intentionally outside the domain envelope because they are consumed as process checks or files rather than application records.

4.3.4 Backend Lifecycle, Concurrency, and Failure Containment

The backend mixes ordinary data retrieval, asynchronous session operations, numerical indicator calculation, and comparatively slow external model calls. Its lifecycle and concurrency controls therefore aim to reuse expensive resources while bounding fan-out.

The FastAPI process constructs the analysis graph and chatbot graph once when their service module is loaded. A request invokes these compiled graph objects instead of rebuilding nodes, prompts, and persistence configuration for every call. Chat history uses LangGraph checkpoint persistence backed by PostgreSQL [32]. The shared PostgreSQL connection pool is configured with deferred opening and initialized while the chatbot session service ensures the required session table. It maintains between one and ten connections and uses autocommit settings compatible with the Supabase pooler. FastAPI’s lifespan handler closes this pool during normal shutdown; an additional process-exit hook covers development shutdown paths in which the Asynchronous Server Gateway Interface (ASGI) lifespan may not complete.

Concurrency is introduced only at bounded points. A normal single-stock request invokes one analysis graph, within which the orchestrator can select and fan out the required specialist branches. Administrative VN30 or VN100 analysis uses a thread pool with four workers. Each worker runs the complete analysis for one symbol, and each future returns either a recommendation or a symbol-specific error record. A temporary provider or data failure for one constituent therefore does not discard successful results for the remainder of the basket. The small fixed pool also prevents an administrative action from issuing dozens of simultaneous language-model calls.

Endpoint definitions use both synchronous and asynchronous functions according to the underlying work. Authentication dependencies and Redis validation are asynchronous because they wait on session input/output (I/O). Many domain services use synchronous Supabase clients or perform numerical work, so their routes are declared as regular functions and can be executed by FastAPI without occupying the event loop as an asynchronous function that performs blocking work would. This arrangement is suitable for the current scale, although very long AI and backtest operations still occupy a request for their full duration. A future production-hardening step could move these operations to a durable job queue and expose progress polling, cancellation, and retry semantics.

Background ingestion is deliberately not scheduled inside the web-process lifecycle. The article updater runs as a separate Railway service, and market-data updaters run as independent worker processes. This separation prevents a weekly news crawl or a continuous market stream from delaying API startup, consuming the web server’s request workers, or being duplicated whenever Railway restarts or horizontally scales the API service. The API reads the resulting normalized records from Supabase, while each updater owns its own retry and scheduling behavior.

Failure containment is applied at multiple boundaries. Pydantic rejects malformed requests before service invocation; authentication dependencies stop unauthorized access before domain queries; service methods normalize missing or non-finite data; graph state exposes an error field that the agentic service converts into a runtime failure; and batch analysis records per-symbol exceptions. Global exception handlers then translate uncaught HTTP and validation failures into the common response structure. Finally, the public health route offers a lightweight indication that the FastAPI process is responsive. It is intentionally a liveness check rather than a complete readiness audit: it does not guarantee that Supabase, Redis, SSI, or an AI provider is currently available.

In summary, the backend architecture gives Stockrium one typed and authenticated boundary over heterogeneous data and analytical components. Domain routers define a coherent API, services concentrate reusable application logic, adapters manage infrastructure-specific communication, and lifecycle controls reuse graph and database resources safely. These foundations allow the following sections to describe the agentic workflow and data-processing mechanisms without coupling them to client or HTTP implementation details.

4.4 Data Architecture and Acquisition Pipelines

Stockrium depends on data with substantially different lifecycles. User preferences change through application requests; prices change throughout a trading session; financial statements are collected periodically through administrative tools; articles are refreshed weekly; and chatbot checkpoints grow one conversation turn at a time. The data architecture therefore separates persistent relational records, generated file artifacts, short-lived security state, and device-local credentials. It also separates acquisition work from interactive API execution: the mobile application reads normalized data through the backend but does not start a Selenium crawler, an SSI stream, or a news-collection job.

4.4.1 Supabase Data Model

Supabase PostgreSQL is the principal system of record. PostgreSQL is exposed through Supabase rather than replaced by a proprietary document model, allowing the application to use keys, constraints, relations, pagination, and conflict-aware upserts [33]. Stockrium organizes its application tables into the groups summarized in Table 4.7. The names shown are the case-sensitive names used by the current backend; quoted identifiers are therefore retained when direct SQL is generated for the chatbot.

Table 4.7: Principal relational-table groups in Stockrium

Data group	Tables and responsibility
Account and personalization	User stores account identity, credential metadata, and role. RiskAppetite stores the selected investment horizon. Favorite implements the user’s stock watchlist. Search_History stores at most six recent unique stock searches per user.
Market data	Stock is the canonical security directory. Stock_Price_1m and Stock_Price_1d store OHLCV candles. Current_Stock_Price is a latest-quote read model. MarketIndex defines index baskets, Current_Market_Index stores the most recent index snapshots, and Stock_MarketIndex associates securities with indices.
Company and fundamentals	BI_Profile , BI_Leader , and BI_Subsidary store company description, management, and subsidiary or associate records. FA_Summary , FA_Indicator , FA_BalanceSheet , FA_CashFlow , and FA_IncomeStatement store company-level financial evidence. FA_Industry_Aggregate stores precomputed annual industry medians and peer counts.
News and classification	Article stores extracted article content, metadata, summary, and sentiment. Article_Stock and Article_Category associate articles with securities and categories. Category defines classification groups used for stock and article organization, while Category_Stock associates categories with securities.
Chatbot persistence	chat_sessions records session ownership, title, and timestamps. LangGraph’s checkpoints , checkpoint_blobs , and checkpoint_writes tables persist graph state and message data, while checkpoint_migrations tracks the checkpointer schema version.

The central entity is **Stock**. Company and financial tables use **stock_id** to associate records with a listed security. Annual financial records are organized as stock–year observations. **FA_Summary** and **BI_Profile** provide stock-level text or profile information, while one company can have multiple leader and subsidiary records. The price tables use (**symbol**, **trading_time**) as their composite identity, and **Current_Stock_Price** maintains the latest record for each symbol. **MarketIndex** defines the supported indices, **Stock_MarketIndex** stores their constituents, and **Current_Market_Index** stores the corresponding current market state.

Account data is separated from market data. **Favorite** records the stocks selected by each user for later monitoring, while **Search_History** uses the same user and stock references and retains the six most recent unique stocks. **RiskAppetite** stores one current investment-horizon profile for each user. These relations allow the backend to enrich lightweight personalization records with company profiles and current prices without

duplicating market data in user-owned tables.

Four association tables represent many-to-many relationships. `Stock_MarketIndex` records index constituents, `Category_Stock` records the classifications to which a stock belongs, and `Article_Stock` and `Article_Category` associate one article with multiple securities or categories without copying its content. `FA_Industry_Aggregate` is identified by `(category_id, year)` and uses the ICB category derived from `BI_Profile.icb_code` to store annual peer medians for comparison with company indicators.

Chat metadata and conversational content are deliberately separated. `chat_sessions` associates each conversation with its user, title, and timestamps. LangGraph’s PostgreSQL saver uses the session identifier as its `thread_id` and stores message-bearing graph states in the checkpoint tables [32]. Deleting a conversation removes its checkpoint writes, blobs, checkpoints, and session record, while `checkpoint_migrations` remains framework metadata.

Figure 4.5 presents the reduced entity–relationship view used in this report. It displays the principal table names and their relationships while omitting columns and framework metadata to preserve legibility.

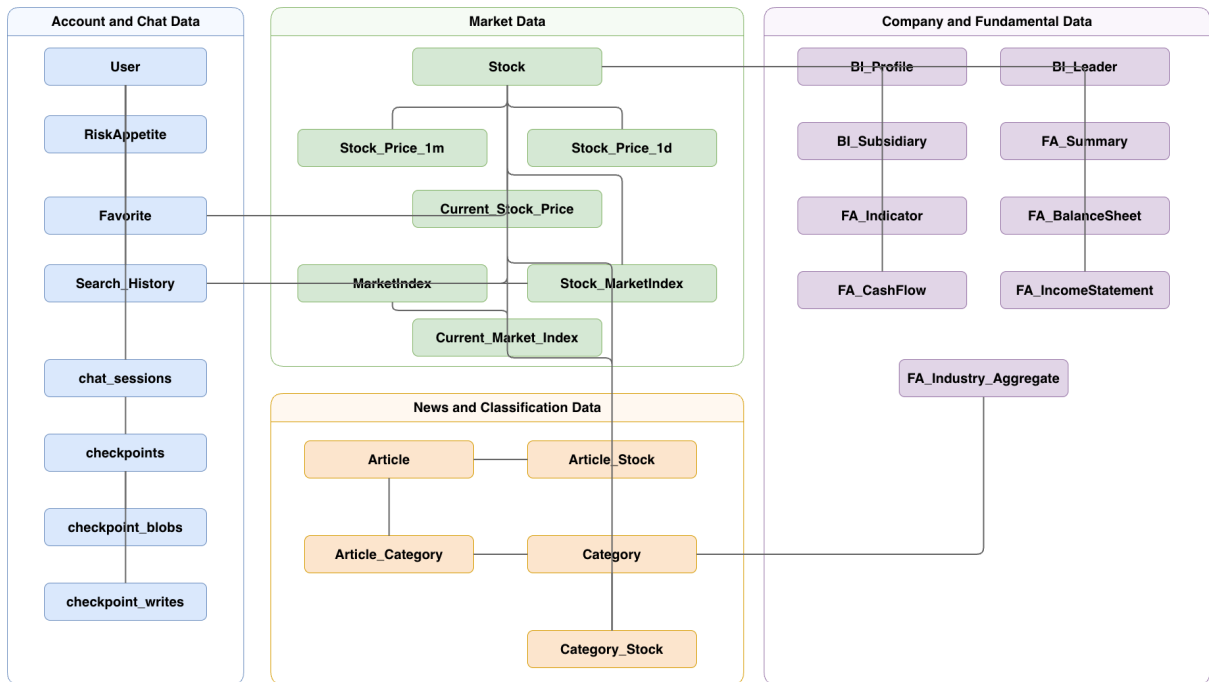


Figure 4.5: Reduced ERD of the principal Stockrium data relationships

4.4.2 Market Price Acquisition Pipeline

The `update_stock_price/` subsystem supports two acquisition modes: historical initialization and daily live operation. Initialization scripts obtain an SSI access token and backfill OHLCV data for stocks in the configured VN-Index universe and for five

retained market-index symbols. Daily initialization is divided into 30-day requests because of the upstream API limit and currently loads approximately five years for daily candles. One-minute initialization retrieves the latest 30 days. Rows are normalized into a common `symbol`, `trading_time`, `open`, `high`, `low`, `close`, and `volume` representation, deduplicated, and upserted on (`symbol`, `trading_time`). Requests are paced by approximately 1.1 seconds to reduce SSI rate-limit pressure.

Live acquisition is controlled by a long-running scheduler using Vietnam time (UTC+7). On Monday through Friday, it ensures that three child processes are running from 09:00 inclusive until 15:00 exclusive: a one-minute stock-price stream, a daily stock-price stream, and a market-index stream. The scheduler polls every 30 seconds and restarts a child process if it has exited unexpectedly. At 15:00, it sends an interrupt signal to each stream, waits for a graceful shutdown, and terminates a process only if it does not stop within the timeout.

The one-minute worker subscribes to SSI bar messages, restricts accepted symbols to the configured stock universe and retained indices, and aggregates incoming ticks in an in-memory candle buffer. The key is the pair of symbol and minute. The first tick establishes open, high, low, close, and volume; later ticks update the high and low, replace the close, and accumulate volume. Closed minutes are flushed from a separate loop so network I/O to Supabase does not block the SSI callback. The current open minute is not written during shutdown because it is incomplete.

The daily worker subscribes to the same bar stream but accumulates one current-session record per symbol in `Stock_Price_1d`. Index-bar history is also stored in the 1-minute and daily price tables using the index identifier as the symbol, while index-wide breadth and turnover snapshots arrive through the SSI MI stream and are upserted into `Current_Market_Index` by `index_id`. Historical index candles and current index state are therefore separate representations with different query purposes.

At or after 15:05, the scheduler runs three reconciliation jobs exactly once for the date and, importantly, runs them sequentially: one-minute candles, daily candles, and market indices. The one-minute job re-fetches a short two-day lookback and retains 30 days of minute data. The daily job reconciles a three-day lookback and retains five years. The index job retrieves the end-of-day state for the supported indices. Sequential execution prevents three full-universe jobs from making SSI requests simultaneously and exceeding provider limits.

Figure 4.6 summarizes the market-data flow.

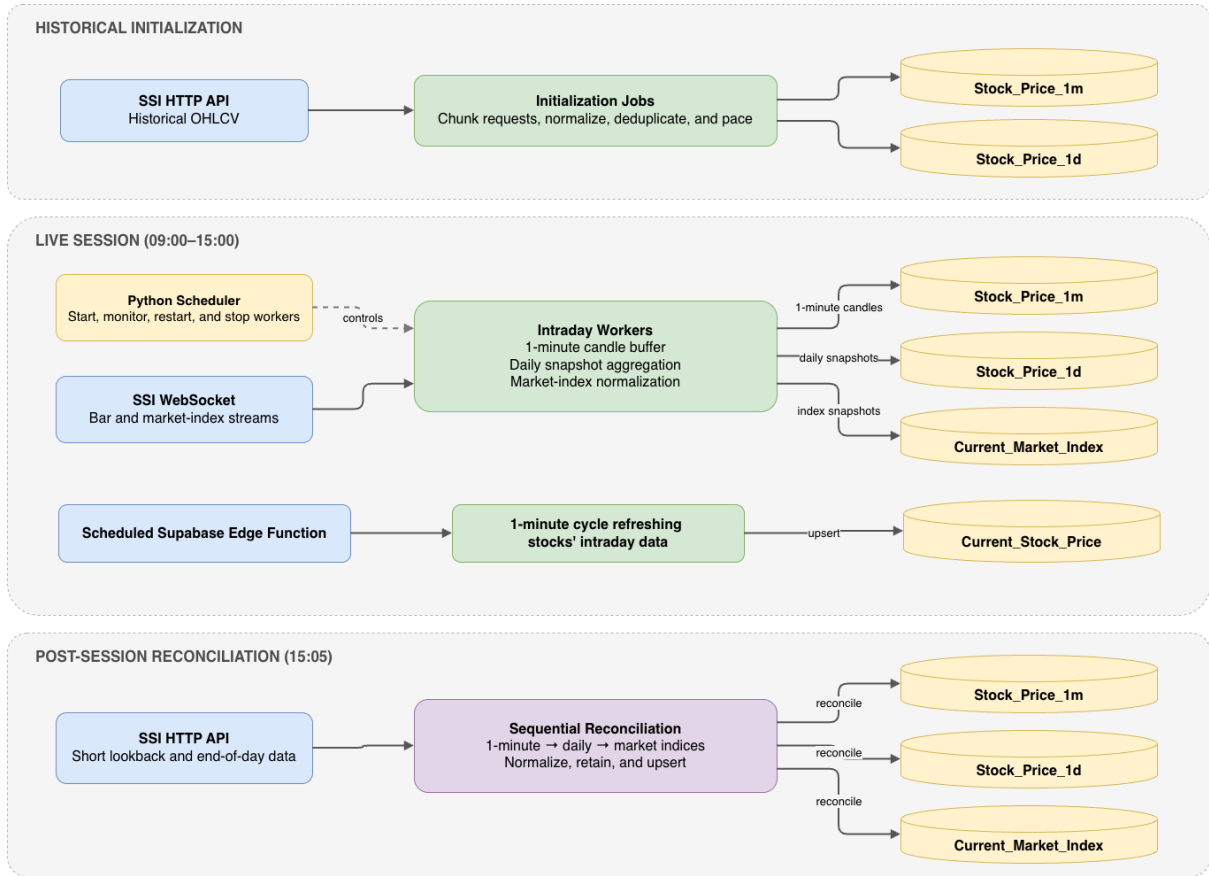


Figure 4.6: Historical, live, and post-session market-data acquisition

The **Current_Stock_Price** table remains important to market lists, favorites, search history, and current-price cards. It is refreshed by scheduled Supabase Edge Function jobs rather than by the Python processes in `update_stock_price/`. This design makes the table a separately maintained latest-quote read model: the Python scheduler owns candle and index-stream persistence, while the managed database-side schedule owns current stock snapshots. Deployment documentation must therefore include both schedules; deploying only the Python service would leave current-price cards dependent on the existing Supabase job.

The scheduler's trading-day test currently recognizes weekdays rather than an exchange-holiday calendar. On a weekday market holiday it may start streams and later execute reconciliation even though no session occurred. Upserts and empty-response handling reduce the risk of corrupting stored candles, but a production version should consume an official Vietnamese exchange calendar.

4.4.3 Company and Fundamental Data Pipeline

Company and financial acquisition is an administrative, offline process. It is not invoked from a mobile request because browser automation, file downloads, report parsing,

and multi-company imports are too slow and operationally fragile for an interactive API path. The backend serves only data that has already been normalized and stored.

Company-profile collection uses Selenium WebDriver to open SSI iBoard, establish an authenticated browser session, search for a stock, enter dynamically loaded page regions, and parse company information. The crawler obtains profile attributes, leadership records, and subsidiaries or associates. A stock is first resolved through **Stock**; profile fields are updated in **BI_Profile**. Leadership and subsidiary collections are replaced for the stock by deleting their previous rows and inserting the newly observed records. The crawler supports batch operation from symbols already present in **BI_Profile** and maintains progress so a long run can resume after a browser or page failure. The use of Selenium for a dynamically rendered source follows the browser-automation rationale introduced in Chapter 2 [36].

A separate SSI financial downloader navigates the fundamental-analysis interface and triggers downloads for the balance sheet, income statement, cash-flow statement, and financial-indicator sections. Excel files are converted where necessary, and pandas transforms the downloaded wide layouts into tidy records containing one symbol and financial year per row. Processing removes unnamed or empty columns, normalizes Vietnamese labels into stable database fields, converts numerical values, drops empty trailing years, and serializes missing values as database nulls. The current application schema associates the processed records with **Stock.id** and stores them in the corresponding **FA_*** tables. Upserts use the stock-year identity so rerunning an import corrects or completes existing observations instead of blindly duplicating them. pandas and its workbook engines are suited to this conversion of heterogeneous tabular reports [37].

Industry aggregation is performed after company profiles and financial indicators are available. The utility maps detailed **BI_Profile.icb_code** values to an ICB industry identifier, groups **FA_Indicator** records by industry and year, and computes the median of every available metric while ignoring missing values. It also counts the contributing stocks. Results are upserted into **FA_Industry_Aggregate** on **(category_id, year)**. The current utility retains the configured 2021–2025 range and removes obsolete years or invalid category identifiers. Precomputation moves repeated peer aggregation out of user requests and lets the fundamental agent compare a company with a consistent industry benchmark.

Fundamental summaries are also generated as an offline maintenance operation rather than during a mobile request. The active backend reads **FA_Summary** by **stock_id**. The OpenAI batch utility performs multi-year evidence retrieval, retry, pacing, and skip-existing behavior before persisting the generated text. Its source code retains the earlier name **fundamental_analysis_summary** and earlier lower-case financial-table names; these identifiers refer to the predecessor schema that was later renamed to the active **FA_*** tables. They are treated as one logical dataset in this report, with **FA_Summary** used consistently

as the current name.

Figure 4.7 shows the separation between acquisition, transformation, enrichment, and serving.

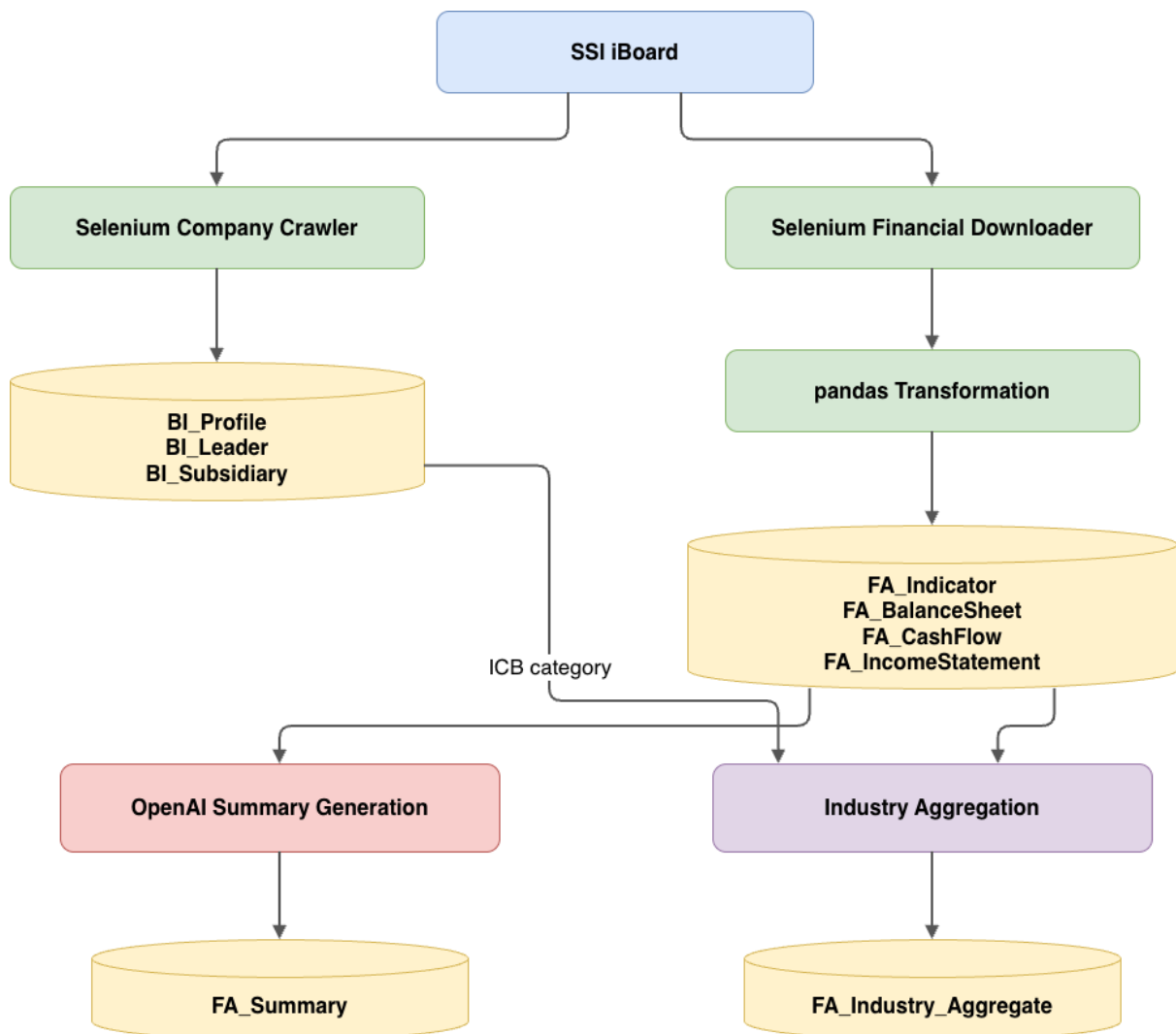


Figure 4.7: Offline company and fundamental-data preparation pipeline

4.4.4 Financial News Pipeline

The `update_articles/` subsystem separates initial collection from periodic refresh. Initial scripts backfill stock-specific, macroeconomic, and category-level news over a longer search horizon and include checkpoint files for resuming multi-page collection. Update scripts request only the latest week and perform one current search page for each configured topic, category, or stock. This distinction reduces the cost of routine updates while preserving a path for initializing a new deployment.

For each target, the pipeline submits a Vietnamese news query to the Serper news endpoint. A result supplies candidate metadata and a source URL. Newspaper3k then

downloads the selected page and extracts the article title, body, publication information, and representative image when available. Failed or empty pages are skipped because an inaccessible URL should not produce an article containing only generated text.

The extracted article is passed to `gpt-4o-mini` with a typed response schema. Stock queries request a short Vietnamese summary and a positive, neutral, or negative label from the perspective of the named security. Category queries use the corresponding industry perspective, while macroeconomic articles receive a general market label. Invalid labels are normalized to neutral before insertion. The original content and source metadata remain in `Article`, so the summary does not replace the source evidence.

Duplicate prevention occurs at two levels. Existing links are loaded before extraction so a known URL can be skipped without another model call. The insertion path checks again before creating the article. After an article exists, the pipeline checks the relevant join table before inserting `Article_Stock` or `Article_Category`. The same article can consequently be reused across associations without duplicated content or duplicate join rows. General macroeconomic articles can remain unassociated with an individual security.

The production scheduler uses Vietnam time and executes once per ISO week on Saturday at 23:00. It runs `update_macro_articles.py`, `update_category_articles.py`, and `update_stock_articles.py` sequentially, waiting for each process to finish. A process-local week key prevents a second execution during the same uninterrupted scheduler run. Even if a service restart causes a repeated attempt, link-level duplicate checks make the update substantially idempotent.

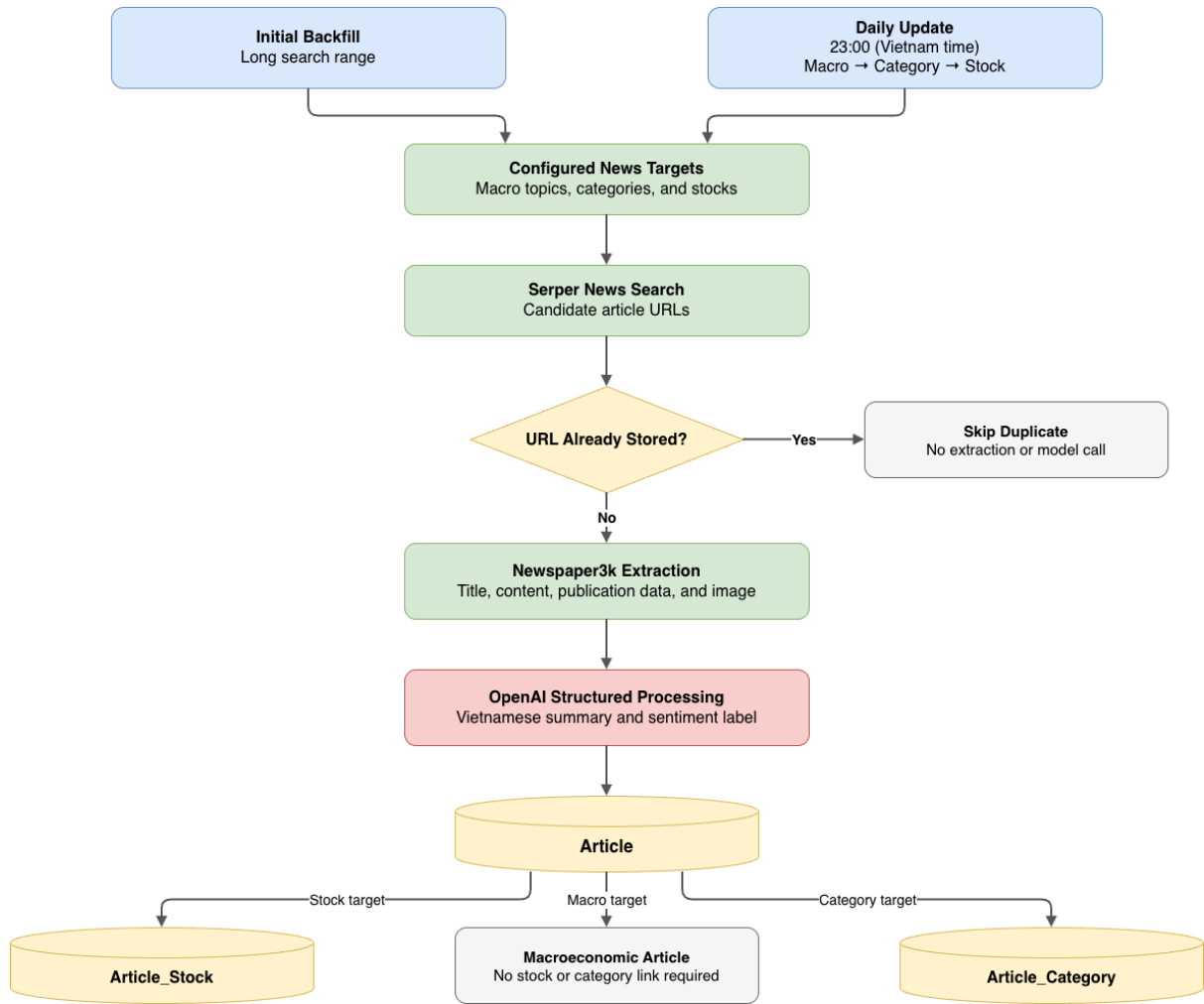


Figure 4.8: Initial and weekly financial-news processing flow

The article updater is deployed as a separate Railway service rather than as an in-process FastAPI job. This separation prevents a crawl and a sequence of model calls from delaying API requests or being duplicated for every web-server worker. It also allows the updater container to install the native parsing dependencies required by Newspaper3k independently of the main backend image.

4.4.5 Storage Responsibilities

Not every value belongs in the relational database. Table 4.8 distinguishes the four storage mechanisms used by Stockrium and clarifies which copy is authoritative.

Table 4.8: Storage responsibility boundaries

Storage	Stored information	Responsibility and lifecycle
Supabase PostgreSQL	Accounts, personalization, security directory, market candles and snapshots, company profiles, financial records, industry aggregates, articles, classifications, chat-session metadata, and LangGraph checkpoints	Authoritative durable relational data. Accessed through Supabase PostgREST for domain services and through pooled PostgreSQL connections for checkpointing and controlled chatbot queries.
Supabase Storage	Manually uploaded JSON backtest reports in the backtests bucket	Durable object artifacts that are listed by the backend and opened by the administrative dashboard. The storage helper supports replacement upload and public-URL generation, but the current backtest execution route does not upload automatically.
Redis	Access token, refresh token, verification or reset OTP, attempt counter, resend cooldown state, and one-time reset-session mappings	Expiring server-side security state. Enables token revocation and automatic removal through TTL; it is not a source of permanent user or financial records [34].
Expo SecureStore	Mobile access token and refresh token; also small local preferences such as onboarding completion, theme, language, and saved analysis configuration	Device-local encrypted key-value state. Tokens are removed when refresh fails or the user signs out. The backend remains authoritative for accounts and sessions [35].

Backtest execution returns its result to the administrative workflow, where the JSON artifact can be inspected or exported. Upload to the **backtests** bucket is a separate manual administrative action. Once an artifact has been uploaded, the backend storage helper lists the available JSON files and produces the public URLs used by the dashboard. This distinction prevents the presence of an upload helper from being mistaken for an automatic persistence step in every backtest request.

Redis key namespaces reflect these responsibilities. Access and refresh sessions use `session:access:{user_id}` and `session:refresh:{user_id}` with configurable token lifetimes. OTP values use `otp:{purpose}:{email}`, while `otp:attempts:{purpose}:{email}` counts failed verification attempts under the same expiry. The default OTP validity is ten minutes with a maximum of five attempts. A successful password-reset verification produces a five-minute one-time reset session and an email-to-session reverse mapping so an older session can be invalidated.

The mobile application stores the access and refresh tokens under `access_token` and `refresh_token` in SecureStore. It does not store a durable copy of favorites, risk horizon, chat history, or financial records as an authoritative local database. Those values are requested from the backend. This boundary prevents a device preference or stale cache from being mistaken for a confirmed server-side state.

The acquisition pipelines and storage boundaries together provide a stable evidence layer for the analytical components. Live and offline processes normalize heterogeneous sources before they reach application services; relational links preserve stock, industry, article, and user context; and short-lived secrets remain outside durable business tables. The next section builds the agentic stock-analysis design on top of these prepared data products.

4.5 Agentic Stock Analysis Design

The principal analytical feature of Stockrium is implemented as a stateful directed graph rather than as one monolithic language-model prompt. The graph separates planning, evidence preparation, specialist analysis, synthesis, and deterministic post-processing. This separation is important because the three evidence domains have different structures: price series require numerical calculation, financial statements require multi-year and industry-aware comparison, and articles require relevance filtering and qualitative interpretation. Treating them as independent branches also makes it possible to disable selected evidence, diagnose a weak branch, and reuse the same components in live analysis and historical evaluation.

LangGraph provides the state-graph abstraction used to declare nodes, edges, shared state, and parallel branches [21]. The language model is used where semantic interpretation or natural-language synthesis is required, while Python functions retain control over interval selection, indicator calculation, technical scoring, confidence calculation, veto conditions, and price-boundary enforcement. The resulting design is therefore hybrid: it benefits from the explanatory ability of a language model without assigning every numerical or safety-sensitive decision to generated text.

4.5.1 Graph Topology and Shared State

The compiled analysis graph contains five nodes: an orchestrator, three specialist branches, and an aggregator. Figure 4.9 presents the execution topology. After the orchestrator creates a plan, LangGraph activates the article, fundamental, and technical nodes from the same state. Each branch writes its own entry into the shared result dictionary. The dictionary uses a merge reducer, so branch updates are combined rather than allowing the last completed branch to replace earlier results. The aggregator is

executed only after the specialist branches have reached their outgoing edges.

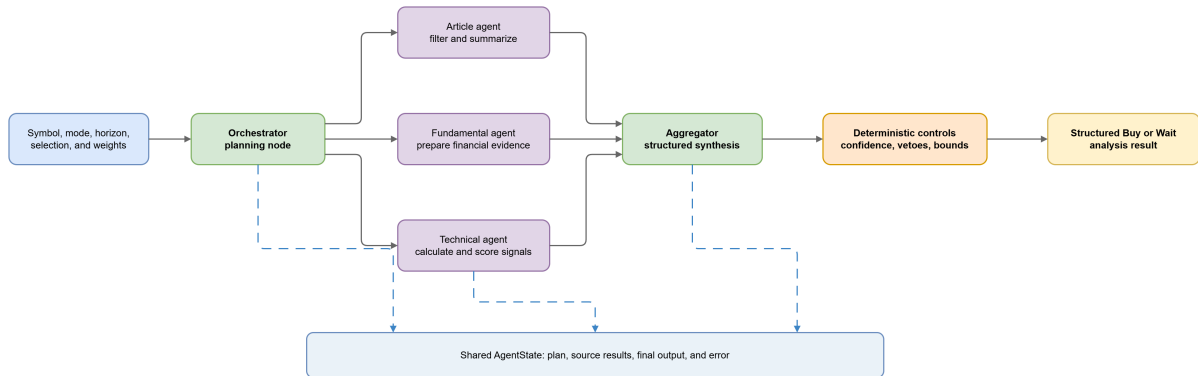


Figure 4.9: Execution topology of the Stockrium analysis graph

The shared **AgentState** is a typed dictionary containing the fields summarized in Table 4.9. State is the contract between graph nodes: a node reads only the context it requires and returns only the fields it changes. The graph can consequently add or replace a specialist without redesigning the public HTTP request.

Table 4.9: Principal fields in the analysis graph state

Field	Producer	Purpose
<code>mode</code>	API service	Distinguishes automatic and manual analysis; the state type also permits reuse by chat-related execution.
<code>user_input</code>	API service	Vietnamese task statement supplied to planning and final synthesis.
<code>risk_appetite</code>	API service	Contains the user’s selected investment horizon, which drives time scale, default weights, and numerical bounds.
<code>symbol</code>	API service	Identifies the Vietnamese listed stock being analyzed.
<code>data_selection</code>	API service	Contains enabled news, technical indicators, fundamental groups, and optional manual weights.
<code>plan</code>	Orchestrator	Supplies date ranges and the deterministic technical interval to specialist nodes.
<code>agent_results</code>	Specialist nodes	Merged dictionary containing separate article, fundamental, and technical evidence.
<code>final_output</code>	Aggregator	Structured recommendation, scores, confidence, explanations, and conditional risk values.
<code>error</code>	Any terminal failure	Carries a pipeline error back to the application service.

Automatic and manual requests enter the same graph. In automatic mode, the application service replaces the submitted selection with an empty configuration; selection normalization interprets every missing switch as enabled and applies horizon-based default weights. In manual mode, the user’s switches and weights remain in the state. All three graph branches are still reached structurally, but a disabled branch returns a clear disabled-source marker instead of performing unnecessary data interpretation or model work. This approach keeps the topology stable while making behavior configurable.

4.5.2 Orchestration and Horizon-Aware Planning

The orchestrator converts the request and horizon into a structured plan. It calls `gpt-4o-mini` with temperature zero and parses the response into a Pydantic

OrchestratorPlan. Structured output is preferable to parsing arbitrary prose because both the article and technical branches require valid YYYY-MM-DD boundaries [31]. The prompt defines a recent lookback of approximately 60 days for short-term analysis, 180–365 days for medium-term analysis, and at least 730 days for long-term analysis. The end date is the current date.

The model does not choose the candle interval. After parsing the generated dates, Python overwrites the technical interval according to the normalized investment horizon. Short-term requests use daily candles, medium-term requests use weekly candles, and long-term requests use monthly candles. If the horizon is absent or unrecognized, the system defaults to the medium-term configuration. This deterministic mapping prevents prompt variation from assigning inconsistent analytical scales to users with the same horizon.

The fundamental branch is intentionally not given a horizon-specific number of financial years. It examines up to the five latest annual periods regardless of whether the user selected a short-, medium-, or long-term profile. This asymmetry is deliberate: the horizon changes the sensitivity of price signals, but company quality should not be inferred from a single recent accounting period. The aggregator later changes the relative influence of the fundamental evidence according to the horizon.

4.5.3 Specialist Evidence Agents

The specialist nodes do not all use the language model in the same way. The technical node is primarily deterministic, the fundamental node prepares a structured textual evidence package without making the final recommendation, and the article node uses a model to filter and summarize natural-language news. Table 4.10 highlights these differences.

Table 4.10: Responsibilities and outputs of specialist analysis nodes

Node	Evidence	Processing	Branch output
Technical	Historical OHLCV candles and the most recent available price	Indicator calculation, latest-versus-previous comparison, deterministic binary scoring, and missing-value sanitation	Indicator values, explanations, current price, <code>total_score</code> , and <code>max_score</code>
Fundamental	Annual financial indicators, state-ments, company profile, ICB classification, and precomputed industry medians	Five-year ordering, CAGR, DuPont preparation, sector-specific metric selection, and industry comparison	Evidence text organized by financial category
Article	Stored articles related to the symbol within the planned dates	Date filtering, compact evidence construction, relevance screening, and model-based summary of benefits and risks	Vietnamese news summary or an explicit no-data marker

Technical-Analysis Agent

The technical agent first obtains the most recent one-minute price when live analysis is being performed. It separately loads candles in the horizon-selected interval and date range. Daily records are aggregated by the market service when weekly or monthly analysis is requested. The indicator service calculates moving averages, RSI, Bollinger Bands, MACD, and KDJ. During backtesting, the same node can use a historical indicator source and suppress current-price lookup so that a future price is not introduced into a past decision.

Each enabled indicator contributes at most one point. The scoring rules are shown in Table 4.11. They focus on a positive transition or strengthening condition rather than assigning a point whenever an indicator has any valid value. A missing indicator receives zero and an explanatory reason. The denominator is the number of indicators enabled by the user, not a fixed value of five, which keeps the later 60% threshold meaningful in manual mode.

Table 4.11: Deterministic positive-signal conditions in the technical agent

Indicator	One-point condition; satisfying any listed condition is sufficient
RSI	Crosses upward through 50; is between 50 and 70 and rising; or is below 35 but recovering from the previous value.
Moving averages	SMA20 crosses upward through SMA50; SMA20 remains above SMA50 with an expanding positive gap; or current price is above SMA20 while SMA20 is above SMA50.
Bollinger Bands	Price crosses upward through the middle band; remains above the middle band with a widening positive distance; or recovers while at or below the lower band.
MACD	MACD crosses upward through its signal line; MACD is above the signal while the histogram strengthens; or the histogram changes from negative to nonnegative.
KDJ	K crosses upward through D; K is above D while J strengthens; or K rises while below 20.

For every enabled indicator, the output contains current and previous values, a binary score, and a rule-generated reason. The agent also returns the actual covered dates, number of candles, interval, and display price. The aggregator can therefore ask the language model to explain concrete signals while retaining a score that was produced outside the model. Overbought RSI, price above the upper Bollinger Band, weakening MACD histogram, or overbought KDJ can be described as cautionary evidence even though the system’s recommendation vocabulary remains limited to Buy and Wait.

Fundamental-Analysis Agent

The fundamental agent loads financial indicators, income statements, cash flows, a stored summary, and company-profile metadata. It retains up to five most recent annual records and orders them chronologically. For applicable values it computes compound annual growth rate as

$$\text{CAGR}(x) = \left(\frac{x_n}{x_0} \right)^{\frac{1}{y_n - y_0}} - 1, \quad (4.1)$$

where x_0 and x_n are the earliest and latest positive observations and $y_n - y_0$ is the elapsed number of years. CAGR is omitted when fewer than two valid points exist, the elapsed period is nonpositive, or either endpoint is nonpositive. This restriction avoids producing a misleading growth percentage for a series that changes sign.

For nonfinancial firms, selected evidence is organized into liquidity, leverage, efficiency, profitability, and valuation. The leverage or profitability preparation can include a DuPont decomposition,

$$ROE = \text{Net Margin} \times \text{Asset Turnover} \times \text{Financial Leverage}, \quad (4.2)$$

which helps distinguish margin-driven returns from efficient asset use or debt-supported returns. For firms whose ICB code begins with 8, the agent uses a financial-sector presentation based on available CAMELS-related measures. It avoids relying on P/E and EV/EBITDA for financial-sector valuation and instead presents P/B. When specialized measures such as CAR, NPL, NIM, LDR, or CIR are unavailable, the evidence explicitly states the limitation rather than filling it with an inferred value.

The company is also compared with precomputed annual medians for the corresponding ICB industry. A comparison table is included only when at least three peers contribute, reducing the risk of treating one or two companies as a representative benchmark. The evidence includes the latest company value, latest industry median, company CAGR, and industry CAGR for selected measures. These deterministic preparations give the aggregator concrete relative context rather than asking it to decide what constitutes the industry from an unrestricted prompt.

Disabling every fundamental category causes the branch to return an explicit disabled marker. Data-access failures and completely missing financial records are converted to a no-data marker and do not terminate the other branches. This is important for newly listed companies and for batch analysis, where one incomplete company should not cancel an entire universe.

Article-Analysis Agent

The article agent retrieves records already associated with the selected stock and filters them to the orchestrator’s date interval. Its input to the model contains each article’s date, title, available summary, and stored sentiment. The model is instructed to retain useful and relevant content and return a Vietnamese structure containing the main information, positive points, and negative points or risks. This branch uses a low but nonzero temperature of 0.2 because limited linguistic variation is acceptable in a summary, while highly variable creative output is undesirable in financial analysis.

If the user disables news, no article falls within the planned period, or the branch cannot provide usable content, it returns a recognizable marker. The aggregator detects these markers in Python. Missing news is therefore treated as absent evidence rather than silently converted into neutral evidence that would still affect confidence.

4.5.4 Aggregation and Structured Recommendation

The aggregator receives the three branch results and first determines which sources contain real evidence. Technical data is active only if the result is a dictionary containing

indicators and no error. Fundamental and article text are active only if they are nonempty and do not contain known disabled or no-data markers. These flags are placed into the aggregation prompt so the model cannot legitimately fill an absent section with invented analysis.

The model parses its response into an `InvestmentRecommendation` schema. The schema restricts the recommendation to (Buy) or (Wait); requires separate technical, fundamental, news, and summary explanations; and carries the raw labels needed by deterministic confidence calculation. It also permits entry, take-profit, stop-loss, and maximum-holding values only for a candidate Buy result. A structured schema guarantees field presence and type conformance, but it does not by itself guarantee factual correctness. Stockrrium therefore applies Python post-processing after parsing.

The aggregator’s prompt requires technical explanations to cover each enabled indicator, fundamental explanations to use the appropriate nonfinancial or financial-sector structure, and news explanations to distinguish macro-level, company-level, and outlook implications. It also instructs the model not to repeat the recommendation or invent confidence inside narrative fields. If generation reaches the configured output-token limit, the aggregator retries once with an instruction to shorten each explanation. Any unrecovered aggregation exception places an error in graph state and produces a safe Wait-shaped fallback with empty risk values.

4.5.5 Deterministic Confidence and Decision Controls

Confidence is calculated after structured parsing and must not be interpreted as a probability of future profit. Let s_t be the technical score and m_t the number of enabled technical indicators. The normalized source components are

$$c_t = \frac{s_t}{m_t}, \quad c_f \in \{1, 0.5, 0\}, \quad c_n \in \{1, 0.5, 0\}, \quad (4.3)$$

where fundamental labels **strong**, **neutral**, and **weak** map respectively to 1, 0.5, and 0, while article labels **positive**, **neutral**, and **negative** use the same mapping. If no technical indicator is enabled, the technical component is set to 0; in that configuration the technical source is inactive and is removed by the effective-weight renormalization in Equation 4.4, so this value does not affect the final confidence.

Default weights vary by horizon as shown in Table 4.12. The short-term profile emphasizes price signals, while the long-term profile emphasizes company fundamentals. Manual weights replace this table after API validation has established that the three values are within $[0, 1]$ and total one.

Table 4.12: Default confidence weights by investment horizon

Horizon	Technical	Fundamental	News
Short term: below 3 months	0.60	0.10	0.30
Medium term: 3–12 months	0.40	0.40	0.20
Long term: above 1 year	0.15	0.70	0.15

An unavailable or disabled source is removed before aggregation. If A is the set of active sources and w_i is the default or manual base weight, the effective weight is

$$w'_i = \begin{cases} \frac{w_i}{\sum_{j \in A} w_j}, & i \in A, \\ 0, & i \notin A, \end{cases} \quad C = \sum_i w'_i c_i. \quad (4.4)$$

Thus, disabling news does not leave its original weight as an invisible neutral contribution. The remaining evidence is renormalized to sum to one. The response exposes the component scores, effective weights, raw technical score, qualitative labels, final confidence, and the fixed acceptance threshold of 0.55, making the calculation inspectable.

The decision layer acts conservatively as a veto system. The model is first instructed to apply the same rules and generates a candidate recommendation. Python never promotes a model-generated Wait to Buy; it can only preserve a candidate Buy or downgrade it. A candidate Buy is changed to Wait when any applicable condition holds:

- the technical score is below $[0.60m_t]$, provided at least one technical indicator is enabled;
- fundamental evidence exists and its label is **weak**;
- news evidence exists and its label is **negative**; or
- deterministic confidence is below 0.55.

If fundamental or news evidence is absent, its veto is skipped rather than treated as a negative signal. If the final result is Wait, all trading-plan values are set to `null`. This long-only design prevents the result from resembling a short-sale or execution instruction and is consistent with the application scope.

For a surviving Buy result, the current technical price is used when the model does not supply an entry price. Python then constrains take-profit distance, stop-loss distance, and maximum holding count according to Table 4.13. An out-of-range value is clamped to the nearest boundary; a missing or invalid value is replaced by the midpoint of the permitted range. Final prices are recalculated from the bounded percentages and appended to the narrative summary only after enforcement, preventing the displayed text from disagreeing with the structured numerical fields.

Table 4.13: Implemented horizon-specific trading-plan boundaries

Horizon	Take-profit distance	Stop-loss distance	Maximum hold
Short term	8–15% above entry	4–7% below entry	5–15 candles
Medium term	15–30% above entry	7–12% below entry	15–60 candles
Long term	40–80% above entry	15–20% below entry	120–260 candles

Two implementation qualifications are necessary for an accurate description. First, the prompt asks the model to maintain a reward-to-risk ratio of at least 1.5, but the current Python post-processing does not recalculate this condition as a final veto after clamping. The ratio displayed in the result is therefore informative rather than a deterministically guaranteed constraint. Second, the boundary configuration describes maximum hold in daily-candle terms, while live technical analysis maps medium- and long-term horizons to weekly and monthly candles and the mobile interface labels the field as a number of candles. The present code does not convert the bounded count between these time scales. Consequently, the values in Table 4.13 document the implementation but should not be reinterpreted as a universally consistent calendar duration. Aligning interval semantics and adding a final reward-to-risk assertion are appropriate hardening tasks for future work.

Overall, the agentic workflow separates evidence acquisition and deterministic calculation from semantic synthesis. Parallel specialist nodes make each analytical perspective inspectable; structured models constrain generated objects; and Python checks can reject a candidate Buy that violates measurable rules. The workflow remains an analytical support mechanism rather than an autonomous trading system: it supplies a traceable Buy-or-Wait assessment, but it neither places an order nor guarantees an investment outcome.

4.6 Backtesting Framework

Stockrium includes a historical simulation subsystem for evaluating whether the analytical pipeline behaves consistently with its intended decision rules. The subsystem is designed as an engineering and research tool rather than an automated trading facility. It reuses the production technical-analysis and agentic components, applies explicit execution assumptions, and produces machine-readable artifacts for later evaluation. Experimental results obtained from this framework are reported in Chapter 5.

The implementation is exposed through the administrator-protected `POST /api/agent/backtest` endpoint. A request specifies the stock symbol, historical boundaries, minimum technical score, maximum holding period, optional score-based exit, and au-

automatic or manual evidence selection. Historical OHLCV data are loaded through the market service, while VN-Index candles provide the reference series for regime analysis. Each completed run returns parity results, model decisions, simulated trades, metrics, benchmark comparisons, statistical tests, and visualization data.

4.6.1 Evaluation Architecture

The backtest reuses the production indicator, specialist-agent, and aggregator implementations rather than maintaining a separate research-only prediction model. Figure 4.10 shows the execution path. Daily candles first pass through deterministic indicator and scoring engines. Only dates on which the combined score crosses the configured threshold become candidates for agentic analysis. A candidate is admitted as a trade only when the final structured recommendation is *Mua* (Buy). The engine-only baseline retains all technical candidates, allowing the filtering effect of the complete agentic pipeline to be isolated.

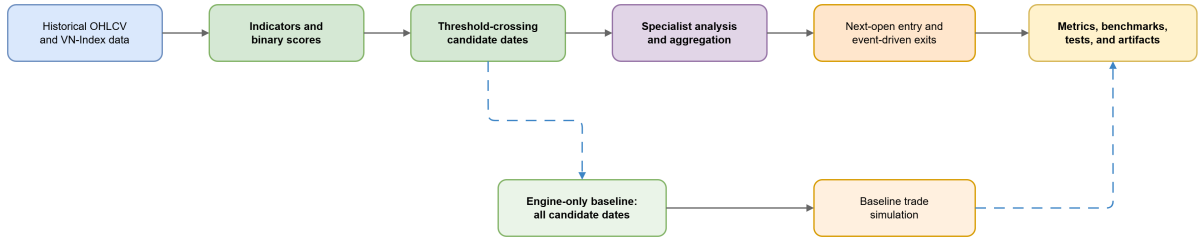


Figure 4.10: Execution flow of the Stockrium backtesting framework

Before a full experiment runs, the system performs a scoring-parity gate. Ten candle indices are sampled between the end of the 50-candle warm-up and the final observation. At each sample, the production technical agent is restricted to data ending on that date and its scores are compared with the vectorized backtest engine. A run is rejected unless the match rate is 100%. This verifies that historical simulation and live technical scoring implement the same rules; it does not by itself establish profitability.

4.6.2 Technical Signal Construction

The indicator engine calculates the same five perspectives described in Section 4.5.3: RSI with a 14-period lookback, SMA20 and SMA50, 20-period Bollinger Bands at two standard deviations, MACD (12, 26, 9), and KDJ with a 9-period range and 3-period smoothing. Each perspective contributes either zero or one point, giving a combined score $s_t \in [0, 5]$ when all indicators are enabled.

The first 50 observations form the warm-up interval because SMA50 is the longest rolling feature. With default threshold $\tau = 3$, a candidate is emitted only when the score

enters the qualified region,

$$\text{candidate}_t = \mathbb{I}(s_t \geq \tau \wedge s_{t-1} < \tau). \quad (4.5)$$

The crossing rule avoids opening a new candidate on every day of a persistent high-score interval. The last candle is never signaled because no next-candle execution price exists. Model-backed analysis is invoked only on candidate dates, reducing the number of language-model calls. Technical and article plans use the candidate date as their upper boundary and a 252-day lookback in the main daily experiment.

4.6.3 Trade Simulation and Cost Model

The simulator is long-only and permits at most one open position per symbol. A signal completed after candle t is entered at the open of candle $t + 1$, avoiding execution at the same close used to construct the signal. Structured entry, take-profit, stop-loss, and maximum-holding values supplied by the aggregator are used after validity checks; otherwise, the next open and configured holding limit are applied.

Starting with the entry candle, the daily low is checked against the stop-loss level and the daily high against the take-profit level. Stop loss is evaluated first, giving a conservative outcome when both levels fall within one daily bar and intraday order is unavailable. If configured, a score below three schedules a `SIGNAL_EXIT` at the following open. Otherwise, a position exits at the final permitted close with reason `TIMEOUT`. The scan resumes after the exit, so same-symbol positions do not overlap.

The default transaction cost is $c = 0.0015$, or 0.15% per side. For entry price P_k^{in} and exit price P_k^{out} , net return is

$$r_k = \frac{P_k^{\text{out}} - P_k^{\text{in}}}{P_k^{\text{in}}} - 2c. \quad (4.6)$$

The default round-trip deduction is therefore 0.30%. The simulation does not separately model bid–ask spread, market impact, partial fills, taxes, price-limit queues, or gap slippage. These assumptions are considered when interpreting the experimental results.

4.6.4 Measures, Benchmarks, and Artifacts

The framework reports trade count, win rate, average and median trade return, total compounded return, annualized return, maximum drawdown, a trade-level Sharpe statistic, Calmar ratio, profit factor, and exit counts. Total return and the post-trade equity sequence are

$$R_{\text{total}} = \prod_{k=1}^K (1 + r_k) - 1, \quad E_k = \prod_{j=1}^k (1 + r_j). \quad (4.7)$$

Let $H_k = \max_{j \leq k} E_j$ denote the running peak. Maximum drawdown is the minimum of $(E_k - H_k)/H_k$. The implemented Sharpe statistic is

$$S = \sqrt{K} \frac{\bar{r}}{s_r}, \quad (4.8)$$

where s_r is the sample standard deviation of trade returns. It follows the risk-adjusted comparison principle of the Sharpe ratio [22], but is scaled by trade count rather than a fixed annual frequency. It is therefore an internal comparison statistic rather than a conventional annualized portfolio Sharpe ratio.

The evaluation suite compares the full pipeline with the engine-only strategy, buy-and-hold exposure, and 1,000 random-entry simulations. It also provides a one-sample t -test, sign-permutation test, score–return information coefficient, confidence grouping, rolling walk-forward windows, and VN-Index regime analysis. Because repeated strategy comparisons can overfit historical data [23], the framework retains negative and non-significant results instead of reporting only successful securities.

Each run produces visualization JSON containing prices, indicators, decisions, entries, exits, and metrics. The artifact can be served by FastAPI and uploaded to Supabase Storage. A separate aggregate JSON file is updated per symbol to support VN30-level analysis. Chapter 6 uses these saved artifacts as the quantitative evidence for evaluation.

Chapter 5 System Testing

This chapter evaluates whether the implemented Stockrium platform satisfies its software requirements. It reports functional, integration, non-functional, compatibility, and user acceptance testing across the mobile application, backend APIs, persistent storage, market-data services, and AI providers.

5.1 Testing Objectives and Scope

System testing evaluated whether Stockrium satisfies its functional requirements as an integrated mobile-backend platform. Unlike Chapter 6, which examines investment-strategy performance, this chapter focuses on software correctness, integration, security, responsiveness, reliability, usability, and device compatibility.

A black-box, requirement-oriented approach compared each observed result with a predefined acceptance criterion. Tests included valid flows and negative or boundary cases such as invalid inputs, unauthorized access, expired sessions, duplicate records, empty data, and connection failures. Table 5.1 summarizes the scope.

Table 5.1: Scope of Stockrium system testing

Test scope	Items covered	Cases
Functional and black-box	Authentication, market data, search, charts, user data, AI analysis, chatbot, news, profile, and navigation.	48
Integration and API	Mobile-FastAPI communication, Supabase, SSI, and OpenAI/Gemini responses.	24
Non-functional	Response time, reliability, JWT security, authorization, and usability.	20
Compatibility and device	Android/iOS, screen sizes, themes, and Vietnamese/English localization.	16
Total	All planned system-test cases	108

The tests covered the user journey from registration to AI-assisted analysis and conversation, including the main external integrations. Historical profitability, source-code unit coverage, third-party penetration testing, and production-scale load testing were excluded. Acceptance required every critical flow to complete without blocking defects, protected resources to reject invalid access, and response times to remain within predefined limits.

5.2 Testing Methods, Tools, and Environment

5.2.1 Testing Methods and Tools

Manual black-box testing checked complete user flows and visible error states. Postman and Swagger UI were used to inspect FastAPI status codes, response bodies, validation, and authorization independently of the mobile client. Expo development builds ran on physical devices and emulators. User acceptance testing involved the developers and Ms. Trinh, an FPT Securities analyst, who reviewed workflow clarity, financial terminology, and presented information.

5.2.2 Test Environment and Data

The backend used Python 3.12 and FastAPI 0.128.0; the client used Expo SDK 54 and React Native 0.81.5. Mobile tests ran on a Pixel 7 Android 15 emulator, Samsung Galaxy A54 (Android 14), iPhone 15 iOS 18 simulator, and iPhone 13 (iOS 18).

Test accounts included verified, unverified, secondary, and administrator users. Representative data covered securities, company profiles, financial statements, news, favorites, portfolios, search histories, risk preferences, and chats. VNM, FPT, and VCB supplied complete positive cases; unknown symbols, missing fields, expired tokens, invalid OTPs, duplicate favorites, and empty ranges supplied negative cases. AI outputs were assessed for schema validity, completeness, consistency, and safe presentation rather than investment profitability.

5.3 Functional and Black-Box Testing

Functional testing followed the main user journey through authentication, market discovery, stock details, personal data, AI analysis, and conversation. Table 5.2 groups related successful, invalid, and boundary cases.

Table 5.2: Summary of functional and black-box test cases

ID	Feature	Representative action	Observed result	Pass
F-AUTH	Authentication	Register, verify, sign in, recover password, refresh token, sign out, and submit invalid credentials.	Valid sessions persisted; invalid or expired credentials were rejected clearly.	8/8
F-MKT	Market and search	View indices, search symbols/companies, refresh lists, and submit unknown queries.	Correct records and navigation appeared; unknown inputs showed a controlled empty state.	7/7
F-DET	Details and charts	Change periods, chart types, and technical indicators; view financial sections.	Stock and period remained consistent; charts rendered without clipping, stale data, or crashes.	8/8
F-USER	User-owned data	Manage favorites, risk appetite, portfolios, and search history.	Data persisted per user; duplicate operations remained controlled.	7/7
F-AI	AI analysis	Run automatic/manual analysis, configure evidence, and submit invalid settings.	Valid requests returned structured evidence and recommendations; invalid settings were blocked.	7/7
F-CHAT	Chatbot	Create, continue, reopen, list, and delete conversations.	Messages remained ordered and owner-scoped; deletion removed the session and history.	5/5
F-UI	News and settings	Browse news, edit profile, change theme/language, navigate, and log out.	Content, settings, and routes behaved as expected.	6/6
Functional total				48/48

Authentication tests confirmed the complete account transition: unverified accounts could not sign in, valid OTPs completed verification, expired access tokens triggered a coordinated refresh, and logout removed access to protected screens. Market entry points consistently opened the same stock context, while empty or unavailable data produced useful messages instead of indefinite loading.

AI automatic mode selected evidence automatically; manual mode required at least one source and valid subcategories. Results included the symbol, evidence, normalized scores, confidence, action, and explanation, and could seed a follow-up chat. Favorites, preferences, portfolios, histories, and conversations remained associated with the authenticated user after refresh or restart. All 48 functional cases passed.

5.4 Integration and API Testing

Integration tests exercised boundaries among the mobile client, FastAPI, Supabase, SSI, and AI providers. Operations were tested directly through Postman/Swagger UI and then through the application to confirm request construction, token injection, parsing, retry behavior, and rendering.

Table 5.3: Summary of integration and API test cases

ID	Boundary	Representative test	Observed result	Pass
I-API	FastAPI contract	Call success/failure routes with varied fields, IDs, ranges, and tokens.	Status codes and the common response structure matched the contract.	7/7
I-MOB	Mobile–backend	Execute authentication, search, detail, favorite, and AI requests.	Requests were parsed once and errors were displayed safely.	5/5
I-DB	Supabase	Create, read, update, and delete user-owned records.	Data persisted and remained isolated by user.	4/4
I-SSI	SSI market data	Request valid, unknown, empty, and unavailable data.	Valid data were transformed correctly; failures returned controlled states.	3/3
I-AI	AI workflow	Run specialist branches, aggregation, chat, and invalid provider output.	Valid responses met the schema; invalid output failed safely.	5/5
Integration/API total				24/24

Successful APIs returned domain data, a zero error code, a request identifier, and a true result. Invalid or unauthorized requests returned a non-zero code without exposing stack traces, secrets, or database details. Supabase operations preserved ownership, while SSI data were checked for symbol, chronology, and OHLCV fields. AI success meant schema-compliant, safely renderable output; provider errors were contained by the backend. All 24 integration cases passed.

5.5 Non-Functional Testing

5.5.1 Performance and Reliability

Performance tests measured user-perceived response time under stable broadband. Limits were two seconds for ordinary APIs, fifteen seconds for multi-agent analysis, and ten seconds for chat.

Table 5.4: Observed response-time results

Operation	Average	95th percentile	Limit	Result
Authentication request	0.31 s	0.58 s	2 s	Pass
Market/search read	0.42 s	0.79 s	2 s	Pass
Stock detail and indicators	0.68 s	1.24 s	2 s	Pass
Full AI analysis	7.84 s	12.61 s	15 s	Pass
Chatbot response	4.36 s	7.92 s	10 s	Pass

All operations met their limits. Loading feedback remained visible and submitted actions were disabled to prevent duplicates. During interrupted connections, expired sessions, empty upstream data, and provider outages, the application preserved its state, offered retry or error feedback, and recovered after service restoration. Four reliability cases passed.

5.5.2 Security Testing

Security tests focused on JWT validation, revocation, roles, ownership, input validation, and local credential protection.

Table 5.5: Summary of security test cases

ID	Condition	Observed result	Status
S-01	Missing, malformed, or expired access token	Unauthorized; no protected data returned.	Pass
S-02	Revoked or expired refresh token	Renewal failed and the user returned to authentication.	Pass
S-03	User requests an administrator route	Request rejected; no operation started.	Pass
S-04	User A accesses User B's chat	Access denied; User B's data unchanged.	Pass
S-05	Invalid, oversized, or injection-like input	Controlled validation error without internal details.	Pass
S-06	Inspect persistence before/after logout	SecureStore held tokens; logout removed the active session.	Pass

No cross-user disclosure, privilege escalation, or continued access after revocation was observed.

5.5.3 Usability and User Acceptance Testing

Participants completed five representative tasks: finding a stock, using charts, configuring AI analysis, saving a favorite, and asking a follow-up question. All tasks were completed successfully. The developers and FPT Securities analyst accepted the workflow, selected states, Vietnamese financial terminology, warnings, confidence information, and analytical presentation. Together, performance, reliability, security, and usability comprised 20 passing cases.

5.6 Compatibility and Device Testing

The critical path was repeated across Android/iOS devices and emulators, compact/larger screens, light/dark themes, and Vietnamese/English. Checks covered safe areas, navigation, keyboard behavior, scrolling, charts, dialogs, text wrapping, and session persistence.

Table 5.6: Summary of compatibility and device test cases

ID	Configuration	Checks	Observed result	Pass
C-AND	Android	Launch, navigation, keyboard, storage, charts, and critical flow.	Stable; all controls reachable.	4/4
C-IOS	iOS	Safe areas, navigation, keyboard, persistence, charts, and critical flow.	Stable; all controls reachable.	4/4
C-DIS	Displays/themes	Screen sizes, themes, long text, sheets, and dialogs.	No hidden actions; readable contrast and responsive charts.	4/4
C-L10N	VI/EN	Labels, messages, dates, numbers, and long translations.	Language persisted; layouts fitted with no missing key.	4/4
Compatibility total				16/16

Platform-specific safe areas, back behavior, and keyboards did not prevent task completion. Compact screens used scrolling, charts adapted to available width, and theme or language changes persisted after restart. All 16 compatibility cases passed.

5.7 Overall Results and Discussion

All 108 planned cases were executed and passed. No unresolved critical or high-severity defect remained.

Table 5.7: Overall system-testing results

Test area	Planned	Executed	Passed	Failed	Pass rate
Functional and black-box	48	48	48	0	100%
Integration and API	24	24	24	0	100%
Non-functional	20	20	20	0	100%
Compatibility and device	16	16	16	0	100%
Total	108	108	108	0	100%

The results show that authentication, market data, user-owned records, AI output validation, and device presentation worked as end-to-end tasks. However, the 100% rate

applies only to the selected cases and environments; it does not cover every device, network condition, record, or future external-provider behavior. Manual testing may also miss regressions better detected by automation. Within the prototype scope, Stockrium met its acceptance criteria and was ready for experimental evaluation and thesis demonstration.

5.8 Chapter Summary

Stockrium was evaluated through 108 functional, integration, non-functional, and compatibility cases using manual testing, Postman/Swagger UI, Expo devices/emulators, and user acceptance testing. All cases passed: features produced expected results, integrations returned valid responses, security controls rejected invalid access, response times met their limits, and the application operated correctly across the tested platforms, themes, screen sizes, and languages.

Chapter 6 Experiments and Evaluation

This chapter evaluates the analytical behavior of Stockrium on historical Vietnamese market data. It compares the complete pipeline with deterministic and passive baselines, examines robustness across securities, indicators, confidence levels, windows, and market regimes, and identifies the limitations that constrain the strength of the conclusions.

6.1 Evaluation Methodology

The evaluation examines the historical behavior of Stockrium’s analytical pipeline. Its purpose is to determine how the complete agentic strategy behaves relative to explicit alternatives, how sensitive the results are to securities and market conditions, and whether the available evidence supports a reliable performance claim. It does not treat historical profit as a guarantee of future return.

Three evaluation questions guide the chapter:

1. **EQ1: Comparative performance.** How does the complete agentic pipeline compare with the engine-only technical strategy, passive buy-and-hold exposure, and random entry timing?
2. **EQ2: Robustness and calibration.** Are results consistent across VN30 securities, technical indicators, confidence tiers, rolling windows, and market regimes?
3. **EQ3: Statistical and practical validity.** Are observed returns statistically distinguishable from simple alternatives, and which data or simulation limitations constrain interpretation?

6.1.1 Evidence Sources

The primary evidence consists of JSON artifacts produced by the backtesting framework described in Section 4.6. The complete-pipeline artifact contains 30 securities with pipeline and engine-only metrics, buy-and-hold and random comparisons, statistical tests, confidence summaries, regime results, and rolling walk-forward windows. Five additional artifacts contain engine-only ablations for RSI, MACD, KDJ, Bollinger Bands, and moving-average conditions over the explicitly recorded period from January 1, 2023 to December 31, 2025.

Source inspection is used only to establish how the artifacts were produced: technical threshold crossings generate candidate dates; agentic approval filters candidates; execution begins at the next open; stop loss is checked before take profit within a daily candle; and 0.15% transaction cost is deducted on each side. This inspection is necessary because an evaluation is meaningful only when the execution assumptions behind its metrics are

stated clearly.

6.1.2 Measures and Comparisons

Performance is summarized by trade count, win rate, total return, maximum drawdown, and the implemented trade-level Sharpe statistic. Cross-sectional means and medians are reported separately because a small number of securities contain sparse or extreme values. The full pipeline is compared with the same technical engine before language-model filtering, which isolates the incremental effect of article, fundamental, aggregation, confidence, and veto stages.

Buy-and-hold provides a passive same-security reference. The random-entry benchmark places pipeline return within 1,000 simulations using randomly selected entry dates and the same requested trade count. One-sample t -tests and sign-permutation tests examine whether per-trade outcomes differ from zero. Confidence groups, VN-Index regimes, and walk-forward windows provide robustness views. Single-indicator ablations show whether results depend disproportionately on one technical perspective.

Repeated historical comparisons can produce backtest overfitting [23]. Accordingly, this chapter reports unfavorable and non-significant results, distinguishes descriptive metrics from statistical evidence, and treats limitations in point-in-time data, universe construction, execution, and experiment metadata as part of the findings rather than as implementation details to omit.

6.2 Historical Backtesting Evaluation

6.2.1 VN30 Experimental Configuration

Two saved experiment groups are available. The complete agentic-pipeline report contains 30 symbols and was accumulated between June 30 and July 5, 2026. It stores full-pipeline and engine-only performance, buy-and-hold and random comparisons, statistical tests, walk-forward summaries, regime results, and confidence calibration. Because the artifact is updated one symbol at a time, it does not retain one authoritative request-level start and end date. Its recorded rolling windows cover 2024–2025 after the daily lookback, but the exact full interval cannot be reconstructed solely from the aggregate file.

The second group isolates each technical indicator. These ablations use the 30 symbols returned by the current VN30 lookup, daily data from January 1, 2023 through December 31, 2025, a 20-candle maximum hold, no language-model calls, and the default cost of 0.15% per side. Each binary indicator score replaces the combined score, and a transition to one creates a candidate. All 30 symbols completed each of the five ablations.

The universe is the VN30 list available when the experiments were executed, not a

date-reconstructed historical membership list. The results therefore describe the selected large-cap stock set and are not a survivorship-bias-free replication of the index.

6.2.2 Full-Pipeline and Baseline Results

Table 6.1 reports cross-sectional results from the 30 symbols. The geometric aggregate is the geometric mean of per-symbol wealth multipliers, not the return of a capital-constrained rebalanced portfolio.

Table 6.1: Full agentic pipeline compared with the engine-only baseline

Measure	Full pipeline	Engine only	Difference
Completed trades	433	741	−308
Profitable symbols	22 of 30	22 of 30	0
Mean per-symbol total return	33.00%	49.27%	−16.27 pp
Median per-symbol total return	23.21%	21.33%	+1.89 pp
Geometric aggregate return	24.67%	30.37%	−5.70 pp
Mean per-symbol win rate	50.55%	53.01%	−2.46 pp
Median per-symbol win rate	50.00%	51.93%	−1.93 pp
Mean per-symbol maximum drawdown	−16.66%	−25.69%	+9.03 pp
Median trade-level Sharpe statistic	0.784	0.637	+0.146

The full pipeline admitted 41.6% fewer trades and reduced mean maximum drawdown by about nine percentage points. Its median per-symbol return and median trade-level Sharpe statistic were modestly higher. However, mean return, geometric aggregate return, and mean win rate were lower. The pipeline exceeded engine-only return for 12 of 30 symbols and buy-and-hold return for only 4 of 30. Passive buy-and-hold is in fact the strongest same-security reference: its cross-sectional mean return of 105.61% and median of 85.54% far exceed the pipeline’s 33.00% mean and 23.21% median, because the pipeline is invested only during selected intervals rather than continuously through the 2023–2025 uptrend. The saved evidence therefore characterizes the agentic layer primarily as a conservative exposure filter rather than a source of broad return outperformance. These full-pipeline figures must be read together with the point-in-time limitation in Section 6.3.3: only the full pipeline uses the fundamental branch, whose latest-record loading can introduce look-ahead, so the cleanest leakage-free evidence is the single-indicator

ablation reported in Section 6.2.4.

The stored mean pipeline Sharpe statistic of 4.689 is distorted by sparse observations. VHM contains only two trades and a reported Sharpe statistic of 122.0. Every symbol has fewer than 30 pipeline trades and four have fewer than five. The median value of 0.784 is consequently more representative. Because the trade-level Sharpe statistic scales with the square root of the trade count (Equation 4.8) and the pipeline executes fewer trades per symbol than the engine, this scaling works against the pipeline; the higher median pipeline Sharpe is therefore not an artifact of a larger sample. LPB has the highest saved pipeline total return at 173.91%, while SAB has the lowest at -44.50% , demonstrating substantial cross-sectional variation.

Only three of the 30 one-sample t -tests and two sign-permutation tests are significant at the 5% level. The median random-entry percentile is 72.35%, but no multiple-comparison-adjusted universe-level test is present. Most individual results therefore do not reject a zero-mean or random-timing explanation, and the evidence does not establish statistically reliable alpha.

The score–return information coefficient reinforces this caution. Correlating the technical score at trade entry with the realized trade return gives a mean information coefficient of 0.064 and a median of 0.032 across the 30 symbols; the coefficient is positive for 16 securities and negative for 12. Only one of the 30 coefficients is significant at the 5% level, and the single extreme value of 1.000 again originates from VHM’s two-trade sample. The entry score therefore exhibits, at best, a weak and statistically unreliable monotonic relationship with subsequent return, which is consistent with interpreting the pipeline as an exposure filter rather than a return-ranking signal.

6.2.3 Confidence, Regime, and Walk-Forward Results

Table 6.2 pools confidence-tier outcomes. High-confidence trades show only a small difference from medium-confidence trades, while the Buy gate prevents low-confidence trades from being executed. The accepted-trade sample therefore cannot provide a calibration curve across the full confidence range.

Table 6.2: Pooled confidence-tier outcomes in the saved pipeline report

Confidence tier	Trades	Win rate	Mean net return
High ($C \geq 0.70$)	362	51.66%	2.00%
Medium ($0.55 \leq C < 0.70$)	71	49.30%	1.96%
Low ($C < 0.55$)	0	—	—

Regime grouping assigns 369 trades to sideways conditions, 63 to uptrends, and

only one to a downtrend. Sideways trades record a 54.20% win rate and 2.08% mean return; uptrend trades record a 34.92% win rate and 1.63% mean return. One downtrend observation cannot support a meaningful regime conclusion.

Across symbols, the mean fraction of profitable walk-forward windows is 19.52% and the median is 19.05%. Most windows contain zero or one trade, causing the implemented Sharpe statistic to be zero. The walk-forward results reveal instability and sparse sampling rather than supporting a strong out-of-window performance claim.

6.2.4 Single-Indicator Ablation

Table 6.3 reports the 2023–2025 engine-only ablations. Return, Sharpe, and drawdown are calculated for each symbol and then averaged across 30 symbols; trade counts are pooled.

Table 6.3: VN30 single-indicator ablation results, 2023–2025

Indicator	Trades	Mean win rate	Mean total return	Mean Sharpe	Mean max drawdown
RSI	824	50.23%	29.81%	0.502	−27.58%
MACD	702	51.26%	35.73%	0.604	−24.72%
KDJ	890	52.11%	42.75%	0.722	−26.34%
Bollinger Bands	806	50.68%	47.11%	0.694	−26.99%
MA crossover/trend	407	51.64%	20.62%	0.576	−16.98%

Bollinger conditions produce the highest mean per-symbol return, while KDJ produces the highest mean Sharpe statistic and win rate. Moving-average conditions generate approximately half as many trades as the reactive oscillators and have the smallest mean drawdown, but also the lowest mean return. No indicator dominates every criterion. This supports using complementary technical perspectives, but it does not prove that equal binary weighting is optimal.

6.3 Discussion and Threats to Validity

6.3.1 Answers to the Evaluation Questions

For EQ1, the agentic layer changes the strategy materially by reducing 741 engine-only trades to 433 executed trades. This filtering lowers mean maximum drawdown and slightly improves median return and the median trade-level Sharpe statistic. However, it

also lowers mean return, geometric aggregate return, and win rate. It exceeds engine-only return for 12 of 30 symbols and buy-and-hold return for 4 of 30. The evidence therefore supports interpreting the agentic stage as a conservative exposure filter, not as a broadly superior return strategy.

For EQ2, robustness is limited. Results vary substantially among securities, no individual technical indicator dominates every measure, high- and medium-confidence outcomes are nearly identical, and the Buy gate leaves no low-confidence trade group. Walk-forward windows are sparse, while the downtrend regime contains only one trade. These findings do not show that the observed behavior is stable across confidence levels, time windows, or market regimes.

For EQ3, only three per-symbol one-sample t -tests and two sign-permutation tests are significant at the 5% level, and the score–return information coefficient is near zero (median 0.032) and significant for only one security. No multiple-comparison-adjusted universe-level hypothesis test is stored. Sparse trades also create unstable risk statistics, including an extreme two-trade Sharpe value. The current experiment is valuable as a transparent comparative evaluation, but it does not establish statistically reliable alpha or expected future profitability.

6.3.2 Practical Efficiency for Financial Analysis

Although the historical evidence does not support a claim of superior investment returns, Stockrium still provides an important practical benefit by reducing the time required to prepare an initial stock analysis. A user without a financial background must first locate the relevant financial statements, learn the formulas and meanings of indicators such as profitability, liquidity, leverage, valuation, and growth ratios, calculate or compare the values, inspect price-based technical indicators, read related news, and combine the findings into one interpretation. Completing this process carefully for one security can reasonably occupy most of a working day, especially when the user must learn the analytical concepts at the same time.

An experienced securities analyst can perform the same work more efficiently because the formulas, warning signs, and information sources are already familiar. Nevertheless, collecting current records, checking financial-statement trends, reviewing technical conditions and news, and writing a coherent assessment may still require approximately one to two hours for a single symbol. Repeating the process across several candidate securities causes this preparation time to increase almost linearly.

Stockrium automates much of this preliminary work. Its specialist agents retrieve and summarize fundamental, technical, and article evidence, while the aggregator organizes the results into scores, confidence, risks, and a structured recommendation. In the system tests, a full analysis completed in 7.84 seconds on average and remained below 12.61 seconds

at the 95th percentile (Table 5.4). The comparison is illustrative rather than a controlled productivity experiment, but it shows the system’s principal efficiency contribution: work that may require hours of manual collection, formula review, calculation, and synthesis can be reduced to an initial report produced within seconds.

This time saving is most valuable as decision support. The generated report gives inexperienced users a structured starting point and allows experts to spend less time on repetitive data preparation and more time validating assumptions, investigating unusual results, and applying professional judgment. It should not be interpreted as eliminating the need to verify source data or as replacing a qualified analyst. The experimental limitations identified below, particularly point-in-time fundamental availability and model nondeterminism, mean that every recommendation still requires human review before an investment decision is made.

6.3.3 Validity Limitations

Table 6.4: Principal evaluation limitations and recommended improvements

Limitation	Effect on interpretation	Recommended improvement
Fundamental point-in-time availability	The fundamental agent loads the latest stored annual records without filtering them by candidate date, permitting historical leakage.	Store filing-availability timestamps and restrict records to information public by the signal date.
Current-constituent universe	Current VN30 membership can omit removed or delisted securities and include later entrants.	Reconstruct membership by effective date and retain delisted securities.
Aggregate meta-data	The full-pipeline aggregate lacks one common interval, model version, prompt version, and random seed.	Store an immutable run manifest and reject aggregation across incompatible configurations.
Model and random nondeterminism	Nonzero model temperature and unseeded random tests can change decisions and p -values.	Pin model snapshots and prompts, cache structured outputs, and record random seeds.

Table 6.4 continued

Limitation	Effect on interpretation	Recommended improvement
Sparse observations	Every symbol has fewer than 30 pipeline trades, making risk and significance estimates unstable.	Extend history and report bootstrap or hierarchical uncertainty intervals.
Simplified execution	Daily bars cannot determine intraday event order and do not model gaps, spread, liquidity, price limits, or market impact.	Use conservative gap-aware fills and intraday or quote data where available.
Portfolio abstraction	Per-symbol results lack shared capital, sizing, concurrent exposure, and portfolio-level risk.	Add a cash and position ledger with allocation and rebalancing rules.
Walk-forward construction	Test windows reinitialize indicator warm-up, and no model parameters are fitted on the nominal training slice.	Carry historical context into feature calculation and define which choices are learned only from training data.
Confidence truncation	Only accepted Buy recommendations become trades, leaving no low-confidence outcome group.	Record forward outcomes for every candidate, including rejected decisions.
Configuration wiring	The request-level transaction cost is not forwarded and loaded one-minute scores do not affect the daily simulator.	Wire exposed settings end to end and store the effective configuration with every result.

The most important limitation is point-in-time fundamental availability. Technical and article inputs are bounded by the candidate date, but the fundamental branch reads the latest stored multi-year series. The full-pipeline results can therefore contain information unavailable at the simulated decision date. Single-indicator experiments avoid this specific leakage because they do not use fundamental or news evidence, although they retain universe and execution-model limitations.

6.3.4 Overall Evaluation

The evaluation demonstrates that Stockrium can produce inspectable historical decisions, compare the agentic workflow with deterministic and passive alternatives, and expose both performance and risk trade-offs. The agentic layer reduces trading frequency and drawdown, but the saved results do not demonstrate consistent benchmark outperformance, strong confidence calibration, or stable behavior across regimes.

The defensible conclusion is therefore limited: Stockrium provides a useful and transparent evaluation environment for Vietnamese equity decision support, while stronger point-in-time data controls, reproducible experiment manifests, realistic portfolio simulation, and larger out-of-sample evidence are required before investment-performance claims can be made.

Chapter 7 Conclusion and Future Work

This chapter summarizes the outcomes and contributions of Stockrium, assesses the degree to which the project objectives were achieved, identifies the remaining limitations, and proposes directions for improving the data, evaluation, analytical workflow, and software platform.

7.1 Conclusion

This thesis presented Stockrium, an integrated decision-support and research platform for the Vietnamese equity market. The work was motivated by a practical information problem: an individual investor must often combine price behavior, company fundamentals, and financial news while navigating data from separate sources and interpreting results without a consistent analytical framework. Stockrium addresses this problem through a mobile application, a layered backend, coordinated analytical agents, structured data-acquisition pipelines, and a historical backtesting subsystem. The system is designed to organize evidence and explain an assessment; it does not replace investor judgment or execute trades.

The implemented architecture separates presentation, domain services, data storage, deterministic financial computation, and language-model reasoning. Within the agentic workflow, technical, fundamental, and article specialists examine different evidence domains before an aggregator produces a structured recommendation. Deterministic post-processing then calculates the final confidence and enforces the acceptance rule. This design makes individual evidence contributions visible, allows analytical branches to be enabled or disabled, and supports reuse of the same logic in interactive analysis and historical evaluation.

The historical experiment provides a deliberately cautious view of the analytical results. On the evaluated VN30 stock set, the complete pipeline reduced 741 engine-only trades to 433 completed trades. It lowered mean maximum drawdown and modestly improved median return and the median trade-level Sharpe statistic, but it also produced lower mean return, geometric aggregate return, and win rate. It outperformed the engine-only strategy for 12 of 30 securities and buy-and-hold for 4 of 30. Statistical and robustness evidence was limited. The agentic stage is therefore best interpreted as a selective exposure filter within an explainable decision-support process, not as evidence of reliable market outperformance or guaranteed future profit.

7.2 Achievement of the Project Objectives

The three objectives introduced in Section 1.2 were achieved to different degrees. Table 7.1 summarizes the outcome.

Table 7.1: Achievement of the Stockrium project objectives

Objective	Assessment	Outcome
O1	Substantially achieved	A working mobile and backend architecture integrates market exploration, company and news information, technical visualization, configurable multi-agent analysis, and conversational support. The result is an implemented decision-support prototype, although broader usability and production validation remain future work.
O2	Substantially achieved	The project implements a reusable historical simulation framework with scoring-parity checks, explicit execution assumptions, configurable signals and exits, transaction costs, benchmarks, risk measures, statistical tests, and stored experiment artifacts. Chapter 6 demonstrates its use and also establishes the limits of the current evidence.
O3	Partially achieved	Acquisition and normalization pipelines organize historical OHLCV data, company profiles and financial records, industry aggregates, and financial articles with generated summaries and sentiment labels. However, a versioned public release, licensing documentation, historical constituent reconstruction, and complete point-in-time metadata are still required for the dataset to be fully reproducible and research-ready.

The first objective was achieved at the prototype level. Stockrium provides a coherent path from market discovery to evidence inspection and AI-assisted synthesis. Its configurable analysis supports both a simplified automatic mode and a manual mode in which analytical perspectives and weights can be selected. The structured response exposes component scores, confidence, explanations, and risk-related price levels when a Buy recommendation is accepted. These properties make the result more inspectable than a single unrestricted language-model response. Nevertheless, completion of the software prototype is not equivalent to proving that every recommendation is correct or that the

application is ready for unrestricted financial use.

The second objective was achieved as an engineering and evaluation capability. The backtesting subsystem reuses production technical and agentic logic, prevents same-close execution, includes a warm-up interval, applies configurable transaction costs, and records both agent-filtered and deterministic baselines. Its parity gate checks that vectorized historical scores reproduce production scoring rules before a full run proceeds. More importantly, the evaluation reports unfavorable and non-significant outcomes rather than selecting only positive cases. This makes the framework useful for diagnosing behavior even though the current evidence does not establish profitable performance.

The third objective remains incomplete. Stockrium establishes a common schema and maintainable pipelines for the three evidence domains, which is a necessary foundation for a Vietnamese-market research dataset. However, public research use requires more than populated application tables. Each release must define its observation interval, source and licensing conditions, schema version, missing-data policy, collection time, correction history, and information-availability time. Until those controls and a versioned release are completed, the data should be described as a curated application dataset rather than a finished public benchmark.

7.3 Principal Contributions

The principal contributions of the thesis are summarized as follows.

C1. An integrated Vietnamese-market decision-support platform.

Stockrium connects market monitoring, stock discovery, company information, interactive technical charts, financial news, user preferences, AI-assisted analysis, and conversational follow-up within one mobile-oriented system. The platform focuses on Vietnamese securities and preserves a clear boundary between information support and order execution.

C2. An inspectable hybrid multi-agent analysis workflow.

The workflow separates orchestration, evidence preparation, specialist analysis, aggregation, and deterministic decision controls. Specialized agents handle evidence with different structures, while typed outputs and deterministic confidence calculations reduce ambiguity at the system boundary. Selectable branches and normalized weights make the influence of each analytical perspective easier to examine.

C3. A unified data architecture for heterogeneous financial evidence.

The project defines acquisition and storage processes for market candles, current prices, company profiles, financial statements and ratios, industry aggregates, and Vietnamese financial articles. Deduplication, normalization, upsert behavior, scheduled refresh, and explicit relational associations make the data usable across

application screens, analytical services, and experiments.

C4. A production-aligned backtesting and evaluation framework.

The framework evaluates the implemented decision logic rather than a separate research-only model. It includes parity verification, threshold-crossing signals, next-open execution, risk-based exits, transaction costs, deterministic and passive benchmarks, ablation experiments, regime summaries, walk-forward views, and machine-readable artifacts. Its evaluation demonstrates both the filtering behavior of the agentic layer and the weaknesses that prevent strong investment-performance claims.

7.4 Remaining Limitations

Several limitations constrain the conclusions of this thesis. The most important is historical information integrity. Technical and article evidence are bounded by the simulated decision date, but the fundamental branch can retrieve the latest stored annual records rather than only records known at that date. The current VN30 universe is also based on membership available when the experiment is run. These conditions introduce look-ahead and survivorship risks that weaken causal interpretation of full-pipeline performance.

The experiment is further limited by sample size and construction. Every security has relatively few completed pipeline trades, walk-forward groups are sparse, and the evaluated data include almost no downtrend trades. The saved results do not show consistent benchmark outperformance, robust confidence separation, or broad statistical significance. Multiple comparisons, model nondeterminism, prompt changes, and unrecorded random seeds can also affect reproducibility.

The execution model is necessarily simplified. Daily OHLCV bars cannot determine the intraday ordering of take-profit and stop-loss events and do not fully represent gaps, bid-ask spread, liquidity, market impact, partial fills, price limits, or latency. Results are calculated per security without a shared cash balance, position-sizing policy, concurrent-exposure constraint, or portfolio rebalancing rule. Consequently, aggregate returns are comparative research measures rather than a directly investable portfolio record.

The system also depends on external market-data, search, storage, and language-model services. Changes in their availability, response formats, quotas, or model behavior can affect system reliability and analytical reproducibility. The current platform requires further security, load, recovery, device-compatibility, accessibility, and longitudinal usability validation before production-scale deployment. Finally, generated recommendations and article labels may contain reasoning or classification errors even when their structure is valid; human review remains necessary.

7.5 Future Work

7.5.1 Point-in-Time Data and Dataset Publication

The highest priority is to establish point-in-time correctness. Fundamental records should store both the reporting period and the date on which the filing became publicly available. Historical analysis should retrieve only records available by the candidate date. Index membership should be reconstructed by effective date and should retain removed, suspended, and delisted securities where data permit. News records should preserve retrieval time, publication-time corrections, source identity, and association provenance.

The dataset can then be released as versioned snapshots rather than as an evolving database export. Each version should include a data dictionary, schema and unit definitions, source and licensing notes, coverage statistics, missing-value rules, deduplication procedures, checksums, and reproducible preparation scripts. A fixed benchmark split would make comparisons between future Stockrium versions and external methods more credible.

7.5.2 More Realistic and Reproducible Evaluation

Every experiment should store an immutable manifest containing the security universe, exact interval, effective parameters, code revision, model and prompt versions, data snapshot, transaction-cost assumptions, and random seeds. Model outputs can be cached for deterministic replay. Statistical analysis should use prespecified hypotheses, multiple-comparison correction, and bootstrap or hierarchical uncertainty intervals where conventional asymptotic tests are unreliable.

The simulator should be extended with a shared capital ledger, position sizing, simultaneous-position limits, cash constraints, and portfolio-level risk measures. Conservative gap-aware fills, Vietnamese price-limit rules, liquidity filters, spread and slippage estimates, and, where available, intraday quote data would improve execution realism. Longer out-of-sample periods and historically reconstructed universes should be used to obtain meaningful bull, sideways, and bear-market samples. Forward outcomes should also be recorded for rejected candidates so that confidence calibration and the value of the Buy gate can be evaluated without selection truncation.

7.5.3 Analytical and Agentic Improvements

Future versions should strengthen evidence grounding. Each specialist output can include references to the exact candles, financial periods, ratios, and articles used in its reasoning. The aggregator can then verify evidence availability and reject unsupported claims before generating a recommendation. Confidence should be calibrated against

observed forward outcomes rather than interpreted only as a weighted internal score.

Additional research can compare the current agent decomposition with deterministic ranking models, classical machine-learning methods, time-series models, and Vietnamese finance-specific language models under the same point-in-time protocol. Model and prompt ablations should identify whether observed changes arise from new information, the specialist decomposition, or language-model variability. Human expert review can be incorporated to assess factual grounding, explanation quality, usefulness, and disagreement between analysts and automated outputs.

7.5.4 System Quality and User Experience

The software platform would benefit from automated unit, integration, contract, end-to-end, security, and performance testing in continuous integration. Long-running collection, analysis, and backtesting jobs should use durable background queues with progress reporting, retry policies, idempotency keys, and operational monitoring. Secrets management, audit logging, rate limiting, dependency scanning, backup restoration, and incident procedures should be strengthened for a production deployment.

User-facing research should examine whether investors correctly understand confidence values, explanations, risk levels, and the distinction between decision support and advice. Accessibility, localization quality, onboarding, notification controls, and explanation design can be evaluated with representative Vietnamese retail investors. Portfolio tracking and paper-trading features could support learning and retrospective review without enabling automatic execution. Any later brokerage integration would require separate legal, security, suitability, and operational analysis beyond the scope of the present thesis.

7.6 Closing Remarks

Stockrium demonstrates that heterogeneous Vietnamese market evidence can be organized into a coherent mobile and research platform and processed through an inspectable combination of deterministic analytics and specialized language-model agents. Its strongest result is not a claim of superior returns, but the construction of an end-to-end environment in which analytical assumptions, decisions, baselines, and limitations can be examined together.

The evaluation also shows why caution is essential in financial AI. A system may generate structured explanations and reduce exposure while still failing to outperform simple alternatives consistently. By reporting this distinction and identifying the data and experimental controls still required, the thesis establishes a practical foundation for future work on more reproducible, realistic, and trustworthy decision support for the Vietnamese stock market.

REFERENCES

BIBLIOGRAPHY

- [1] Viet Nam Securities Depository and Clearing Corporation, “Information on the number of domestic and foreign trading accounts in December 2025, Fourth quarter of 2025 (as of 31/12/2025),” Jan. 2026. [Online]. Available: <https://www.vsd.vn/en/ad/190896>.
- [2] VinaCapital, “Vietnam’s Resilient Stock Market,” Insights Report, 2024. [Online]. Available: <https://vinacapital.com/wp-content/uploads/2024/07/VinaCapital-Insights-Vietnams-Resilient-Stock-Market.pdf>.
- [3] B. M. Barber and T. Odean, “The behavior of individual investors,” in *Handbook of the Economics of Finance*, vol. 2B, G. M. Constantinides, M. Harris, and R. M. Stulz, Eds. Elsevier, 2013, pp. 1533–1570.
- [4] S. Wu et al., “BloombergGPT: A large language model for finance,” arXiv preprint arXiv:2303.17564, 2023.
- [5] H. Yang, X.-Y. Liu, and C. D. Wang, “FinGPT: Open-source financial large language models,” arXiv preprint arXiv:2306.06031, 2023.
- [6] Y. Xiao, E. Sun, D. Luo, and W. Wang, “TradingAgents: Multi-agents LLM financial trading framework,” arXiv preprint arXiv:2412.20138, 2024.
- [7] Y. Yu, H. Li, Z. Chen, Y. Jiang, Y. Li, D. Zhang, R. Liu, J. W. Suchow, and K. Khashanah, “FinMem: A performance-enhanced LLM trading agent with layered memory and character design,” arXiv preprint arXiv:2311.13743, 2023.
- [8] D. Araci, “FinBERT: Financial sentiment analysis with pre-trained language models,” arXiv preprint arXiv:1908.10063, 2019.
- [9] S. Halder, “FinBERT-LSTM: Deep learning based stock price prediction using news sentiment analysis,” arXiv preprint arXiv:2211.07392, 2022.
- [10] N. S. Tung, N. N. Long, T. Tran, N. T. Thao, D. T. T. Phuong, and T. Nguyen, “Stock article title sentiment-based classification using PhoBERT,” in *Proceedings of the 2nd International Conference on Human-centered Artificial Intelligence*, CEUR Workshop Proceedings, vol. 3026, pp. 225–233, 2021. [Online]. Available: <https://ceur-ws.org/Vol-3026/paper25.pdf>.
- [11] L. T. Vu, D. N. Pham, H. T. Kieu, and T. T. T. Pham, “Sentiments extracted from news and stock market reactions in Vietnam,” *International Journal of Financial Studies*, vol. 11, no. 3, art. 101, 2023, doi: 10.3390/ijfs11030101.
- [12] C.-D. Truong, D.-Q. Tran, V.-D. Nguyen, H.-T. Tran, and T.-D. Hoang, “Predicting Vietnamese stock market using the variants of LSTM architecture,” in *Nature of Computation and Communication*, Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering. Springer, 2022, doi: 10.1007/978-3-030-92942-8_11.

- [13] T. Phuoc, P. T. K. Anh, P. H. Tam, and C. V. Nguyen, “Applying machine learning algorithms to predict the stock price trend in the stock market: The case of Vietnam,” *Humanities and Social Sciences Communications*, vol. 11, art. 393, 2024, doi: 10.1057/s41599-024-02807-x.
- [14] H. H. Son, N. T. Phuong, N. T. Linh, and P. T. Loan, “A comprehensive comparison of deep learning models for forecasting developing market indices: Evidence from the Vietnamese stock market,” *Procedia Computer Science*, vol. 272, pp. 210–217, 2025, doi: 10.1016/j.procs.2025.10.198.
- [15] J. J. Murphy, *Technical Analysis of the Financial Markets: A Comprehensive Guide to Trading Methods and Applications*. New York, NY, USA: New York Institute of Finance, 1999.
- [16] S. H. Penman, *Financial Statement Analysis and Security Valuation*, 5th ed. New York, NY, USA: McGraw-Hill/Irwin, 2013.
- [17] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [18] T. B. Brown et al., “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
- [19] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proceedings of the International Conference on Learning Representations*, 2023.
- [20] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Chichester, U.K.: Wiley, 2009.
- [21] LangChain, “LangGraph Graph API overview,” 2026. [Online]. Available: <https://docs.langchain.com/oss/python/langgraph/graph-api>. Accessed: Jul. 14, 2026.
- [22] W. F. Sharpe, “The Sharpe ratio,” *The Journal of Portfolio Management*, vol. 21, no. 1, pp. 49–58, 1994.
- [23] D. H. Bailey, J. M. Borwein, M. Lopez de Prado, and Q. J. Zhu, “The probability of backtest overfitting,” *The Journal of Computational Finance*, vol. 20, no. 4, pp. 39–69, 2017, doi: 10.21314/JCF.2016.322.
- [24] Meta Platforms, Inc., “React Native: Learn once, write anywhere,” 2026. [Online]. Available: <https://reactnative.dev/>. Accessed: Jul. 14, 2026.
- [25] Expo, “Expo Router,” Expo SDK 54 Documentation, 2026. [Online]. Available: <https://docs.expo.dev/versions/v54.0.0/sdk/router/>. Accessed: Jul. 14, 2026.
- [26] TradingView, “Lightweight Charts documentation,” 2026. [Online]. Available: <https://tradingview.github.io/lightweight-charts/>. Accessed: Jul. 14, 2026.
- [27] Shopify, “React Native Skia documentation,” 2026. [Online]. Available: <https://shopify.github.io/react-native-skia/>. Accessed: Jul. 14, 2026.
- [28] FastAPI, “FastAPI features,” 2026. [Online]. Available: <https://fastapi.tiangolo.com/features/>. Accessed: Jul. 14, 2026.

- [29] Pydantic Services Inc., “Models,” Pydantic Documentation, 2026. [Online]. Available: <https://docs.pydantic.dev/latest/concepts/models/>. Accessed: Jul. 14, 2026.
- [30] Pydantic Services Inc., “Settings management,” Pydantic Documentation, 2026. [Online]. Available: https://docs.pydantic.dev/latest/concepts/pydantic_settings/. Accessed: Jul. 14, 2026.
- [31] OpenAI, “Structured model outputs,” OpenAI API Documentation, 2026. [Online]. Available: <https://developers.openai.com/api/docs/guides/structured-outputs>. Accessed: Jul. 14, 2026.
- [32] LangChain, “Persistence,” LangGraph Documentation, 2026. [Online]. Available: <https://docs.langchain.com/oss/python/langgraph/persistence>. Accessed: Jul. 14, 2026.
- [33] Supabase, “Database overview,” 2026. [Online]. Available: <https://supabase.com/docs/guides/database/overview>. Accessed: Jul. 14, 2026.
- [34] Redis, “Redis data types,” 2026. [Online]. Available: <https://redis.io/docs/latest/develop/data-types/>. Accessed: Jul. 14, 2026.
- [35] Expo, “SecureStore,” Expo Documentation, 2026. [Online]. Available: <https://docs.expo.dev/versions/latest/sdk/securestore/>. Accessed: Jul. 14, 2026.
- [36] Selenium Project, “The Selenium Browser Automation Project,” 2025. [Online]. Available: <https://www.selenium.dev/documentation/>. Accessed: Jul. 14, 2026.
- [37] The pandas Development Team, “IO tools,” pandas Documentation, 2026. [Online]. Available: https://pandas.pydata.org/docs/user_guide/io.html. Accessed: Jul. 14, 2026.
- [38] TA-Lib, “TA-Lib functions,” 2026. [Online]. Available: <https://ta-lib.org/functions/>. Accessed: Jul. 14, 2026.
- [39] Resend, “API introduction,” 2026. [Online]. Available: <https://resend.com/docs/api-reference/introduction>. Accessed: Jul. 14, 2026.
- [40] Railway, “Services,” Railway Documentation, 2026. [Online]. Available: <https://docs.railway.com/services>. Accessed: Jul. 14, 2026.
- [41] Expo, “EAS Build,” 2026. [Online]. Available: <https://docs.expo.dev/build/>. Accessed: Jul. 14, 2026.